

PwC Technology Interview Preparation

Modern Data Systems, SQL, Databases, APIs & Data Integration

A complete, interview-oriented study guide built on the seven modules of the Tekstac *Modern Data Systems* course. Each session is re-taught in simple language, with diagrams, PostgreSQL examples that were actually run, interview questions with model answers, practice sets and a final revision kit.

PART I	Introduction to Modern Data Systems
PART II	Data Modeling Fundamentals
PART III	Basics of SQL (Foundational) · SQL Interview Patterns
PART IV	Cloud Databases & Storage Systems
PART V	Data Integration & APIs
PART VI	Integration Patterns in the Enterprise
PART VII	Modern Trends & Future Directions
FINAL	Revision Kit · 100 Questions · SQL Bank · One-Day Sheet

191

INTERVIEW Q&AS

102

SQL EXAMPLES RUN ON
POSTGRESQL

16

DIAGRAMS

55

PRACTICE QUESTIONS

Contents

START HERE	How to Use This Handbook	3
	What Is Inside	3
	How Every Session Is Built	3
	Labels You Will See	4
	Study Plans	4
	About the Source Material	5
START HERE	Course Coverage Map	6
START HERE	PwC Interview Orientation	10
	Four Kinds of Information — Keep Them Separate	10
	A. Common Software and Database Questions	10
	B. Reported Candidate Experiences	10
	C. Likely Topics for Technology and Data Roles	11
	D. Recommended Preparation	12
PART I • MODULE 1	Introduction to Modern Data Systems	13
	Session 1.0 — Recommended Prerequisite: Data and Database Basics	14
	Session 1.1 — Modern Data Systems & Integration (Part 1): What a Data System Is	17
	Session 1.2 — Modern Data Systems & Integration (Part 2): How Data Moves Between Systems	22
	Session 1.3 — Modern Data Systems & Integration (Part 3): Processing, OLTP vs OLAP, and Why One Database Is Not Enough	25
	Session 1.4 — Module 1 Summary & Rapid Revision	31
PART II • MODULE 2	Data Modeling Fundamentals	32
	Session 2.1 — What Is Data Modeling?	33
	Session 2.2 — Entities, Attributes and Relationships (ER Modeling)	36
	Session 2.3 — Keys and Constraints	39
	Session 2.4 — The Relational Model, Schema and Table Design	45
	Session 2.5 — Normalization	50
	Session 2.6 — Denormalization: When and Why	57
	Session 2.7 — Data Warehousing and Dimensional Modeling	61
	Session 2.8 — NoSQL Data Modeling	69
	Session 2.9 — Module 2 Summary & Rapid Revision	75
PART III • MODULE 3	Basics of SQL (Foundational)	76
	Session 3.0 — Meet the Sample Database	77
	Session 3.1 — PostgreSQL Basics (Part 1)	80
	Session 3.2 — PostgreSQL Basics (Part 2)	89
	Session 3.3 — DDL: Data Definition Language	100
	Session 3.4 — DML: Data Manipulation Language (plus DCL and TCL)	105
	Session 3.5 — Grouping and Aggregation: GROUP BY and HAVING	113
	Session 3.6 — Joins	119

START HERE

How to Use This Handbook

This handbook replaces the course videos for interview preparation. It follows the seven modules of the Tekstac *Modern Data Systems* course, re-teaches every topic in plain English, and adds what interviewers usually ask on top of the course.

You do not need to watch the videos first. Read a session, run through its interview questions out loud, try the practice set, and tick the checklist at the end of the module.

What Is Inside

Part	Module	What you will be able to do after it
I	Introduction to Modern Data Systems	Explain how an app stores, moves and analyses data; OLTP vs OLAP; batch vs real-time
II	Data Modeling Fundamentals	Design tables, choose keys, normalize to 3NF, design a star schema, model documents
III	Basics of SQL (Foundational)	Write SELECT, joins, GROUP BY, subqueries, CTEs, window functions; explain indexes and transactions; solve 22 classic SQL interview patterns
IV	Cloud Databases & Storage Systems	Choose SQL vs NoSQL, explain CAP, use MongoDB CRUD, compare warehouse, lake and lakehouse
V	Data Integration & APIs	Explain a REST request end to end, HTTP methods and status codes, auth, retries, idempotency
VI	Integration Patterns in the Enterprise	Compare APIs vs queues, pub/sub, event-driven design, ETL vs ELT, batch vs streaming
VII	Modern Trends & Future Directions	Discuss data mesh, serverless and AI-ready data critically, with trade-offs
Final	Final Interview Revision	100 key questions, a 50-question SQL bank, scenario questions, rapid fire, a one-day revision sheet and an answering strategy

How Every Session Is Built

Every session follows the same rhythm, so you always know where to look:

1. **The idea in simple words**, then *why it exists*, *how it works*, an example and where it is used in real systems.
2. **Diagrams and tables** wherever a picture is faster than a paragraph.
3. **Code with real output**. Every SQL query marked with an *Output* panel was actually run on PostgreSQL while this PDF was built. Important queries are followed by a **Line by Line** explanation.
4. **How I Would Explain This in an Interview**: a natural 30–60 second spoken answer.
5. **Common Traps & Mistakes**: the confusions that cost candidates marks.
6. **Interview Questions** in four groups (Basic, Intermediate, Scenario-based, Follow-up/Trap), each with a short model answer.
7. **Practice Questions** at three levels, with the answers kept in a separate **Answer / Explanation** box so you can test yourself first.
8. At the end of each module: a **Module Summary** and a **Rapid Revision Checklist**.

Labels You Will See

Label	Meaning
DEFINITION	"What is X?" questions. Answer in one or two clean sentences.
WHY	"Why do we need X?" Talk about the problem it solves.
HOW	"How does X work?" Walk through the mechanism step by step.
COMPARISON	"X vs Y". Use a small table in your head: purpose, how, when to use.
SCENARIO	A real situation. Ask a clarifying question, then propose and justify.
SQL PROBLEM	Write a query. Think aloud, start simple, then refine.
DESIGN QUESTION	Design a small system or schema. Cover data, flow and trade-offs.
TRAP QUESTION	A question designed to catch a common misunderstanding.
EASY MEDIUM HARD	Difficulty of a SQL pattern or practice question.
HIGH PRIORITY MEDIUM PRIORITY LOWER PRIORITY	How much interview time a topic usually gets. Spend your time accordingly.
INTERVIEW EXTENSION	Useful knowledge that goes beyond the course content.

INTERVIEW EXTENSION BOXES

Purple boxes and the **INTERVIEW EXTENSION** label mark material that is **not** part of the original course but that interviewers commonly expect (for example, isolation levels, SCD types or idempotency keys). Everything else follows the course topics.

RECOMMENDED PREREQUISITE BOXES

Grey boxes mark background knowledge you need before a topic makes sense, such as "what is a database?" before data modeling.

Study Plans

Pick the plan that matches the time you have. Sessions tagged **HIGH PRIORITY** come first in every plan.

If you have...	Do this
1 day (about 8 hours)	One-Day Revision Sheet (1 h) → Module 3 Sessions 3.1–3.8 skim plus SQL Patterns 1–12 (3 h) → Sessions 2.3–2.5 keys and normalization (1 h) → Sessions 4.2–4.5 SQL vs NoSQL and MongoDB (1 h) → Sessions 5.2–5.5 REST and HTTP (45 min) → Sessions 6.2, 6.3 and 6.6 (30 min) → Module 7 summary (15 min) → 100 Most Important Questions, read aloud (30 min)
3 days	Day 1: Modules 1–2 plus their practice sets. Day 2: all of Module 3 with every SQL example typed out once. Day 3: Modules 4–7 summaries and questions, the SQL Question Bank, then the One-Day Sheet in the evening.
7 days	One module per day (Module 3 gets two days), finishing every practice set. Day 7: the Final Interview Revision part, a mock interview with a friend, and your project story.

THE SINGLE BEST HABIT

Say answers **out loud**. Reading an answer and being able to say it clearly are different skills. The "How I Would Explain This" boxes exist for that reason.

About the Source Material

This handbook was built from the module and topic structure of the Tekstac *Modern Data Systems* course as described in the preparation brief: seven modules and their listed sessions, quizzes, practice programs and subtopics. The course screenshots themselves were not available when this edition was produced, so:

- Session titles that were named explicitly (for example *PostgreSQL Basics – Part 1, DDL, DML and Window Functions & Indexing*) are used as they are.
- Where a module's sessions were described only by topic, the session titles were rebuilt from those topics and are marked † in the Course Coverage Map. The topics themselves are all covered; only the exact titles or the split into parts may differ from the videos.
- Nothing is presented as course content unless it was part of the course topic list. Extra material is labelled

INTERVIEW EXTENSION .

If your course shows a session that is not listed in the coverage map, look it up in the index-like map on the next pages by topic. Every topic from the brief is covered somewhere.

START HERE

Course Coverage Map

Use this map to check that every course session and topic is covered, and to jump straight to it. Titles marked † were reconstructed from the module's topic list (see "About the Source Material").

MODULE 1 – INTRODUCTION TO MODERN DATA SYSTEMS

Course session / topic	Covered in	Page
(Prerequisite) Data, databases and DBMS basics	Session 1.0 INTERVIEW EXTENSION	p. 14
Modern Data Systems & Integration — Part 1: data systems, how applications store data †	Session 1.1	p. 17
Modern Data Systems & Integration — Part 2: how data moves, data integration, APIs, backend systems †	Session 1.2	p. 22
Modern Data Systems & Integration — Part 3: batch vs real-time, OLTP vs OLAP, operational vs analytical data, why many data technologies †	Session 1.3	p. 25
Module 1 quiz concepts, summary and checklist	Session 1.4	p. 31

MODULE 2 – DATA MODELING FUNDAMENTALS

Course session / topic	Covered in	Page
Introduction to data modeling (conceptual, logical, physical) †	Session 2.1	p. 33
Entities, attributes and relationships; ER concepts †	Session 2.2	p. 36
Primary keys, foreign keys and constraints †	Session 2.3	p. 39
Relational model, schema, table design; one-to-one, one-to-many, many-to-many †	Session 2.4	p. 45
Normalization: functional dependency, anomalies, 1NF, 2NF, 3NF, BCNF †	Session 2.5	p. 50
Denormalization: when to normalize, when to denormalize †	Session 2.6	p. 57
Data warehousing: fact and dimension tables, star vs snowflake, OLTP vs OLAP †	Session 2.7	p. 61
NoSQL modeling: schema flexibility, embedding vs referencing, access patterns †	Session 2.8	p. 69
Module 2 quiz concepts, summary and checklist	Session 2.9	p. 75

MODULE 3 – BASICS OF SQL (FOUNDATIONAL)

Course session / topic	Covered in	Page
(Prerequisite) The sample database used in every example	Session 3.0	p. 77
PostgreSQL Basics — Part 1 (SELECT, FROM, WHERE, DISTINCT, ORDER BY, LIMIT, OFFSET, aliases, data types)	Session 3.1	p. 80
PostgreSQL Basics — Part 2 (operators, NULL, aggregate, string, date and numeric functions, CASE)	Session 3.2	p. 89
DDL (CREATE, ALTER, DROP, TRUNCATE, constraints)	Session 3.3	p. 100
DML (INSERT, UPDATE, DELETE) plus DCL and TCL	Session 3.4	p. 105
Grouping and aggregation: GROUP BY, HAVING, WHERE vs HAVING †	Session 3.5	p. 113
Joins: inner, left, right, full, cross, self; referential integrity †	Session 3.6	p. 119
Subqueries, CTEs (including recursive) †	Session 3.7	p.
Window Functions & Indexing — window functions	Session 3.8	p.
Window Functions & Indexing — indexing	Session 3.9	p.
Transactions, ACID, isolation and read anomalies †	Session 3.10	p.
Practice programs: SQL interview patterns (22 patterns)	Session 3.11	p.
Module 3 quiz concepts, summary and checklist	Session 3.12	p.

MODULE 4 – CLOUD DATABASES & STORAGE SYSTEMS

Course session / topic	Covered in	Page
Cloud databases: managed, RDS / Cloud SQL, serverless, analytical; scaling, availability, backups, replication, failover †	Session 4.1	p.
SQL vs NoSQL: relational, document, key-value, graph, column-family, columnar †	Session 4.2	p.
NoSQL fundamentals and the CAP theorem; eventual consistency †	Session 4.3	p.
MongoDB: database, collection, document, field, BSON †	Session 4.4	p.
MongoDB CRUD: insert, find, update, delete, updateMany, deleteMany †	Session 4.5	p.
MongoDB indexing, aggregation, embedding vs referencing †	Session 4.6	p.
Data storage: database, warehouse, lake, lakehouse; structured, semi-structured, unstructured data †	Session 4.7	p.
Module 4 quiz concepts, summary and checklist	Session 4.8	p.

MODULE 5 – DATA INTEGRATION & APIS

Course session / topic	Covered in	Page
What an API is; client-server; request/response †	Session 5.1	p.
HTTP and REST: methods, status codes, JSON †	Session 5.2	p.
Designing and consuming REST APIs: pagination, rate limiting, versioning †	Session 5.3	p.
Authentication and authorization †	Session 5.4	p.
API integration and external APIs: sync vs async, failures, retries, idempotency †	Session 5.5	p.
Data integration †	Session 5.6	p.
Module 5 quiz concepts, summary and checklist	Session 5.7	p.

MODULE 6 – INTEGRATION PATTERNS IN THE ENTERPRISE

Course session / topic	Covered in	Page
Enterprise integration and system-to-system communication; integration patterns †	Session 6.1	p.
Synchronous vs asynchronous; API vs message queue †	Session 6.2	p.
Messaging: queues, publish/subscribe, producers, consumers †	Session 6.3	p.
Event-driven architecture †	Session 6.4	p.
Data pipelines; batch vs real-time processing †	Session 6.5	p.
ETL vs ELT †	Session 6.6	p.
Streaming †	Session 6.7	p.
Salesforce and cloud data warehouse integration †	Session 6.8	p.
Choosing an integration pattern INTERVIEW EXTENSION	Session 6.9	p.
Module 6 quiz concepts, summary and checklist	Session 6.10	p.

MODULE 7 – MODERN TRENDS & FUTURE DIRECTIONS

Course session / topic	Covered in	Page
Data mesh †	Session 7.1	p.
Serverless †	Session 7.2	p.
AI and data management †	Session 7.3	p.
Modern data systems: real-time analytics, event-driven, cloud-native, distributed, data platforms †	Session 7.4	p.
Thinking critically about trends INTERVIEW EXTENSION	Session 7.5	p.
Module 7 summary and checklist	Session 7.6	p.

FINAL INTERVIEW REVISION

Section	Page
100 Most Important Questions	p.
SQL Question Bank — Top 50 SQL Questions	p.
Top Database Questions	p.
Top Scenario Questions	p.
Rapid-Fire Questions	p.
Project Connection	p.
One-Day Revision Sheet	p.
Interview Answering Strategy	p.
Glossary · Coverage Verification	p.

QUIZZES AND PRACTICE PROGRAMS

The course modules include quizzes and practice programs. Their concepts are covered by the **Interview Questions** and **Practice Questions** boxes in each session, by Session 3.11 (SQL patterns, which mirror typical SQL practice programs) and by the SQL Question Bank in the final part.

START HERE

PwC Interview Orientation

This chapter explains what is publicly known about PwC technology interviews in India, what is only reported by candidates, and how to prepare. It does not invent "official PwC questions".

Four Kinds of Information — Keep Them Separate

Category	What it means	How much to trust it
A. Common software and database questions	Questions asked across the industry for any software or data role (joins, normalization, ACID, REST...).	Very reliable. These are the core of every technical interview.
B. Reported candidate experiences REPORTED	What individual candidates wrote about their own PwC interviews on public sites.	Useful signal, but each report is one person, one year, one role, one campus.
C. Likely topics for the role LIKELY	Topics that fit the job description of technology, data and consulting roles.	Reasonable inference, not a promise.
D. Recommended preparation RECOMMENDED	This handbook's advice on what to study and how.	Our judgement, based on A–C.

DO NOT OVER-TRUST "PWC QUESTION LISTS"

Websites that publish "PwC interview questions" are usually generic question banks. PwC has many business units (Advisory, Tax, Assurance, the Acceleration Centers) and many roles, and the process changes from year to year. Prepare the fundamentals well and you will be ready for any variation.

A. Common Software and Database Questions

These appear in almost every entry-level technical interview, including consulting firms. They are the backbone of this handbook:

- **SQL:** write a query with a join, GROUP BY with HAVING, the second-highest salary, duplicates, top N per group, a window function.
- **DBMS:** primary vs foreign key, normalization (1NF–3NF), ACID, transactions, indexes, DELETE vs TRUNCATE vs DROP.
- **Data modeling:** design tables for a small business problem (library, hospital, e-commerce).
- **APIs:** what is REST, HTTP methods, status codes, PUT vs PATCH, authentication vs authorization.
- **Programming and OOP:** the four pillars of OOP, one language in depth, basic data structures.
- **Your project:** architecture, your role, a problem you solved, what you would improve.

B. Reported Candidate Experiences

The following themes come from public interview write-ups (sources at the end of this chapter). They were collected from search summaries in October 2026; individual details vary a lot.

Theme	What candidates reported REPORTED
Process	An online assessment, then a technical interview, then a managerial or partner round, then HR. Some write-ups describe a "Launchpad" programme and a hackathon before the interviews.
Online assessment	Aptitude sections (quantitative, logical reasoning, verbal/English). Some reports also mention technical multiple-choice questions on OOP, DBMS, networking and data structures.
Technical round	Often around 30 minutes, frequently resume- and project-based. SQL joins and their types, normalization, ACID properties and basic DBMS terms appear repeatedly.
Practical SQL	One report describes being asked to create a table, alter it to add columns, create a table from existing tables and write a GROUP BY query.
Programming	Easy-to-medium problems: sort an array without a built-in sort, swap two numbers without a third variable, list vs tuple and dictionaries in Python, OOP concepts.
APIs and backend	Reports for software roles mention HTTP methods, status codes, query vs path parameters, PUT vs PATCH, idempotency, REST and SOLID principles.
Data roles	Experienced data-engineering candidates report scenario-based questions on SQL, Spark, Databricks and Power BI.
Final rounds	The partner or managerial round may be technical or about fit and attitude. HR questions include "Why PwC?".

C. Likely Topics for Technology and Data Roles

Based on the role descriptions and the reports above, a technology or data candidate should expect **LIKELY** :

1. **SQL writing on the spot**, from simple filters to joins, grouping, subqueries and window functions.
2. **DBMS concepts** asked as "what is" and "difference between" questions.
3. **Designing a small schema** and normalizing it.
4. **SQL vs NoSQL** and when to use MongoDB.
5. **How a web request flows** through frontend, API and database.
6. **Cloud and data basics**: managed databases, warehouse vs lake, ETL vs ELT.
7. **Your project**, explained with clear architecture and your contribution.
8. **Consulting behaviours**: communication, teamwork, client focus and learning attitude.

PWC'S OWN FRAMEWORK FOR BEHAVIOURAL QUESTIONS

PwC publicly describes the **PwC Professional** framework with five dimensions: **Whole leadership, Business acumen, Technical and digital, Global and inclusive**, and **Relationships**. Prepare one short story for each (use STAR: Situation, Task, Action, Result). For example: a project where you learned a new tool quickly (*Technical and digital*), or a team conflict you resolved (*Relationships*).

D. Recommended Preparation

Priority	Study	Where in this handbook
1	SQL fundamentals and the 22 interview patterns	Module 3, Session 3.11, SQL Question Bank
2	Keys, normalization, ACID, indexes, transactions	Sessions 2.3–2.5, 3.9, 3.10
3	REST, HTTP, authentication, request flow	Module 5
4	SQL vs NoSQL, MongoDB basics, CAP	Sessions 4.2–4.6
5	ETL/ELT, warehouses, lakes, queues, event-driven basics	Sessions 2.7, 4.7, Module 6
6	Modern trends, at a "can discuss sensibly" level	Module 7
7	Your project story and behavioural stories	Project Connection, Interview Answering Strategy

HOW PWC INTERVIEWS FEEL, ACCORDING TO MOST REPORTS

Fundamentals are tested more than tricks. Interviewers often start from your resume and keep asking "why?" until they find the edge of your understanding. Being honest ("I have not used that, but I understand it as...") is better than guessing.

SOURCES CONSULTED (VIA WEB SEARCH, OCTOBER 2026)

- PwC Acceleration Centers India — *Entry Level Campus Launchpad Program* and *AC India Assurance Launchpad Program* pages (jobs-ta.pwc.com). The programme page describes hands-on training in areas such as IT, SQL, data engineering and Python.
- PwC India — *Want to be a PwCite?* campus careers page (pwc.in): register → pre-placement talk → assessment → interview → final selection.
- PwC — *The PwC Professional / The skills we look for* (pwc.com).
- GeeksforGeeks — PwC India interview experiences for Technology Consultant, Technology Consulting Intern (2023) and Associate Engineer (on-campus) roles; *PwC Recruitment Process*.
- Naukri Code360 — PwC AC India interview experience (on-campus, October 2025).
- Glassdoor — PwC Associate Software Engineer interview questions (India).
- Medium — "PwC India Interview Experience" (S. Palit), a data-engineering interview.
- InterviewBit and PrepInsta — PwC interview process summaries.

These sources describe individual experiences and change over time. Treat them as signals, not as a syllabus.

PART I · MODULE 1

Introduction to Modern Data Systems

This module gives you the big picture: what a "data system" is, how a modern application stores and moves data, and why companies end up using several kinds of databases at once. The rest of the handbook zooms into each piece.

COURSE COVERAGE

- ✓ **Modern Data Systems & Integration — Part 1** †: what a data system is, how modern applications store data, applications and databases, backend systems → Session 1.1
- ✓ **Modern Data Systems & Integration — Part 2** †: how data moves between systems, data integration, APIs, data processing → Session 1.2
- ✓ **Modern Data Systems & Integration — Part 3** †: batch vs real-time, transactional vs analytical systems (OLTP vs OLAP), operational data, why many data technologies → Session 1.3
- ✓ Module quiz concepts → interview questions in every session and Session 1.4
- ✓ *Recommended prerequisite (not course content):* data, database and DBMS basics → Session 1.0

Interview priority	Why
MEDIUM PRIORITY overall	Rarely asked as "theory", but the vocabulary (OLTP, OLAP, batch, streaming, integration) appears in many answers.
HIGH PRIORITY OLTP vs OLAP, batch vs streaming	Classic comparison questions.
HIGH PRIORITY "Explain how your app works end to end"	Almost every project discussion turns into this.

HOW THIS MODULE IS TESTED

- **DEFINITION** "What is a data system / data integration?"
- **COMPARISON** "OLTP vs OLAP", "batch vs real-time"
- **HOW** "Walk me through what happens when a user places an order."
- **DESIGN QUESTION** "Which databases would you use for a food-delivery app, and why?"

Session 1.0 — Recommended Prerequisite: Data and Database Basics

WHY THIS SESSION EXISTS

The course assumes you already know what a database is. Interviewers often start right here ("What is a DBMS?"), so a ten-minute refresher pays off. This session is an **INTERVIEW EXTENSION**, not a course video.

Data vs information

Data is raw facts: 42, Pune, 2024-03-05. **Information** is data with meaning: "Order 42 was delivered to Pune on 5 March 2024." Databases store data; reports and applications turn it into information.

Metadata is "data about data": the column name `order_date`, its type `DATE`, who created the table, when it was last updated. You will meet metadata again in Module 7, where it becomes very important for AI.

Why not just use files or Excel?

Imagine a college keeps student records in Excel files, one file per department. It works for a while, then problems appear:

Problem with files	What goes wrong	How a database solves it
Duplication	The same student's phone number is typed in three files	Store each fact once and link it with keys
Inconsistency	One file is updated, the others are not	One source of truth; constraints keep it valid
Concurrent access	Two people edit the same file and overwrite each other	Locking and transactions let many users work safely
Security	Anyone with the file sees everything	Users, roles and permissions per table or row
Crash recovery	The laptop dies mid-save and the file is corrupted	Logs and transactions recover to a consistent state
Searching	Finding "all students from Pune with CGPA > 8" is slow and manual	A query language (SQL) and indexes

Database, DBMS and RDBMS

- A **database** is an organised collection of related data, for example everything about a college's students, courses and marks.
- A **DBMS (Database Management System)** is the software that stores, retrieves and protects that data: it handles queries, users, security, backups and crashes. Examples: PostgreSQL, MySQL, Oracle, SQL Server, MongoDB.
- An **RDBMS (Relational DBMS)** stores data in **tables** (relations) made of **rows** and **columns**, and links tables using **keys**. PostgreSQL, MySQL, Oracle and SQL Server are relational. MongoDB is a DBMS but *not* relational: it stores documents.

The vocabulary of a table

STUDENTS

student_id	name	city	cgpa
1	Asha Rao	Pune	8.6
2	Vikram Shah	Delhi	7.9

Term	Meaning in this table	Other names
Table	students	relation
Row	one student, e.g. 1, Asha Rao, Pune, 8.6	record, tuple
Column	city	field, attribute
Schema	the table's design: column names, types and rules	structure
Primary key	student_id : uniquely identifies each row	—

SQL (Structured Query Language) is the standard language for working with relational databases: `SELECT` to read, `INSERT`, `UPDATE` and `DELETE` to change data, `CREATE TABLE` to define structure. Module 3 covers it in depth.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A DBMS is the software that manages a database. Instead of keeping data in loose files, it lets many users store and query data safely: it gives us a query language, enforces rules like unique IDs, controls who can see what, and recovers after a crash. A relational DBMS such as PostgreSQL stores data in tables and connects them with keys. For example, in a college system we keep students and courses in separate tables and link them through enrollments."

COMMON TRAPS & MISTAKES

- Saying "SQL is a database." SQL is a *language*; PostgreSQL and MySQL are databases (DBMS software).
- Saying "MongoDB is an RDBMS." It is a DBMS, but document-oriented, not relational.
- Mixing up a *database* (the data) with a *DBMS* (the software that manages it).

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a DBMS?

A Software that stores, retrieves and manages data in a database, handling queries, security, concurrency and recovery. Examples: PostgreSQL, MySQL, Oracle.

Q2 COMPARISON What is the difference between a DBMS and an RDBMS?

A An RDBMS is a type of DBMS that stores data in related tables and uses keys and SQL. Every RDBMS is a DBMS, but not every DBMS is relational (MongoDB and Redis are not).

Q3 DEFINITION What are a row, a column and a schema?

A A row is one record (one student). A column is one attribute (city). The schema is the design of the table: its columns, data types and rules.

INTERMEDIATE

Q4 **WHY** What are the advantages of a DBMS over a file system?

A Less duplication, consistent data, safe concurrent access, security and permissions, backup and crash recovery, and fast searching with a query language and indexes.

Q5 **COMPARISON** Data vs information vs metadata?

A Data is raw facts, information is data with context and meaning, and metadata describes the data itself (column names, types, owner, last-updated time).

SCENARIO-BASED

Q6 **SCENARIO** A small shop tracks orders in Excel and keeps losing changes when two staff edit at once. What would you suggest?

A Move the data into a relational database (even a small managed PostgreSQL). It gives concurrent access with transactions, one source of truth, backups, and the ability to build a simple app or report on top of it.

FOLLOW-UP / TRAP

Q7 **TRAP QUESTION** Is Excel a database?

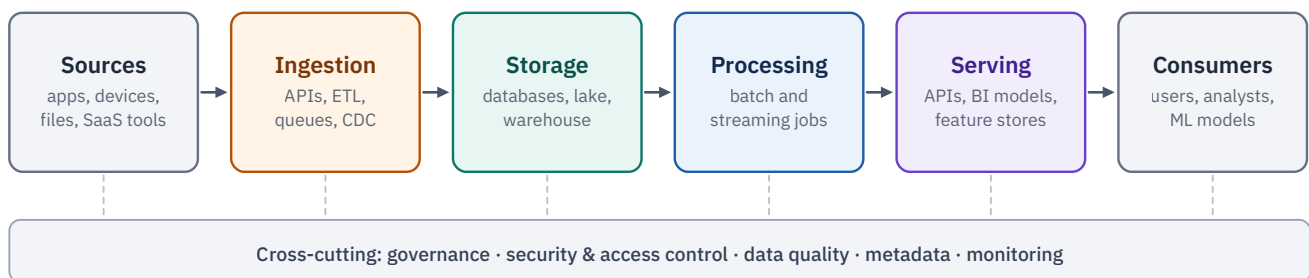
A Not really. It stores data in a grid, but it has no real concurrency control, transactions, enforced relationships or fine-grained security. It is a spreadsheet tool, fine for analysis, weak as a system of record.

Session 1.1 — Modern Data Systems & Integration (Part 1): What a Data System Is

What is a data system?

Simple meaning: a data system is *everything* an organisation uses to collect, store, process, move and serve data. It is not one database. It is the whole chain from the moment data is created (a customer taps "Pay") to the moment someone uses it (a manager reads a sales dashboard, or a model flags fraud).

Why we need the idea: in a real company, data is created in many places and used in many others. Thinking in terms of a *system* helps you see where data comes from, where it goes, what can break and who is responsible.



Data flows left to right; every stage must also be secured, monitored and documented.

Figure 1. The building blocks of a data system. Every real system has some version of each stage.

Building block	Plain-English job	Real examples
Sources	Where data is born	mobile app, website, payment gateway, IoT sensor, Excel upload, Salesforce
Ingestion	Bringing data in	REST APIs, nightly file transfers, message queues, change data capture (CDC)
Storage	Keeping data safely	PostgreSQL, MongoDB, object storage (S3), data warehouse, data lake
Processing	Cleaning, combining, calculating	SQL jobs, Spark jobs, stream processors
Serving	Making data usable	APIs, dashboards, reports, ML feature stores
Governance	Rules around all of it	access control, data quality checks, privacy, metadata catalogue, monitoring

A CITY'S WATER SYSTEM

Water is collected from rivers (**sources**), pumped through pipes (**ingestion and integration**), stored in reservoirs (**storage**), treated and filtered (**processing**), and delivered to taps (**serving**). Inspectors test quality all along the way (**governance**). If any stage fails, people at the tap notice. A data system works the same way.

How modern applications store data

Almost every app you have used (food delivery, banking, e-commerce, job portals) has the same basic shape:

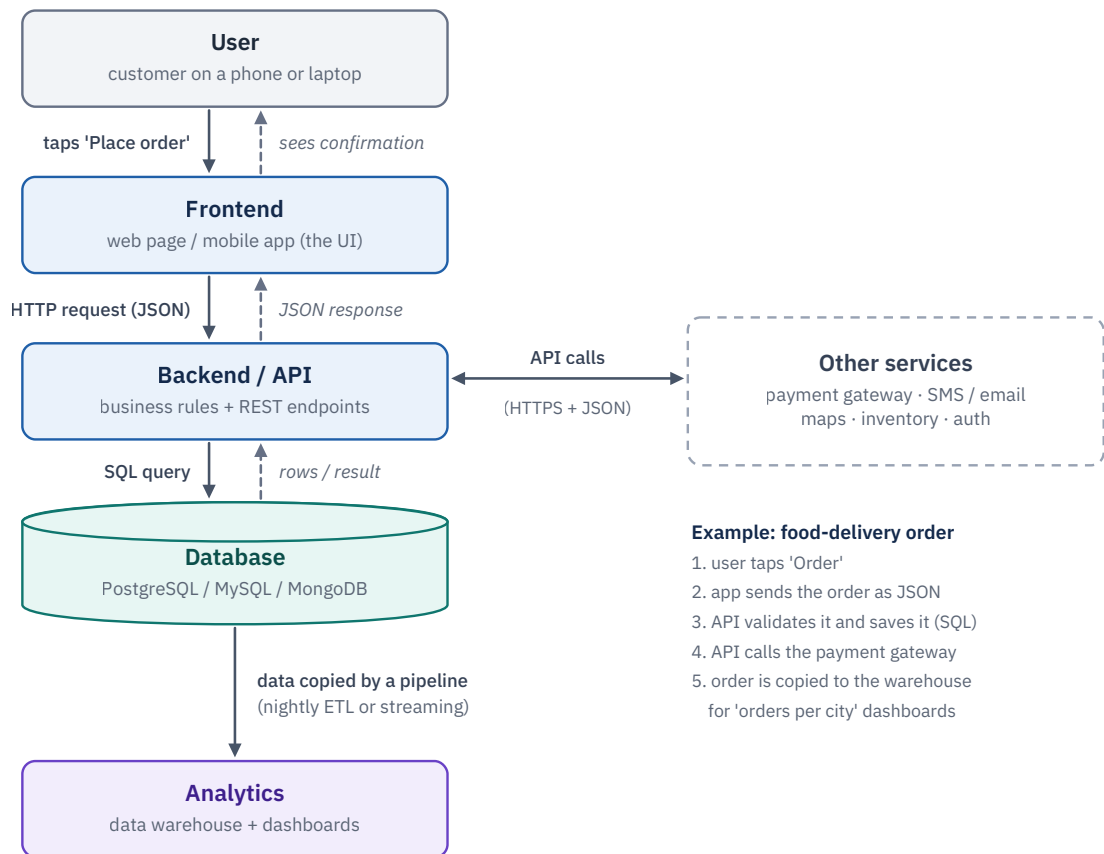


Figure 2. The typical layers of a modern application, using a food-delivery order as the example.

Layer	What it does	Example technologies
User	Interacts with the app	phone, laptop
Frontend	The screens. Collects input, shows results. Holds no permanent data of its own.	React, Angular, Android, iOS
Backend / API	The brain. Validates input, applies business rules, talks to the database and to other services. Exposes REST endpoints such as <code>POST /orders</code> .	Spring Boot, Node.js, Django, .NET
Database	The memory. Stores data permanently and safely.	PostgreSQL, MySQL, MongoDB
Other services	Specialist systems the backend calls	payment gateway, SMS/email, maps, login provider
Analytics	A copy of the data, organised for reporting and analysis	data warehouse + dashboards (Power BI, Tableau)

Why the frontend never talks to the database directly: security (you would have to put database passwords in the app), control (business rules must run on the server, where users cannot tamper with them), and flexibility (the database can change without updating every phone).

Applications and databases: how the backend talks to the database

1. The backend opens a **connection** to the database (in practice it keeps a *connection pool*, a set of ready connections, because opening one per request is slow).
2. It sends **SQL** queries, either written by hand or generated by an **ORM** (Object-Relational Mapper, such as Hibernate/JPA in Java) that turns objects into SQL.

- For changes that must happen together (create order + reduce stock), it wraps the queries in a **transaction** so either all succeed or none do.
- It turns the results into objects and then into **JSON** for the frontend.

What a backend system really does

- **Validation:** is the quantity positive? Is the email well-formed?
- **Business rules:** apply a coupon, calculate delivery charges, check stock.
- **Security:** check who the user is (authentication) and what they may do (authorization).
- **Orchestration:** call the payment gateway, then save the order, then send a notification.
- **Data access:** read and write the database efficiently and safely.

Where data lives in a modern app

One application often uses several storage technologies, each for what it does best:

Kind of data	Typical store	Why
Orders, payments, accounts	Relational database (PostgreSQL)	Needs correctness, relationships, transactions
Sessions, cached pages, OTPs	In-memory key-value store (Redis)	Extremely fast, data can expire
Product images, invoices, videos	Object storage (Amazon S3, Azure Blob)	Cheap for large files
Full-text search ("red shoes size 9")	Search engine (Elasticsearch, OpenSearch)	Ranked, fuzzy text search
Business reports	Data warehouse (Snowflake, BigQuery, Redshift)	Fast analytics over years of history
Events between services	Message broker (Kafka, RabbitMQ)	Decouples services, buffers spikes
Application logs	Log platform (ELK, CloudWatch)	Debugging and monitoring

Worked example: one food-delivery order, end to end

- You tap **Place order**. The app (frontend) sends `POST /api/orders` with JSON such as `{"restaurantId": 12, "items": [...]}`.
- The backend checks your login token, validates the items and calculates the bill.
- It calls the **payment gateway** API and waits for success.
- It saves the order in **PostgreSQL** inside a transaction.
- It publishes an `OrderPlaced` event to **Kafka**. The restaurant service, the rider-assignment service and the notification service react to it.
- Your live rider location is kept in **Redis**, because it changes every few seconds.
- Overnight (or continuously) the order is copied into the **data warehouse**, where the business sees "orders per city per hour".

MONOLITH VS MICROSERVICES (HOW ARCHITECTURE AFFECTS DATA)

A **monolith** is one application with one database. **Microservices** split the app into small services (users, orders, payments), and the usual rule is *database per service*: each service owns its data and others reach it only through its API or events. That gives teams independence, but it makes cross-service queries and consistency harder, which is exactly why integration patterns (Modules 5 and 6) matter.

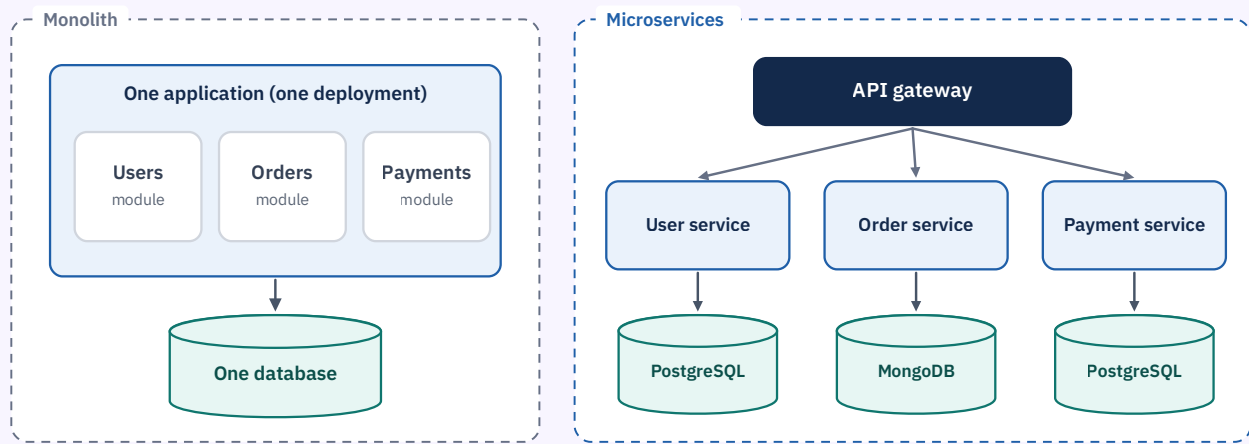


Figure 3. One shared database (monolith) vs one database per service (microservices).

USING THIS IN YOUR PROJECT STORY

If your project is a web app, describe it in these layers: "React frontend → Spring Boot REST API → PostgreSQL, with an external payment API and an export for reports." Then say which layer you built and one design decision you made in it. Interviewers love a clear map before details.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A modern data system is the whole chain that collects, stores, processes and serves data, not just one database. A typical application has a frontend for the user, a backend API that holds the business logic, and a database that stores data safely. The backend also calls other services, like a payment gateway, and the data is copied to an analytics system for reporting. For example, in a food-delivery app the order is saved in PostgreSQL, the rider's live location sits in Redis, and the warehouse powers the city-wise sales dashboard."

COMMON TRAPS & MISTAKES

- Describing the system as "frontend + database". The backend/API layer is essential; the frontend should never connect to the database directly.
- Assuming one database does everything. Real systems mix stores (relational, cache, search, object storage, warehouse).
- Forgetting governance: security, quality and monitoring are part of the system, not extras.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a data system?

A The complete set of components that collect, store, process, move and serve an organisation's data: sources, ingestion, storage, processing, serving and governance.

Q2 **DEFINITION** What does the backend of an application do?

A It validates input, applies business rules, handles authentication and authorization, talks to the database and other services, and returns responses (usually JSON) to the frontend.

Q3 WHY Why shouldn't a mobile app connect directly to the database?

- A** It would expose database credentials, let users bypass business rules, and make every schema change break installed apps. The API in the middle protects and controls access.

INTERMEDIATE**Q4 HOW What happens between a backend and a database when a request comes in?**

- A** The backend takes a connection from its pool, runs SQL (often generated by an ORM), wraps related changes in a transaction, maps rows into objects and returns JSON.

Q5 COMPARISON Where would you store product images vs order details?

- A** Images in object storage (cheap, built for large files) with only the file URL in the database; order details in a relational database, because they need structure, relationships and transactions.

Q6 DEFINITION What is a connection pool and why is it used?

- A** A set of already-open database connections that requests borrow and return. Opening a new connection for every request is slow and can overload the database.

SCENARIO-BASED**Q7 DESIGN QUESTION Sketch the architecture of a job portal.**

- A** Web/mobile frontend → REST API (job search, apply, recruiter dashboards) → PostgreSQL for users, jobs and applications; object storage for resumes; a search engine for keyword job search; email service for alerts; a warehouse for hiring analytics.

Q8 SCENARIO Your dashboard queries are slowing down the checkout page. Why might that happen and what would you do?

- A** Heavy analytical queries compete with checkout transactions on the same database. Move reporting to a read replica or, better, copy data to a warehouse (OLAP) and run dashboards there.

FOLLOW-UP / TRAP**Q9 TRAP QUESTION "Our app uses MongoDB, so we don't have a backend." Is that right?**

- A** No. The choice of database does not remove the need for a backend; something still has to validate requests, enforce rules and protect the data. Even "backend-as-a-service" products are a backend you rent.

Session 1.2 — Modern Data Systems & Integration (Part 2): How Data Moves Between Systems

What is data integration?

Simple meaning: data integration is combining data from different systems so it can be used together, or keeping several systems in sync with each other.

Why it exists: companies run many separate systems. Each team picks the best tool for its job, and soon the same customer exists in five places:

System	What it knows about customer "Aarav Patel"
E-commerce database	his orders and addresses
Payment gateway	his payments and refunds
CRM (Salesforce)	his support calls and sales opportunities
Marketing tool	which emails he opened
Mobile analytics	what he clicked in the app

Each system is a **data silo**: useful alone, but nobody can answer "Who are our most valuable customers, and are they happy?" without integrating them. A unified picture of the customer is often called **Customer 360**.

The common ways data moves

Method	How it works	Speed	Typical use
API call	One system asks another over HTTP and gets a reply	Real time	Checkout calls the payment gateway
File transfer	A system exports a CSV/Excel/JSON file; another imports it (often via SFTP or cloud storage)	Hours (batch)	Bank sends daily settlement files
Database replication / CDC	Changes in one database are copied to another, often by reading the database's change log	Seconds to minutes	Keep a reporting copy or search index fresh
Messaging / events	A system publishes a message; others consume it when ready	Near real time, asynchronous	<code>OrderPlaced</code> event triggers inventory and email
ETL / ELT pipeline	Extract from sources, transform, load into a warehouse	Minutes to hours	Nightly load of sales into the warehouse

You will study each of these properly in Modules 5 and 6. For now, notice the trade-off: **faster integration is usually more complex** to build and run.

APIs: the doors between systems

An **API (Application Programming Interface)** is a contract that says "send me a request in this format and I will reply in that format". Systems do not need to know each other's internals; they only need the contract.

THE RESTAURANT WAITER

You (the client) do not walk into the kitchen. You read the menu (API documentation), tell the waiter (API) what you want, and the waiter brings back your food (the response). The kitchen can change its ovens without changing the menu.

What "data processing" means

Between moving and storing, data is usually processed in a few standard steps:

1. **Collect** the raw data.
2. **Validate** it: are required fields present? Are dates real dates?
3. **Clean** it: remove duplicates, fix formats ("PUNE" , "Pune " → "Pune").
4. **Transform** it: join with other data, calculate totals, convert currencies.
5. **Store** it where it will be used.
6. **Serve** it to an app, report or model.

Why integration is hard

Challenge	Example	Usual fix
Different formats	One system sends XML, another expects JSON	Transformation/mapping layer
Different meanings	cust_no in ERP vs customer_id in CRM; "active customer" defined differently	Data mapping documents, master data management
Duplicates	Same customer created twice with slightly different names	Matching and de-duplication rules
Timing	Report shows yesterday's stock while the app shows today's	Agree on freshness needs; use CDC or events if needed
Failures	Target system is down during the nightly load	Retries, idempotent loads, alerts
Security and privacy	Sensitive data copied to too many places	Masking, least-privilege access, encryption

Worked example: one order touches many systems

When an e-commerce order is placed, the order system integrates with:

- the **payment gateway** (synchronous API call: we must know the payment worked);
- **inventory** (reserve stock, often through an event);
- **notifications** (send email/SMS, asynchronously; the customer should not wait for it);
- **the warehouse** (copy for reporting, in batch or by streaming);
- **the CRM** (so support staff can see the order history).

Choosing *how* each link works (API, event, file, pipeline) is the heart of integration design.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Data integration means bringing data from different systems together so it can be used as one, or keeping those systems in sync. Companies have silos: orders in one database, customers in a CRM, payments with a gateway. We connect them using APIs for real-time requests, events or message queues for asynchronous updates, change data capture to copy database changes, and ETL or ELT pipelines to load a warehouse. The hard parts are mismatched formats and definitions, duplicates, failures and keeping the data fresh enough."

COMMON TRAPS & MISTAKES

- Thinking integration is only about APIs. Files, CDC, events and pipelines are just as common.
- Ignoring meaning: two systems can both have a "customer" field that means different things.
- Assuming real-time is always better. It costs more; many reports are fine with daily data.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is data integration?

A Combining data from multiple sources into a consistent, usable view, or keeping multiple systems synchronised.

Q2 **DEFINITION** What is a data silo?

A Data locked inside one system or team that others cannot easily access or combine, like customer support data that never reaches the sales reports.

Q3 **DEFINITION** What is an API?

A A defined contract that lets one system request data or actions from another without knowing its internals, typically over HTTP with JSON.

INTERMEDIATE**Q4** **COMPARISON** Name four ways to move data between systems and when you would use each.

A API calls for real-time request/response (payments); file transfer for simple batch exchange (daily bank files); CDC/replication to mirror database changes (search index); events/queues for asynchronous notifications (order placed); ETL/ELT for loading warehouses.

Q5 **WHY** Why is data integration difficult?

A Different formats and definitions, duplicates, timing differences, failures and retries, and security/privacy concerns when copying data.

SCENARIO-BASED**Q6** **SCENARIO** Sales says the dashboard shows fewer orders than the app. What could be wrong?

A The pipeline may run in batch (data is stale), a load may have failed silently, filters or definitions may differ (cancelled orders excluded), or duplicates/time-zone issues. I would compare counts per day between source and warehouse to find where they diverge.

Q7 **DESIGN QUESTION** A hospital wants lab results from the lab system to appear in the doctor's app within a minute. How would you integrate them?

A Event-based or CDC integration: the lab system publishes a "ResultReady" event (or CDC captures new rows), a consumer transforms it to the app's format and stores it in the app's database. A nightly batch file would be too slow.

FOLLOW-UP / TRAP**Q8** **TRAP QUESTION** If both systems have a `customer_id`, is integrating them easy?

A Not necessarily. The IDs may come from different numbering schemes, so the same number can mean different people. You need a mapping or a shared master key (for example an email or a master customer ID).

Session 1.3 – Modern Data Systems & Integration (Part 3): Processing, OLTP vs OLAP, and Why One Database Is Not Enough

Batch processing vs real-time (stream) processing

Batch processing collects data over a period and processes it together at a scheduled time.

- *Examples:* monthly payroll, nightly sales report, end-of-day bank statement, daily load into a warehouse.
- *Why use it:* simple, efficient for large volumes, easy to re-run if something fails.

Real-time (stream) processing handles each event within seconds (or less) of it happening.

- *Examples:* card fraud detection, live delivery tracking, stock price alerts, "trending now" lists.
- *Why use it:* some decisions lose their value if they are late. A fraud alert tomorrow is useless.

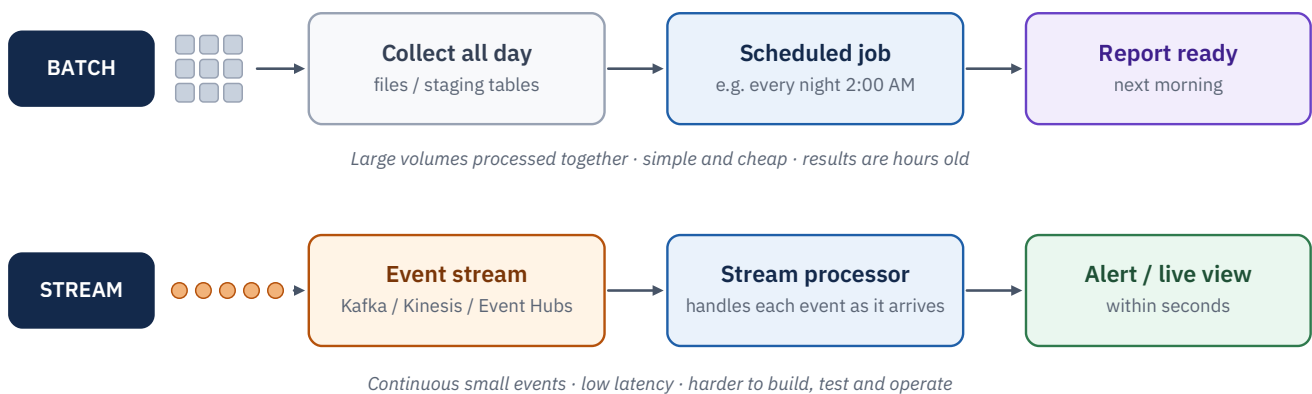


Figure 4. Batch waits and processes everything together; streaming processes each event as it arrives.

Aspect	Batch	Streaming (real-time)
When it runs	On a schedule (hourly, nightly)	Continuously
Latency (delay)	Minutes to hours	Milliseconds to seconds
Data size per run	Large chunks	One event or a small window
Complexity	Lower	Higher (ordering, late events, failures)
Cost	Usually cheaper	Usually more expensive
Re-processing	Easy: run the job again	Harder: needs replayable streams
Typical tools	SQL jobs, Spark, Airflow-scheduled jobs	Kafka, Kinesis, Flink, Spark Structured Streaming
Example	Daily sales report	Fraud detection on card swipes

Between the two sits **near-real-time** or **micro-batch** processing (for example, every 1–5 minutes), which is often "real-time enough" at a fraction of the cost.

THE INTERVIEW-WINNING QUESTION

When asked "batch or real-time?", ask back: **"How fresh does the data need to be for the decision?"** If the answer is "by tomorrow morning", batch is simpler and cheaper.

Transactional systems (OLTP) vs analytical systems (OLAP)

OLTP — Online Transaction Processing systems run the business day to day: placing orders, transferring money, booking tickets. They handle many small, fast reads and writes, and they must be correct every time.

OLAP — Online Analytical Processing systems help people *understand* the business: "revenue by city by month for three years". They run fewer but much heavier queries that scan and summarise lots of history.

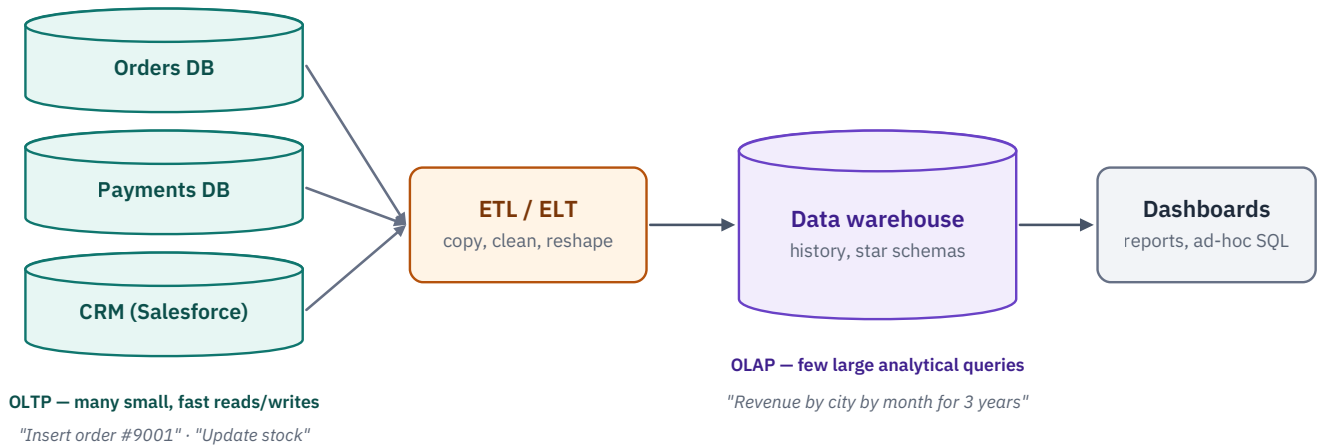


Figure 5. Operational (OLTP) databases feed an analytical (OLAP) warehouse through ETL/ELT pipelines.

Aspect	OLTP	OLAP
Purpose	Run the business	Analyse the business
Typical query	Insert order #9001; fetch one customer's cart	Total sales by region and quarter for 3 years
Query shape	Short, touches few rows	Long, scans millions of rows, aggregates
Users	Customers, staff, applications (thousands at once)	Analysts, managers, data scientists
Data	Current, up to the second	Historical, often refreshed hourly/daily
Design	Normalized (avoid duplication, protect consistency)	Denormalized star/snowflake schemas (fewer joins, fast reads)
Storage	Row-oriented	Often column-oriented
Priority	Fast writes, correctness, concurrency (ACID)	Fast reads and aggregation over large data
Examples	PostgreSQL, MySQL, Oracle, SQL Server running the app	Snowflake, BigQuery, Redshift, Azure Synapse

Why keep them separate? If a manager runs a three-year sales report on the same database that handles checkout, the report competes for CPU, memory and disk, and customers feel it as a slow checkout. Copying data into a warehouse lets each system be designed for its own job.

Operational data vs analytical data

- **Operational data** is the *current state* needed to run the business: today's stock level, the customer's current address, the open orders.
- **Analytical data** is *history and summaries* used for decisions: stock levels every day for two years, how many customers moved city, monthly revenue trends.

Operational systems often overwrite old values (the new address replaces the old one). Analytical systems usually *keep* history so trends can be studied (Module 2 shows how, with slowly changing dimensions).

Why modern systems need multiple data technologies

No single database is best at everything. Using several, each for the job it does best, is called **polyglot persistence**.

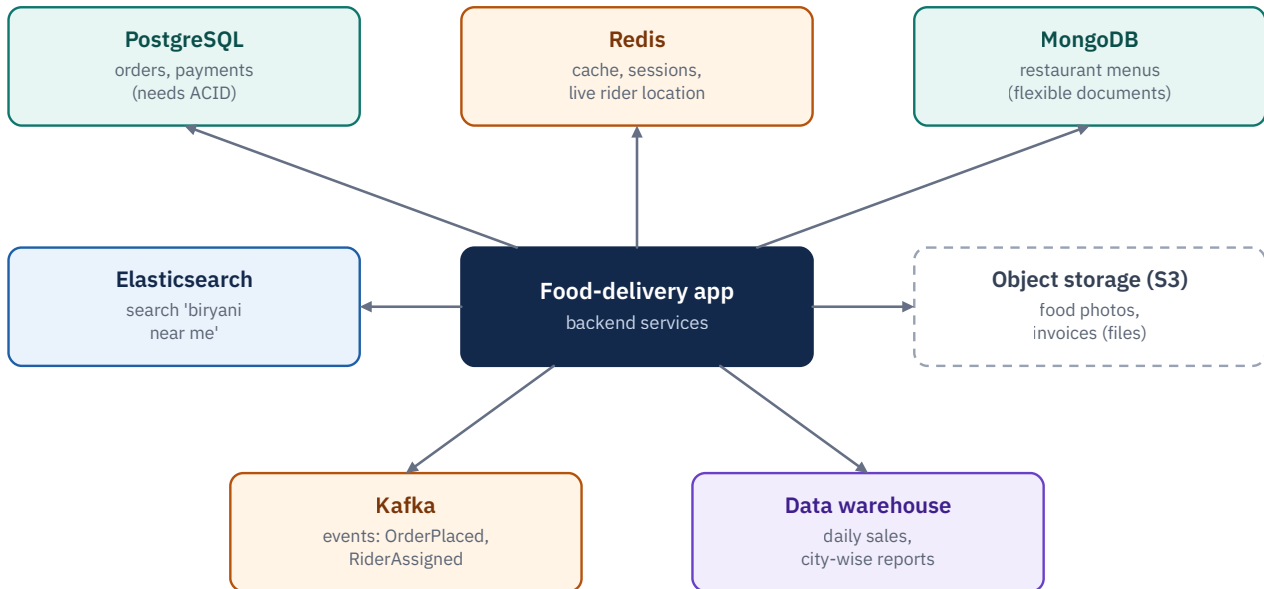


Figure 6. One food-delivery app, seven data technologies, each chosen for a specific job.

Job	Best-fit technology	Why not the relational database?
Money and orders	Relational (PostgreSQL)	— it is the right tool: ACID and relationships
Microsecond lookups, sessions, counters	Redis	A relational DB is slower and wastes resources on this
Flexible, changing product/menu attributes	Document DB (MongoDB)	Constant schema changes in a relational DB are painful
Free-text search with typos and ranking	Elasticsearch	<code>LIKE '%biryani%'</code> cannot rank results or handle typos well
Large files (images, PDFs)	Object storage	Storing large binary files in tables bloats the database
Events between services	Kafka	A table used as a queue does not scale well to many consumers
Years of history for reports	Data warehouse	Heavy scans would slow down the transactional database

A BALANCED VIEW (INTERVIEWERS LIKE THIS)

More technologies mean more to learn, monitor, secure and keep consistent. A good engineer starts simple (a single PostgreSQL database can go a long way, and it even supports JSON documents and full-text search) and adds a new store only when there is a clear need.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"OLTP systems run the business: lots of small, fast transactions like placing an order or transferring money, so they are normalized and ACID-compliant. OLAP systems analyse the business: fewer but heavy queries over years of history, like revenue by region per quarter, so they use denormalized star schemas, often in a columnar warehouse such as Snowflake or BigQuery. We keep them separate so reports don't slow down customers, and we move data from OLTP to OLAP with ETL or ELT pipelines."

COMMON TRAPS & MISTAKES

- Saying "OLAP is a type of database product." OLAP is a *workload*; many products serve it.
- Claiming real-time is always better than batch. It is costlier and more complex; choose by the freshness the business needs.
- Thinking polyglot persistence is always good. Each extra store adds operational cost.
- Mixing up "operational data" (current state) with "analytical data" (history).

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is batch processing?

A Processing data collected over a period together at a scheduled time, for example a nightly sales report or monthly payroll.

Q2 **DEFINITION** What is stream (real-time) processing?

A Processing each event continuously within seconds of it happening, for example fraud checks on card transactions or live order tracking.

Q3 **COMPARISON** What is the difference between OLTP and OLAP?

A OLTP handles many small, fast transactions that run the business and needs correctness (ACID); OLAP handles fewer, heavy analytical queries over historical data and is optimised for reading and aggregating.

INTERMEDIATE**Q4** **WHY** Why do companies keep separate OLTP and OLAP systems?

A Analytical queries scan huge amounts of data and would slow down customer transactions; the two workloads also need different designs (normalized vs star schema, row vs column storage).

Q5 **DEFINITION** What is polyglot persistence?

A Using different data storage technologies for different needs within one application, such as PostgreSQL for orders, Redis for caching and Elasticsearch for search.

Q6 **COMPARISON** Operational data vs analytical data?

A Operational data is the current state needed for daily work; analytical data is historical and summarised for decision-making, and usually keeps history instead of overwriting it.

Q7 **DEFINITION** What is micro-batch processing?

A Running small batches very frequently (every few seconds or minutes) to get near-real-time results with batch-style simplicity.

SCENARIO-BASED

Q8 SCENARIO A bank wants to block suspicious card transactions. Batch or streaming? Why?

A Streaming. The decision must be made before or within seconds of the transaction; a nightly batch would only find the fraud after the money is gone.

Q9 SCENARIO HR wants a monthly attrition report. Batch or streaming?

A Batch. Monthly freshness is enough, batch is simpler and cheaper, and it is easy to re-run.

Q10 DESIGN QUESTION Which data stores would you use for a ride-sharing app?

A PostgreSQL for riders, drivers, trips and payments (ACID); Redis or a geospatial store for live driver locations; Kafka for trip events; object storage for documents; a warehouse for analytics such as trips per city per hour.

FOLLOW-UP / TRAP

Q11 TRAP QUESTION Can PostgreSQL be used for analytics?

A Yes, for moderate data sizes PostgreSQL handles reporting well, especially on a read replica. Dedicated warehouses become worth it when data grows very large or many analysts run heavy queries.

Q12 TRAP QUESTION Is "real-time" always worth it?

A No. It adds cost and complexity (ordering, late events, failure handling). If the business acts on data daily, batch is the better engineering choice.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. List the six building blocks of a data system.

P2. Classify each as OLTP or OLAP: (a) withdrawing cash at an ATM, (b) a yearly sales trend chart, (c) adding an item to a cart, (d) comparing branch performance across five years.

P3. Give one example each of batch and streaming processing from daily life.

LEVEL 2 – INTERVIEW APPLICATION

P4. Explain, in under a minute, why the frontend should not connect directly to the database.

P5. A colleague wants to store product images inside a PostgreSQL table. What would you suggest instead, and why?

P6. Explain polyglot persistence and give one disadvantage.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. An e-commerce company's checkout becomes slow every morning at 9 AM, when managers open their dashboards. Diagnose the likely cause and propose two fixes (one quick, one long-term).

P8. Design the data flow for a food-delivery "orders per city per hour" dashboard that must be at most 5 minutes old. Which integration method would you choose?

ANSWER / EXPLANATION

P1. Sources, ingestion, storage, processing, serving, and governance (security, quality, metadata, monitoring) across all of them.

P2. (a) OLTP, (b) OLAP, (c) OLTP, (d) OLAP.

P3. Batch: your monthly credit card statement. Streaming: the live location of your cab or delivery rider.

P4. "Putting database credentials inside an app is a security risk, and users could bypass the business rules. An API in the middle validates requests, checks permissions and hides the database design, so we can change the schema without breaking every installed app."

P5. Store the images in object storage such as Amazon S3 or Azure Blob and keep only the image URL (and maybe size and type) in PostgreSQL. Object storage is cheaper for large files, keeps the database small and fast, and works with a CDN for quick delivery.

P6. Using different storage technologies for different jobs within one system (relational for orders, Redis for cache, Elasticsearch for search). Disadvantage: more systems to run, secure and monitor, and keeping data consistent across them is harder.

P7. Heavy analytical queries are running on the same OLTP database as checkout and competing for resources. Quick fix: point dashboards at a read replica. Long-term fix: load the data into a data warehouse with ETL/ELT (or streaming) and run all dashboards there.

P8. Five minutes is near-real-time, so use change data capture or order events (Kafka) feeding a stream or micro-batch job that aggregates orders by city and hour into the warehouse or a small serving table. A nightly batch would be too stale; full streaming with per-second updates is more than needed.

Session 1.4 — Module 1 Summary & Rapid Revision

MODULE 1 SUMMARY

MOST IMPORTANT CONCEPTS

- A **data system** = sources → ingestion → storage → processing → serving, with governance across all.
- A modern app = **frontend** → **backend/API** → **database**, plus other services and analytics.
- **Data integration** connects silos using APIs, files, CDC, events and ETL/ELT.
- **Batch** (scheduled, cheap, simple) vs **streaming** (continuous, low latency, complex).
- **OLTP** (run the business, many small ACID transactions) vs **OLAP** (analyse the business, heavy queries over history).
- **Polyglot persistence**: different stores for different jobs, at the cost of complexity.

WHAT TO MEMORIZE

- OLTP = transactions, normalized, row store, current data. OLAP = analytics, star schema, column store, historical data.
- Five ways data moves: API, file, CDC/replication, events/messaging, ETL/ELT.

WHAT TO UNDERSTAND (NOT MEMORIZE)

- *Why* the frontend never talks to the database directly.
- *Why* analytics is separated from transactions.
- *How* to choose batch vs streaming: by the freshness the decision needs.

MOST COMMON INTERVIEW QUESTIONS

- OLTP vs OLAP? • Batch vs streaming? • What is data integration? • Walk me through your app's architecture.
- Why use several databases?

COMMON MISTAKES

- "SQL is a database." • "Real-time is always better." • "One database for everything." • Forgetting the backend layer.

RAPID REVISION CHECKLIST

- | | |
|---|--|
| ☐ I can define a data system and its building blocks | ☐ I can compare batch and streaming with examples |
| ☐ I can draw frontend → API → database → analytics | ☐ I can compare OLTP and OLAP in a table |
| ☐ I can explain what a backend does | ☐ I can explain operational vs analytical data |
| ☐ I can define data integration and data silos | ☐ I can explain polyglot persistence and its cost |
| ☐ I can list five ways data moves between systems | ☐ I can describe my own project in layers |

PART II · MODULE 2

Data Modeling Fundamentals

Data modeling is deciding *what* data you store, *how* it is split into tables (or documents) and *how* the pieces connect. A good model makes queries simple and data trustworthy; a bad one causes duplicated, contradictory data for years. Interviewers love this module because it shows whether you can think, not just memorize.

COURSE COVERAGE

- ✓ Introduction to data modeling (conceptual, logical, physical) † → Session 2.1
- ✓ Entities, attributes, relationships, ER concepts † → Session 2.2
- ✓ Primary keys, foreign keys, constraints † → Session 2.3
- ✓ Relational model, schema, table design; one-to-one, one-to-many, many-to-many † → Session 2.4
- ✓ Normalization: functional dependency, update/insert/delete anomalies, 1NF, 2NF, 3NF, BCNF † → Session 2.5
- ✓ Denormalization; when to normalize and when to denormalize † → Session 2.6
- ✓ Data warehousing: data warehouse, fact and dimension tables, star and snowflake schemas, star vs snowflake, OLTP vs OLAP † → Session 2.7
- ✓ NoSQL modeling: why NoSQL, schema flexibility, document modeling, embedding vs referencing, access-pattern-driven design † → Session 2.8
- ✓ Module quiz concepts → interview questions in each session and Session 2.9

Topic	Priority	Typical question
Keys and constraints	HIGH PRIORITY	"Primary key vs foreign key vs unique key?"
Normalization (1NF–3NF, anomalies)	HIGH PRIORITY	"Normalize this table." "2NF vs 3NF?"
Relationships, many-to-many	HIGH PRIORITY	"How do you model students and courses?"
Star vs snowflake, facts vs dimensions	MEDIUM PRIORITY	"Design a star schema for sales."
Denormalization	MEDIUM PRIORITY	"When would you denormalize?"
NoSQL modeling, embedding vs referencing	MEDIUM PRIORITY	"Embed or reference order items in MongoDB?"

PWC ANGLE

Consulting work often starts with messy client data: spreadsheets, legacy tables and multiple systems describing the same customer differently. Being able to spot duplication, propose clean keys and explain normalization in business terms ("we store the customer's address once, so a change is made in one place") is directly useful, and it is the kind of reasoning a "design a schema for..." question is testing.

Session 2.1 — What Is Data Modeling?

Simple meaning

Data modeling is planning how data will be organised before you build the database, in the same way an architect draws a blueprint before anyone pours concrete. It answers three questions:

1. *What* things do we need to store data about? (customers, orders, products)
2. *What* do we need to know about each one? (a customer's name, email, city)
3. *How* are they connected? (a customer places many orders)

Why it matters

- **Fewer bugs in the data.** A good model makes it hard to store contradictions, like two different prices for the same product.
- **Simpler, faster queries.** When data is organised the way it will be used, queries are short and quick.
- **Cheaper changes.** Fixing a model after millions of rows exist is painful and risky.
- **A shared language.** Business people and engineers can agree on a picture before anyone writes code.

The three levels of a data model

Level	Question it answers	Who reads it	E-commerce example
Conceptual	What are the main things and how do they relate?	business stakeholders	"A Customer places Orders. An Order contains Products."
Logical	What exactly do we store about each thing? Which keys?	analysts, architects	Customer(customer_id, name, email, city); Order(order_id, customer_id, order_date, status)
Physical	How is it built in <i>this</i> database?	developers, DBAs	customers table in PostgreSQL, email VARCHAR(80) UNIQUE, index on orders(customer_id)

A logical model is independent of any product. The physical model adds the product-specific details: exact data types, indexes, partitions and storage settings.

The modeling process, step by step

1. **Understand the requirements.** What questions must the system answer? ("Which customers ordered in the last 30 days?")
2. **Find the entities**, the nouns that matter (Customer, Order, Product).
3. **List their attributes** (name, email, price).
4. **Define relationships and cardinality** (one customer, many orders).
5. **Choose keys** (primary and foreign keys).
6. **Normalize** to remove redundancy (Session 2.5).
7. **Design the physical tables:** types, constraints, indexes.
8. **Validate against access patterns:** can every important query be written simply and run fast?

Different kinds of data model

Model	Shape	Best for	Session
Relational	tables linked by keys	transactional systems (OLTP)	2.2–2.6
Dimensional	facts surrounded by dimensions (star schema)	analytics (OLAP), data warehouses	2.7
Document	JSON-like documents	flexible, nested data; read patterns known up front	2.8
Key-value / wide-column / graph	key → value; rows of sparse columns; nodes and edges	caching, huge write volumes, relationships	2.8, 4.2

WORKED EXAMPLE: A SMALL LIBRARY SYSTEM

Requirement: "Members borrow books. A book can have several authors. We must know who has which book and when it is due."

- *Entities:* Member, Book, Author, Loan.
- *Relationships:* a Member has many Loans; a Book has many Loans over time; Book ↔ Author is many-to-many.
- *Keys:* member_id, book_id, author_id; Loan(loan_id, member_id, book_id, loan_date, due_date, return_date).
- *Physical decisions:* index `loans(member_id)` for "my books" pages; `return_date` NULL means "not yet returned".

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Data modeling is designing how data will be structured before building the database: which entities we store, what attributes they have and how they relate. We usually go from a conceptual model for the business, to a logical model with entities, attributes and keys, to a physical model for a specific database with data types and indexes. A good model avoids duplicate and inconsistent data and makes the important queries simple. For example, for e-commerce I'd model customers, orders, products and an order_items table between orders and products."

COMMON TRAPS & MISTAKES

- Jumping straight to `CREATE TABLE` without asking what questions the data must answer.
- Mixing up logical (product-independent) and physical (product-specific) models.
- Thinking modeling is only for relational databases. NoSQL needs modeling too, just driven by access patterns.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is data modeling?

A Planning the structure of data, meaning entities, attributes, relationships and keys, before implementing it in a database.

Q2 **COMPARISON** What are conceptual, logical and physical data models?

- A** Conceptual: high-level entities and relationships for the business. Logical: detailed attributes, keys and relationships, independent of any product. Physical: the actual tables, types, constraints and indexes in a specific DBMS.

INTERMEDIATE**Q3** **WHY** Why is data modeling important?

- A** It prevents redundancy and inconsistency, makes queries simpler and faster, reduces the cost of later changes, and gives business and technical teams a shared picture.

Q4 **HOW** What steps do you follow to model a new system?

- A** Gather requirements and key questions, identify entities, attributes and relationships, choose keys, normalize, design the physical schema, then validate it against the main queries.

SCENARIO-BASED**Q5** **DESIGN QUESTION** You are asked to model a hospital appointment system. What entities would you start with?

- A** Patient, Doctor, Department, Appointment, and possibly Prescription and Bill. Doctor belongs to one Department; Patient has many Appointments; each Appointment links one Patient and one Doctor at a time slot.

FOLLOW-UP / TRAP**Q6** **TRAP QUESTION** Does a NoSQL database mean you don't need a data model?

- A** No. The model moves into the application and is driven by access patterns. Without design you get inconsistent documents and slow queries.

Session 2.2 — Entities, Attributes and Relationships (ER Modeling)

Entities

An **entity** is a real-world "thing" we store data about: *Customer, Product, Order, Employee, Department*. Each specific customer, such as Aarav Patel, is an **entity instance**, which becomes one **row** in the table.

- A **strong entity** can be identified on its own (a Customer has a `customer_id`).
- A **weak entity** cannot exist or be identified without its owner. An *order item* only makes sense inside an order: it is identified by (`order_id`, line number or `product_id`).

Attributes

An **attribute** is a property of an entity: a customer's name, email and city.

Attribute type	Meaning	Example	How it becomes tables
Simple	cannot be split usefully	<code>email</code>	one column
Composite	made of parts	<code>address</code> = street + city + pin code	several columns (<code>street</code> , <code>city</code> , <code>pin_code</code>)
Single-valued	one value per entity	<code>date_of_birth</code>	one column
Multi-valued	many values per entity	a customer's several phone numbers	a separate table <code>customer_phones</code>
Derived	can be calculated from others	<code>age</code> from <code>date_of_birth</code> ; order total from items	usually <i>not</i> stored (or stored on purpose for speed, see Session 2.6)
Key	identifies the entity	<code>customer_id</code>	primary key

Relationships and cardinality

A **relationship** is how entities are connected: *Customer places Order, Employee works in Department*. **Cardinality** says *how many* on each side:

Cardinality	Meaning	Example
One-to-one (1:1)	one A ↔ one B	each user has one profile
One-to-many (1:N)	one A ↔ many B	one customer places many orders
Many-to-many (M:N)	many A ↔ many B	students take many courses; each course has many students

Participation (optionality) says whether the relationship is required:

- Every order *must* belong to a customer → **mandatory** (total participation). In SQL: `customer_id NOT NULL`.
- A customer *may* have zero orders → **optional** (partial participation).

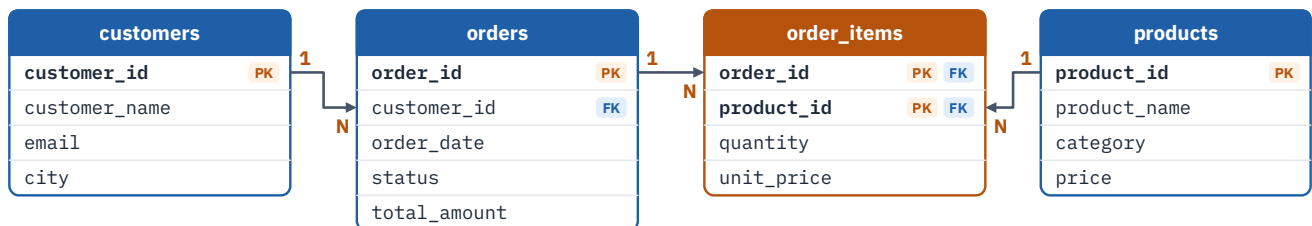
Reading an ER diagram

An **ER (Entity-Relationship) diagram** draws entities as boxes and relationships as lines. Two notations are common:

- **Chen notation** (textbooks): entities as rectangles, attributes as ovals, relationships as diamonds.
- **Crow's foot notation** (industry tools): entities as boxes listing their columns; the line ends show cardinality.

Line-end symbol (crow's foot)	Meaning
(two bars)	exactly one
o (circle + bar)	zero or one
< (bar + crow's foot)	one or many
o< (circle + crow's foot)	zero or many

In this handbook the diagrams simply write **1** and **N** at the line ends:



customers 1—N orders 1—N order_items N—1 products (orders ↔ products is many-to-many)

Figure 7. The e-commerce part of this handbook's sample database as an ER diagram.

Finding entities in a requirement

A practical trick: **nouns** are candidate entities or attributes; **verbs** are candidate relationships.

"Patients **book** appointments with doctors. Each doctor **belongs to** one department. A patient **can have** many appointments, and each appointment **may produce** a prescription."

- Entities: Patient, Appointment, Doctor, Department, Prescription.
- Relationships: Patient 1:N Appointment; Doctor 1:N Appointment; Department 1:N Doctor; Appointment 1:0..1 Prescription.

ENTITY OR ATTRIBUTE?

Make something its own entity when it has its own attributes or is shared. "City" can be a plain attribute of a customer, but if you need each city's state, region and delivery charges, make **cities** a table and reference it.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"An entity is a thing we store data about, like a customer or an order; each row is one instance. Attributes are its properties, like name and email. Relationships connect entities, and cardinality tells us how many: a customer places many orders, so that's one-to-many, while students and courses are many-to-many. In an ER diagram I'd draw the entities as boxes, list their keys and mark the cardinality on the lines, for example with crow's foot notation."

COMMON TRAPS & MISTAKES

- Storing a multi-valued attribute (several phone numbers) as a comma-separated string. It belongs in its own table.
- Forgetting optionality: "every order has a customer" should become **NOT NULL** on the foreign key.
- Confusing an *entity* (Customer) with an *entity instance* (Aarav).

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is an entity? Give an example.

A A real-world object or concept we store data about, such as Customer or Product. Each instance (one customer) becomes a row.

Q2 **DEFINITION** What is an attribute?

A A property of an entity, such as a customer's name or email. It becomes a column.

Q3 **DEFINITION** What is cardinality?

A The number of instances of one entity that can relate to another: one-to-one, one-to-many or many-to-many.

INTERMEDIATE

Q4 **COMPARISON** What is the difference between a strong and a weak entity?

A A strong entity has its own identifier; a weak entity depends on an owner entity for its identity, for example an order line identified by order_id plus line number.

Q5 **COMPARISON** What are composite, multi-valued and derived attributes?

A Composite has parts (address = street, city, pin); multi-valued has several values (phone numbers); derived can be calculated (age from date of birth).

Q6 **HOW** How do you represent a multi-valued attribute in tables?

A Put it in a separate child table with a foreign key to the parent, for example
`customer_phones(customer_id, phone)`.

SCENARIO-BASED

Q7 **SCENARIO** From "Employees work in departments; each department has one manager who is also an employee", identify entities and relationships.

A Entities: Employee, Department. Relationships: Department 1:N Employee (works in); Department 1:1 Employee (managed by), which is stored as `manager_emp_id` in departments, referencing employees.

FOLLOW-UP / TRAP

Q8 **TRAP QUESTION** Should you store a customer's age?

A Usually no. Store the date of birth and calculate the age, because a stored age becomes wrong every year.

Session 2.3 — Keys and Constraints

HIGH PRIORITY This is one of the most asked topics in any DBMS interview.

Why keys exist

Keys do two jobs: they **identify each row uniquely**, and they **link tables together**. Without them, you could not reliably say "update *this* customer" or "find *this* customer's orders".

Every kind of key, on one table

Take an `employees` table where each employee has `emp_id`, `email`, `pan_number` (a tax ID that is unique per person), `name` and `dept_id`.

Key	Definition	Example
Super key	any set of columns that uniquely identifies a row	{ <code>emp_id</code> }, { <code>emp_id</code> , <code>name</code> }, { <code>email</code> }, { <code>email</code> , <code>dept_id</code> } ...
Candidate key	a <i>minimal</i> super key: remove any column and it stops being unique	{ <code>emp_id</code> }, { <code>email</code> }, { <code>pan_number</code> }
Primary key (PK)	the candidate key we <i>choose</i> as the main identifier; unique and NOT NULL	<code>emp_id</code>
Alternate key	candidate keys that were not chosen as the PK	<code>email</code> , <code>pan_number</code>
Unique key	a column (or set) declared <code>UNIQUE</code> ; no duplicates, but may allow NULL	<code>email</code>
Composite key	a key made of two or more columns	(<code>order_id</code> , <code>product_id</code>) in <code>order_items</code>
Foreign key (FK)	a column that refers to the primary (or unique) key of another table	<code>employees.dept_id</code> → <code>departments.dept_id</code>
Surrogate key	an artificial ID with no business meaning	auto-generated <code>emp_id</code> 1, 2, 3
Natural key	a key with real-world meaning	PAN number, email

Primary key rules

- Must be **unique** and **NOT NULL**.
- **One primary key per table**, though it may be composite.
- Should be **stable** (never changes) and **minimal** (no unnecessary columns).

Foreign keys and referential integrity

A **foreign key** says "this value must exist in that other table". The guarantee it provides is called **referential integrity**: there are no orphan rows, such as an order for a customer who does not exist.

CUSTOMERS (PARENT)

<code>customer_id</code>	<code>customer_name</code>	<code>email</code>
1	Aarav Patel	aarav@mail.com
2	Isha Kulkarni	isha@mail.com

ORDERS (CHILD)

order_id	customer_id	order_date	total_amount
101	1	2024-01-05	2843.00
102	2	2024-01-12	3499.00
103	1	2024-01-28	8999.00

`orders.customer_id` is a foreign key to `customers.customer_id`. Customer 1 has two orders, which is the one-to-many relationship in action. Watch PostgreSQL protect the data in the sample database. First, try to create an order for a customer who does not exist:

```
INSERT INTO orders (order_id, customer_id, order_date, status)
VALUES (200, 999, '2024-05-01', 'PENDING');
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
DETAIL: Key (customer_id)=(999) is not present in table "customers".

Now try to delete a customer who still has orders:

```
DELETE FROM customers WHERE customer_id = 1;
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table "orders"
DETAIL: Key (customer_id)=(1) is still referenced from table "orders".

What happens to the children when a parent is deleted is controlled by the **referential action** on the foreign key:

Action	When the parent row is deleted...	Typical use
NO ACTION / RESTRICT (default)	the delete is refused while children exist	orders, payments: never lose history silently
CASCADE	the children are deleted too	a post's comments; a cart's items
SET NULL	the child's FK becomes NULL	employee's manager leaves → <code>manager_id</code> NULL
SET DEFAULT	the child's FK gets its default value	rare

The other constraints

Constraints are rules the database enforces on every insert and update, no matter which application writes the data.

Constraint	Rule	Example
NOT NULL	a value is required	<code>customer_name VARCHAR(60) NOT NULL</code>
UNIQUE	no duplicate values	<code>email VARCHAR(80) UNIQUE</code>
PRIMARY KEY	UNIQUE + NOT NULL, the row's identity	<code>customer_id INT PRIMARY KEY</code>
FOREIGN KEY	value must exist in the parent table	<code>customer_id INT REFERENCES customers(customer_id)</code>
CHECK	value must satisfy a condition	<code>price NUMERIC(10,2) CHECK (price >= 0)</code>
DEFAULT	value used when none is given	<code>status VARCHAR(12) DEFAULT 'PENDING'</code>

Each violation produces a clear error. A duplicate email:

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Aarav P', 'aarav@mail.com', 'Mumbai', '2024-05-01');
```

SQL · PostgreSQL

PostgreSQL Says

ERROR duplicate key value violates unique constraint "customers_email_key"
 DETAIL: Key (email)=(aarav@mail.com) already exists.

A negative price, blocked by a CHECK constraint:

```
UPDATE products SET price = -10 WHERE product_id = 1;
```

SQL · PostgreSQL

PostgreSQL Says

ERROR new row for relation "products" violates check constraint "products_price_check"
 DETAIL: Failing row contains (1, Wireless Mouse, Electronics, -10.00).

A missing required value:

```
INSERT INTO products (product_id, product_name, category, price)
VALUES (9, NULL, 'Electronics', 499);
```

SQL · PostgreSQL

PostgreSQL Says

ERROR null value in column "product_name" of relation "products" violates not-null constraint
 DETAIL: Failing row contains (9, null, Electronics, 499.00).

Primary key vs unique key: the NULL difference

A **UNIQUE** column in PostgreSQL allows **many NULLs**, because NULL means "unknown" and two unknowns are not considered equal. A primary key allows **no NULLs**. Customer 7 already has a NULL email; adding another customer with a NULL email works:

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Test User', NULL, 'Pune', '2024-05-01');
```

SQL · PostgreSQL

```
SELECT customer_id, customer_name, email
FROM customers
WHERE email IS NULL;
```

Output

customer_id	customer_name	email
7	Reyansh Bose	NULL
8	Test User	NULL

2 rows

Line by Line

Line	What it does
INSERT INTO customers (...) VALUES (8, 'Test User', NULL, ...)	adds a customer whose email is unknown (NULL)
SELECT customer_id, customer_name, email	shows these three columns
FROM customers	from the customers table
WHERE email IS NULL	only rows with no email; note IS NULL, never = NULL

The example was rolled back after it ran, so the sample data is unchanged for the next query.

DATABASE DIFFERENCES

SQL Server allows only **one** NULL in a UNIQUE column. PostgreSQL 15+ can behave the same way with `UNIQUE NULLS NOT DISTINCT`. Mentioning this shows real depth.

Comparison tables interviewers expect

	Primary key	Unique key
Purpose	the row's main identity	prevent duplicates in another column
NULLs	not allowed	allowed (PostgreSQL: many)
How many per table	one	many
Index	created automatically	created automatically
Referenced by FKs	yes (usual)	yes (allowed)

	Primary key	Foreign key
Purpose	identifies rows in <i>its own</i> table	links to rows in <i>another</i> table
Uniqueness	must be unique	can repeat (many orders per customer)
NULLs	not allowed	allowed unless declared NOT NULL
Count per table	one	any number

Surrogate vs natural keys

	Surrogate key (id 1, 2, 3 or UUID)	Natural key (email, PAN)
Meaning	none, purely technical	has business meaning
Stability	never changes	can change (people change emails)
Size and speed	small integer, fast joins	often long strings
Risk	duplicates possible unless you also add UNIQUE on the natural key	business changes break keys

Common practice: use a surrogate primary key (`INT GENERATED ALWAYS AS IDENTITY` , `BIGSERIAL` , or a UUID) and a `UNIQUE` constraint on the natural key.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A primary key uniquely identifies each row in a table. It must be unique and not null, and there is only one per table, although it can be made of several columns. A foreign key is a column in one table that points to the primary key of another, which enforces referential integrity: you cannot create an order for a customer who doesn't exist, and you can't delete a customer who still has orders unless you choose `CASCADE` or `SET NULL`. A unique key also prevents duplicates, but it can allow NULLs and a table can have many of them."

COMMON TRAPS & MISTAKES

- "A primary key can be NULL if it's composite." No: no part of a primary key may be NULL.
- "A foreign key must be unique." No: many child rows can share the same FK value.
- "A table can have two primary keys." It has one PK; the other candidates become alternate or unique keys.

- "UNIQUE allows exactly one NULL." That depends on the database: PostgreSQL and MySQL allow many; SQL Server allows one.
- Forgetting that a FK must reference a PRIMARY KEY or UNIQUE column.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a primary key?

A A column or set of columns that uniquely identifies each row. It must be unique and not null, and each table has only one.

Q2 **DEFINITION** What is a foreign key?

A A column whose values must match the primary or unique key of another table. It links tables and enforces referential integrity.

Q3 **COMPARISON** Primary key vs unique key?

A Both prevent duplicates. A primary key cannot be NULL and there is only one per table; a unique key may allow NULLs and a table can have several.

Q4 **DEFINITION** What is a composite key?

A A key made of two or more columns, such as (order_id, product_id) in an order_items table.

INTERMEDIATE

Q5 **COMPARISON** Super key vs candidate key vs primary key?

A A super key is any set of columns that is unique; a candidate key is a minimal super key; the primary key is the candidate key chosen as the main identifier.

Q6 **DEFINITION** What is referential integrity?

A The guarantee that every foreign key value points to an existing parent row, so there are no orphan records.

Q7 **HOW** What happens when you delete a parent row that has child rows?

A By default the delete is rejected. With `ON DELETE CASCADE` the children are deleted too; with `SET NULL` their foreign key becomes NULL.

Q8 **COMPARISON** Surrogate key vs natural key?

A A surrogate key is an artificial, meaningless ID (auto-increment or UUID); a natural key has business meaning (email, PAN). Surrogates are stable and compact, so they are commonly used as the PK, with a UNIQUE constraint on the natural key.

Q9 **DEFINITION** What is a CHECK constraint?

A A rule that every row must satisfy, such as `CHECK (price >= 0)` or `CHECK (status IN ('PENDING', 'SHIPPED'))`.

SCENARIO-BASED

Q10 **SCENARIO** An employee table uses email as its primary key. What problems could this cause?

A Emails change, so the PK and every foreign key pointing to it would have to change. Emails are also long strings, which makes joins and indexes larger. Better: a surrogate `emp_id` PK plus `UNIQUE (email)`.

Q11 **SCENARIO** Users are getting orders linked to deleted customers. What is missing?

A A foreign key from `orders.customer_id` to `customers.customer_id` (and probably `NOT NULL`). Then the database itself blocks orphan rows.

FOLLOW-UP / TRAP

Q12 **TRAP QUESTION** Can a foreign key be NULL?

A Yes, unless it is declared NOT NULL. A NULL foreign key means "no related row", for example an employee with no manager.

Q13 **TRAP QUESTION** Can a foreign key reference a column that is not the primary key?

A Yes, as long as that column has a UNIQUE (or PRIMARY KEY) constraint.

Q14 **TRAP QUESTION** Can a table have no primary key?

A Technically yes, but it is bad practice: you cannot reliably identify, update or de-duplicate rows, and many tools require a PK.

Session 2.4 – The Relational Model, Schema and Table Design

The relational model in one picture

The relational model, proposed by E. F. Codd in 1970, stores data in **relations**, which we call tables. Its vocabulary still appears in interviews:

Formal term	Everyday term	Meaning
Relation	table	a set of rows with the same columns
Tuple	row	one record
Attribute	column	one property
Domain	data type + allowed values	e.g. <code>status</code> must be one of five words
Degree	number of columns	<code>customers</code> has degree 5
Cardinality (of a table)	number of rows	<code>customers</code> has 7 rows

Rows in a relation have **no guaranteed order**, which is why SQL needs `ORDER BY` when order matters.

Two meanings of "schema"

1. **The design** of the database: which tables, columns, types and constraints exist ("the schema of the orders table").
2. In **PostgreSQL**, a *schema* is also a **namespace (a folder) inside a database** that holds tables. The default one is `public`, so `public.orders` is the full name of `orders`. Teams use schemas such as `sales`, `hr` and `staging` to organise objects and permissions.

Turning relationships into tables

One-to-one

1:1

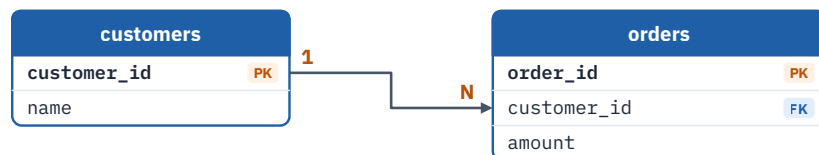
each user has exactly one profile



One-to-many

1:N

one customer places many orders



Many-to-many

M:N

students take many courses;
courses have many students

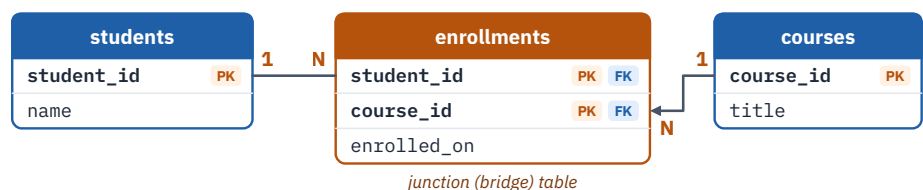


Figure 8. How each kind of relationship becomes tables and keys.

Relationship	Rule	Example
1 : N	put the foreign key on the "many" side	<code>orders.customer_id → customers</code>
1 : 1	foreign key on either side with a UNIQUE constraint , or share the same primary key	<code>user_profiles.user_id</code> is both PK and FK
M : N	create a junction (bridge) table holding both foreign keys, usually as a composite PK	<code>enrollments(student_id, course_id)</code>
Multi-valued attribute	child table	<code>customer_phones(customer_id, phone)</code>
Weak entity	child table whose key includes the owner's key	<code>order_items(order_id, product_id, ...)</code>

Building a many-to-many design for real

Students and courses are the classic many-to-many example. The block below creates the three tables with every kind of constraint, then loads a little data. It is kept for the rest of this chapter (keep):

```

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    full_name  VARCHAR(60) NOT NULL,
    email      VARCHAR(80) UNIQUE,
    city       VARCHAR(40) NOT NULL DEFAULT 'Unknown'
);

CREATE TABLE courses (
    course_id  VARCHAR(10) PRIMARY KEY,
    title      VARCHAR(80) NOT NULL,
    credits    INT NOT NULL CHECK (credits BETWEEN 1 AND 6)
);

CREATE TABLE enrollments (
    student_id INT NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,
    course_id  VARCHAR(10) NOT NULL REFERENCES courses(course_id),
    enrolled_on DATE NOT NULL,
    grade      CHAR(1) CHECK (grade IN ('A','B','C','D','F')),
    PRIMARY KEY (student_id, course_id)
);

INSERT INTO students VALUES
    (1, 'Asha Rao', 'asha@uni.edu', 'Pune'),
    (2, 'Vikram Shah', 'vikram@uni.edu', 'Delhi'),
    (3, 'Neha Iyer', NULL, 'Chennai');

INSERT INTO courses VALUES
    ('DB101', 'Database Systems', 4),
    ('PY101', 'Python Programming', 3),
    ('DS201', 'Data Structures', 4);

INSERT INTO enrollments VALUES
    (1, 'DB101', '2024-07-01', 'A'),
    (1, 'PY101', '2024-07-01', 'B'),
    (2, 'DB101', '2024-07-02', NULL),
    (3, 'DS201', '2024-07-03', 'A'),
    (3, 'DB101', '2024-07-03', 'B');

```

LINE BY LINE

Part of the code	What it means
<code>student_id INT PRIMARY KEY</code>	every student has a unique, non-null ID
<code>email VARCHAR(80) UNIQUE</code>	no two students share an email (NULL allowed)
<code>DEFAULT 'Unknown'</code>	used if no city is supplied
<code>CHECK (credits BETWEEN 1 AND 6)</code>	rejects impossible credit values
<code>REFERENCES students(student_id) ON DELETE CASCADE</code>	each enrollment must point to a real student; deleting a student removes their enrollments
<code>PRIMARY KEY (student_id, course_id)</code>	a composite key: the same student cannot enroll in the same course twice

Now the junction table lets us answer "who takes what" with two joins:

```
SELECT s.full_name, c.title, e.grade
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
ORDER BY s.full_name, c.title;
```

SQL · POSTGRESQL

OUTPUT

full_name	title	grade
Asha Rao	Database Systems	A
Asha Rao	Python Programming	B
Neha Iyer	Data Structures	A
Neha Iyer	Database Systems	B
Vikram Shah	Database Systems	NULL

5 rows

And `ON DELETE CASCADE` in action: deleting Asha removes her two enrollments automatically.

```
DELETE FROM students WHERE student_id = 1;
```

SQL · POSTGRESQL

```
SELECT student_id, course_id, grade
FROM enrollments
ORDER BY student_id, course_id;
```

OUTPUT

student_id	course_id	grade
2	DB101	NULL
3	DB101	B
3	DS201	A

3 rows

Table design best practices

Practice	Why
Every table has a primary key	rows can be identified, updated and joined reliably
Use the right data types: <code>NUMERIC(10,2)</code> for money, <code>DATE</code> / <code>TIMESTAMP</code> for time, <code>BOOLEAN</code> for flags	<code>FLOAT</code> rounds money ($0.1 + 0.2 \neq 0.3$); dates stored as text cannot be compared or validated
One fact per column, one value per cell	no "Mouse, Keyboard" lists; they break searching and violate 1NF
Declare foreign keys and <code>NOT NULL</code> where required	the database enforces correctness for every application
Consistent naming (<code>snake_case</code> , clear names like <code>customer_id</code>)	readable queries, fewer mistakes
Don't store what you can calculate cheaply (age, totals) unless you have a reason	avoids stale values (see Session 2.6 for when to break this rule)
Add audit columns such as <code>created_at</code> , <code>updated_at</code>	debugging, incremental loads, history

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"To model a many-to-many relationship, like students and courses, I create a junction table, say `enrollments` , with two foreign keys: one to students and one to courses. The pair is the composite primary key, so a student can't enroll twice in the same course, and the table can also hold attributes of the relationship itself, like the enrollment date and grade. One-to-many is simpler: the foreign key goes on the many side, so `orders` gets a `customer_id` ."

COMMON TRAPS & MISTAKES

- Putting the foreign key on the wrong side of a 1:N relationship (for example, an `order_id` column in `customers`).
- Modeling M:N with comma-separated IDs in one column instead of a junction table.
- Using `FLOAT` for currency.
- Storing dates as text (`'05/03/2024'`): is that 5 March or 3 May?

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a relation in the relational model?

A A table: a set of tuples (rows) that share the same attributes (columns).

Q2 **HOW** How do you implement a one-to-many relationship?

A Put a foreign key on the "many" side that references the primary key of the "one" side, e.g. `orders.customer_id` → `customers.customer_id` .

Q3 **HOW** How do you implement a many-to-many relationship?

A With a junction table holding foreign keys to both tables, usually forming a composite primary key, e.g. `enrollments(student_id, course_id)` .

INTERMEDIATE

Q4 **HOW** How do you implement a one-to-one relationship?

A Put a foreign key with a UNIQUE constraint in one table, or make the child's primary key also a foreign key to the parent (`user_profiles.user_id`).

Q5 **DEFINITION** What is a schema in PostgreSQL?

A A namespace inside a database that groups tables, views and functions; the default is `public` . "Schema" also means the overall design of the tables.

Q6 **WHY** Why should money not be stored as FLOAT?

A Floating-point numbers are binary approximations and produce rounding errors; `NUMERIC` / `DECIMAL` stores exact values.

SCENARIO-BASED

Q7 **DESIGN QUESTION** Design tables for a library where books can have multiple authors and members borrow books.

A `books(book_id PK, title, isbn UNIQUE)` , `authors(author_id PK, name)` , `book_authors(book_id FK, author_id FK, PK both)` , `members(member_id PK, name, email UNIQUE)` , `loans(loan_id PK, member_id FK, book_id FK, loan_date, due_date, return_date NULL)` .

Q8 **SCENARIO** A colleague stores `course_ids` as "DB101,PY101" in the students table. What's wrong and how do you fix it?

A It breaks 1NF, cannot be indexed or joined properly, and allows invalid IDs. Replace it with an `enrollments` junction table with foreign keys to both sides.

FOLLOW-UP / TRAP

Q9 **TRAP QUESTION** Are rows in a table stored in insertion order?

A You must not rely on it. A relation is an unordered set; use `ORDER BY` whenever order matters.

Q10 **TRAP QUESTION** Can a junction table have extra columns?

A Yes. Attributes of the relationship itself, such as enrollment date and grade, belong there.

Session 2.5 — Normalization

HIGH PRIORITY Expect at least one normalization question in almost every DBMS interview.

Simple meaning

Normalization means splitting data into sensible tables so the same fact is not stored over and over. Each fact lives in exactly one place, and tables are linked with keys.

Why: the three anomalies

Look at this "everything in one sheet" table. It works, until you try to change it:

ORDER_DETAILS_FLAT (UNNORMALIZED)

order_id	order_date	customer_name	customer_phone	customer_city	product_name	unit_price	qty
101	2024-01-05	Aarav Patel	98200 11111	Mumbai	Wireless Mouse	799	2
101	2024-01-05	Aarav Patel	98200 11111	Mumbai	Notebook Pack	249	5
103	2024-01-28	Aarav Patel	98200 11111	Mumbai	Office Chair	8999	1
102	2024-01-12	Isha Kulkarni	98900 22222	Pune	Mechanical Keyboard	3499	1

Anomaly	What goes wrong here
Update anomaly	Aarav changes his phone number. It is stored in three rows; miss one and the data contradicts itself.
Insert anomaly	We cannot record a new product, "Monitor Arm", until somebody orders it, because there is no order_id to put in the row.
Delete anomaly	If order 102 is deleted, we also lose the only record that Isha exists <i>and</i> that a keyboard costs 3499.

All three come from the same root cause: **one table is storing facts about several different things** (orders, customers, products).

Functional dependency: the idea behind normalization

$X \rightarrow Y$ (read "X determines Y") means: **if you know X, you know exactly one Y.**

- $\text{customer_id} \rightarrow \text{customer_name}, \text{customer_phone}, \text{customer_city}$
- $\text{product_id} \rightarrow \text{product_name}, \text{unit_price}$
- $\text{order_id} \rightarrow \text{order_date}, \text{customer_id}$
- $(\text{order_id}, \text{product_id}) \rightarrow \text{qty}$

Three kinds of dependency matter:

Kind	Meaning	Example
Full	a column depends on the <i>whole</i> key	qty depends on $(\text{order_id}, \text{product_id})$ together
Partial	a column depends on only <i>part</i> of a composite key	product_name depends only on product_id , not on order_id
Transitive	a non-key column depends on another non-key column	$\text{order_id} \rightarrow \text{customer_id} \rightarrow \text{customer_city}$

Normalization removes partial and transitive dependencies step by step.

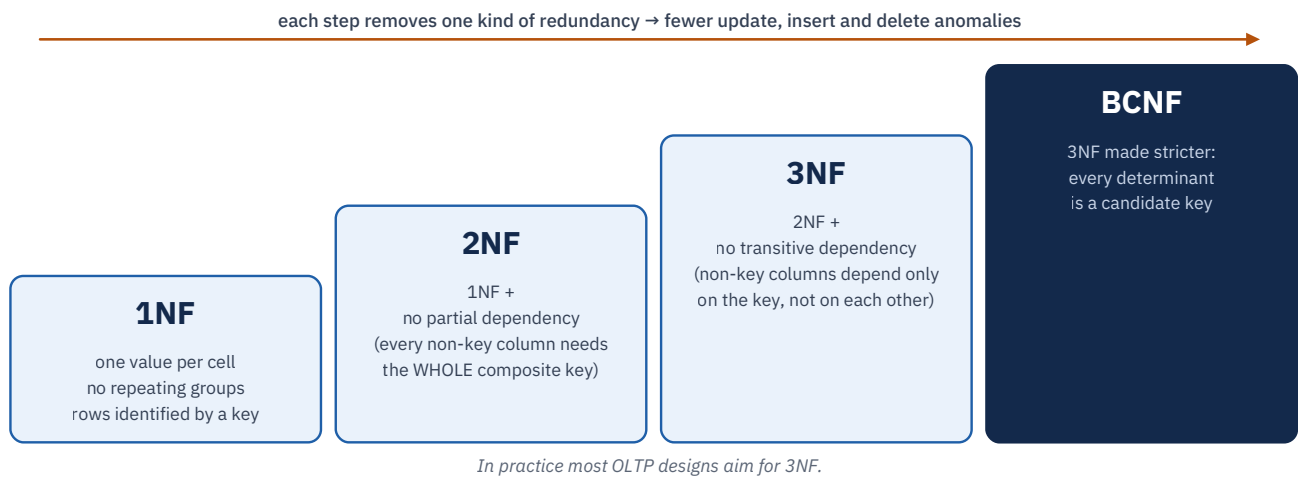


Figure 9. The normal forms build on each other; each removes one kind of redundancy.

First Normal Form (1NF)

Rule: every cell holds **one atomic value**, there are **no repeating groups**, and each row can be identified by a key.

Before (not 1NF), one cell holds a list:

order_id	customer_name	products
101	Aarav Patel	Wireless Mouse, Notebook Pack

After (1NF), one product per row. The key becomes (order_id, product_name), or better (order_id, product_id):

order_id	customer_name	product_name
101	Aarav Patel	Wireless Mouse
101	Aarav Patel	Notebook Pack

Second Normal Form (2NF)

Rule: 1NF **and** no partial dependency. Every non-key column must depend on the **whole** primary key. (This only matters when the key is composite.)

In `order_lines(order_id, product_id, qty, product_name, unit_price)`, the key is (order_id, product_id), but `product_name` and `unit_price` depend only on `product_id`. Move them out:

order_lines (order_id, product_id, qty)	products (product_id, product_name, unit_price)
(101, 1, 2), (101, 6, 5), (103, 4, 1)	(1, Wireless Mouse, 799), (6, Notebook Pack, 249), (4, Office Chair, 8999)

Third Normal Form (3NF)

Rule: 2NF **and** no transitive dependency. Non-key columns depend **only on the key**, not on other non-key columns.

In `orders(order_id, order_date, customer_id, customer_city)`, `customer_city` depends on `customer_id`, which is not the key. Move customer details to `customers`:

orders (order_id, order_date, customer_id)	customers (customer_id, customer_name, phone, city)
(101, 2024-01-05, 1), (103, 2024-01-28, 1)	(1, Aarav Patel, 98200 11111, Mumbai)

A memory line many interviewers like: *every non-key attribute must depend on "the key, the whole key, and nothing but the key".*

Boyce–Codd Normal Form (BCNF)

Rule: for every functional dependency $X \rightarrow Y$, **X must be a super key**. BCNF is a stricter 3NF; the difference shows up only when a table has overlapping candidate keys.

Example: `tutoring(student, subject, teacher)`. Each teacher teaches exactly one subject (`teacher → subject`), and each student has one teacher per subject. The candidate keys are `(student, subject)` and `(student, teacher)`. The table is in 3NF, but `teacher → subject` has a determinant (`teacher`) that is not a super key, so it is **not** in BCNF. The fix is to split it into `teacher_subject(teacher, subject)` and `student_teacher(student, teacher)`.

4NF AND 5NF

4NF removes multi-valued dependencies (independent lists stored in one table, like an employee's skills and languages in the same table). **5NF** deals with join dependencies. They are rarely asked at entry level; knowing their names and the one-line idea is enough.

The summary table to memorize

Normal form	Rule	Removes	Quick test
1NF	atomic values, no repeating groups, has a key	multi-valued cells	"Any cell with a list?"
2NF	1NF + no partial dependency	facts about <i>part</i> of a composite key	"Does a column depend on only part of the key?"
3NF	2NF + no transitive dependency	facts about non-key columns	"Does a column depend on another non-key column?"
BCNF	every determinant is a super key	remaining anomalies with overlapping keys	"Is there an $X \rightarrow Y$ where X isn't a key?"

The result: the handbook's sample database is the normalized version

Normalizing `order_details_flat` gives exactly the four tables this handbook uses everywhere: `customers`, `orders`, `products` and `order_items`. Nothing is lost: a join rebuilds the flat view whenever we need it. This is called a **lossless decomposition**.

```
SELECT o.order_id, o.order_date, c.customer_name, c.city,
       p.product_name, i.unit_price, i.quantity
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items i ON i.order_id = o.order_id
JOIN products p ON p.product_id = i.product_id
WHERE o.order_id IN (101, 102, 103)
ORDER BY o.order_id, p.product_name;
```

SQL · POSTGRES SQL

OUTPUT

order_id	order_date	customer_name	city	product_name	unit_price	quantity
101	2024-01-05	Aarav Patel	Mumbai	Notebook Pack	249.00	5
101	2024-01-05	Aarav Patel	Mumbai	Wireless Mouse	799.00	2
102	2024-01-12	Isha Kulkarni	Pune	Mechanical Keyboard	3499.00	1
103	2024-01-28	Aarav Patel	Mumbai	Office Chair	8999.00	1

4 rows

LINE BY LINE

Line	What it does
FROM orders o	start from orders; o is a short alias
JOIN customers c ON c.customer_id = o.customer_id	attach each order's customer
JOIN order_items i ON i.order_id = o.order_id	attach the order's lines (one row per product)
JOIN products p ON p.product_id = i.product_id	attach product names
WHERE o.order_id IN (101, 102, 103)	only three orders, to keep the output short
ORDER BY ...	predictable order for reading

Aarav's phone or city now lives in **one** row of `customers` ; changing it fixes every order at once.

Advantages and disadvantages

Advantages	Disadvantages
No update, insert or delete anomalies	More tables, so queries need joins
Less storage wasted on duplicates	Very heavy reporting queries can get slower
Data stays consistent	Over-normalizing can make the design hard to use
Easier to change one fact	Requires understanding of the data's dependencies

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Normalization is a design technique that reduces duplicate data and prevents update, insert and delete anomalies. We split a table so that each fact is stored once and connect the pieces with keys. In 1NF every cell has one value; in 2NF every non-key column depends on the whole key; in 3NF non-key columns depend only on the key, not on each other. For example, instead of repeating a customer's phone and city on every order, I keep a customers table and store just the customer_id in orders. For OLTP systems I normally aim for 3NF."

COMMON TRAPS & MISTAKES

- Saying 2NF applies to every table. Partial dependency is only possible with a **composite** key; a table with a single-column key in 1NF is automatically in 2NF.
- Confusing 2NF (partial dependency on part of the key) with 3NF (dependency between non-key columns).
- Claiming normalization always makes queries faster. Writes and consistency improve; complex reads may need more joins.
- Thinking "more normalization is always better". Warehouses deliberately denormalize.

INTERVIEW QUESTIONS**BASIC**

Q1 **DEFINITION** What is normalization?

A Organising data into related tables to remove redundancy and avoid insert, update and delete anomalies, linking the tables with keys.

Q2 DEFINITION What is 1NF?

A Every column holds atomic (single) values, there are no repeating groups, and each row is uniquely identifiable.

Q3 DEFINITION What is a functional dependency?

A $X \rightarrow Y$ means a value of X determines exactly one value of Y , e.g. $\text{customer_id} \rightarrow \text{customer_name}$.

Q4 DEFINITION What are update, insert and delete anomalies?

A Update: the same fact stored in many rows becomes inconsistent when only some are changed. Insert: you cannot add a fact without unrelated data. Delete: removing a row unintentionally removes another fact.

INTERMEDIATE**Q5 COMPARISON What is the difference between 2NF and 3NF?**

A 2NF removes partial dependencies (a column depending on part of a composite key). 3NF removes transitive dependencies (a non-key column depending on another non-key column).

Q6 COMPARISON What is the difference between 3NF and BCNF?

A BCNF requires every determinant to be a super key. 3NF allows an exception when the dependent column is part of a candidate key, so a table can be in 3NF but not in BCNF when it has overlapping candidate keys.

Q7 DEFINITION What is a transitive dependency? Give an example.

A $A \rightarrow B$ and $B \rightarrow C$, so $A \rightarrow C$ indirectly through a non-key column. Example: $\text{emp_id} \rightarrow \text{dept_id} \rightarrow \text{dept_name}$ in an employees table.

Q8 DEFINITION What is a lossless decomposition?

A Splitting a table into smaller tables such that joining them back gives exactly the original data, with no lost or extra rows.

SCENARIO-BASED**Q9 SCENARIO You have a table `orders(order_id, customer_name, customer_email, product, price, qty)`. How would you redesign it?**

A Split it into `customers(customer_id, name, email UNIQUE)`, `products(product_id, name, price)`, `orders(order_id, customer_id FK, order_date)` and `order_items(order_id FK, product_id FK, qty, unit_price, PK(order_id, product_id))`.

Q10 SCENARIO `employees(emp_id, emp_name, dept_id, dept_name, dept_location)`: which normal form is violated?

A 3NF, because `dept_name` and `dept_location` depend on `dept_id` (a non-key column). Move them into `departments(dept_id, dept_name, dept_location)`.

Q11 SCENARIO In `enrollments(student_id, course_id, course_title, grade)`, what's wrong?

A `course_title` depends only on `course_id`, which is part of the composite key, so it is a partial dependency and violates 2NF. Move it to `courses`.

FOLLOW-UP / TRAP**Q12 TRAP QUESTION Can normalization ever hurt performance?**

A Yes. Read-heavy reports may need many joins. That is why analytical systems denormalize, and OLTP systems sometimes keep a few derived columns or materialized views.

Q13 TRAP QUESTION Is a table with a single-column primary key always in 2NF?

A If it is already in 1NF, yes. Partial dependency needs a composite key.

Q14 TRAP QUESTION Up to which normal form do you normalize in practice?

A Usually 3NF (sometimes BCNF) for transactional systems; beyond that the benefit is rarely worth the complexity.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. In one sentence each, define 1NF, 2NF and 3NF.

P2. Name the anomaly: "We cannot add a new course to the database until a student enrolls in it."

P3. Write two functional dependencies that hold in the sample `employees` table.

LEVEL 2 – INTERVIEW APPLICATION

P4. Is this table in 1NF? `students(student_id, name, phone_numbers)` where `phone_numbers = "98200 11111, 98900 22222"`. Fix it.

P5. Explain to an interviewer why `order_items(order_id, product_id, qty, product_name)` is not in 2NF.

P6. Give an example of a transitive dependency from a hospital database.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Normalize this table to 3NF and list the final tables with their keys: `invoice(invoice_no, invoice_date, customer_id, customer_name, customer_gstin, item_code, item_desc, item_rate, qty)`.

P8. A table `exam_results(student_id, subject, marks, teacher_name, teacher_phone)` is causing problems when teachers change phone numbers. Explain the anomaly and redesign it.

ANSWER / EXPLANATION

P1. 1NF: every cell holds a single atomic value and rows are uniquely identifiable. 2NF: in 1NF, and every non-key column depends on the whole primary key (no partial dependency). 3NF: in 2NF, and no non-key column depends on another non-key column (no transitive dependency).

P2. Insert anomaly.

P3. `emp_id → emp_name, email, dept_id, salary, hire_date, city` (`emp_id` is the primary key, so it determines every column), and `email → emp_id` (`email` is unique, so knowing it identifies the employee).

P4. No, the phone column holds a list. Create `student_phones(student_id FK, phone, PRIMARY KEY (student_id, phone))` and remove the list column from `students`.

P5. "The key is the pair (`order_id, product_id`), but `product_name` depends only on `product_id`, which is part of the key. That's a partial dependency, so it violates 2NF. If a product is renamed we'd have to update every order line. Product name belongs in the products table."

P6. `appointments(appointment_id, doctor_id, doctor_department)` : `appointment_id → doctor_id → doctor_department`. The department depends on the doctor, not on the appointment.

P7. Dependencies: `invoice_no → invoice_date, customer_id`; `customer_id → customer_name, customer_gstin`; `item_code → item_desc, item_rate`; (`invoice_no, item_code`) $\rightarrow$ `qty`. Final tables: `customers(customer_id PK, customer_name, customer_gstin UNIQUE)`; `items(item_code PK, item_desc, item_rate)`; `invoices(invoice_no PK, invoice_date, customer_id FK)`; `invoice_lines(invoice_no FK, item_code FK, qty, rate_charged, PK(invoice_no, item_code))`. Storing `rate_charged` on the line is deliberate: the price on an old invoice must not change when today's rate changes.

P8. Teacher details are repeated on every result row, so a phone change must be applied to many rows: an update anomaly. If a teacher's only result row is deleted, their phone is lost: a delete anomaly. Redesign: `teachers(teacher_id PK, teacher_name, teacher_phone)`, `subjects(subject_id PK, subject_name, teacher_id FK)` (if each subject has one teacher), `exam_results(student_id FK, subject_id FK, marks, PK(student_id, subject_id))`.

Session 2.6 — Denormalization: When and Why

Simple meaning

Denormalization means deliberately storing some data more than once, or storing pre-calculated results, so that reading becomes faster or simpler. It is a conscious trade: you accept extra storage and extra work on writes in exchange for quicker reads.

Normalization is the default for transactional systems. Denormalization is a targeted exception, applied where the reads justify it.

Why we denormalize

- **Joins cost time at scale.** Joining five large tables for every page view adds up.
- **Read-heavy workloads.** If a value is read a million times and changed once a day, pre-computing it makes sense.
- **Reporting and analytics.** Analysts want wide, simple tables (Session 2.7).
- **Distributed and NoSQL databases** often cannot join efficiently, so related data is stored together (Session 2.8).

Common denormalization techniques

Technique	Example	Cost you accept
Store a derived value	<code>orders.total_amount</code> instead of summing <code>order_items</code> every time	must be kept in sync when items change
Copy a column into another table	store <code>customer_name</code> on the invoice	name can drift from the customer table, which is sometimes <i>wanted</i> : the invoice should show the name at the time of sale
Summary (aggregate) table	<code>daily_sales(date, revenue)</code>	refresh jobs, possible staleness
Materialized view	a stored, refreshable query result	refresh cost; data only as fresh as the last refresh
Star schema	wide dimension tables in a warehouse	duplication inside dimensions

Example 1: a derived column in the sample database

The sample database itself is slightly denormalized: `orders.total_amount` repeats information that could be computed from `order_items`. That makes listing orders fast, but now we must check that the two never disagree. This query looks for any order whose stored total differs from its items:

```
SELECT o.order_id,
       o.total_amount,
       SUM(i.quantity * i.unit_price) AS items_total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING o.total_amount <> SUM(i.quantity * i.unit_price);
```

SQL · POSTGRESQL

OUTPUT

order_id	total_amount	items_total
----------	--------------	-------------

0 rows

Zero rows means the copies are consistent. In production this check (or a trigger that recalculates the total) is part of the price of denormalizing.

Example 2: a materialized view

A **view** is a saved query that runs every time you read it. A **materialized view** stores the query's *result*, so reading it is as cheap as reading a table, until you refresh it.

```
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT date_trunc('month', order_date)::date AS month,
       COUNT(*) AS orders,
       SUM(total_amount) AS revenue
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY 1;

SELECT * FROM monthly_sales ORDER BY month;
```

SQL · POSTGRESQL

OUTPUT

month	orders	revenue
2024-01-01	3	15341.00
2024-02-01	2	5284.00
2024-03-01	4	38391.00
2024-04-01	4	42730.00

4 rows

LINE BY LINE

Line	What it does
CREATE MATERIALIZED VIEW monthly_sales AS	saves the result of the query below as a stored object
date_trunc('month', order_date)::date AS month	cuts each date down to the first day of its month, e.g. 2024-01-28 → 2024-01-01
COUNT(*) AS orders, SUM(total_amount) AS revenue	number of orders and revenue per month
WHERE status <> 'CANCELLED'	cancelled orders are not revenue
GROUP BY 1	group by the first column in the SELECT list (the month)
SELECT * FROM monthly_sales	reading it now costs no aggregation at all

When new orders arrive, `REFRESH MATERIALIZED VIEW monthly_sales;` recomputes it, typically on a schedule.

Normalize or denormalize? A decision table

Situation	Lean towards	Why
OLTP: orders, payments, bookings	Normalize (3NF)	correctness and cheap updates matter most
Write-heavy data	Normalize	each write touches one place
Read-heavy pages with stable data (product listing)	Denormalize selectively	fewer joins per page view
Data warehouse / BI	Denormalize (star schema)	simple, fast analytical queries
NoSQL document store	Denormalize (embed)	joins are limited; read patterns drive design
Legal or historical snapshot (invoice)	Copy on purpose	the value at that moment must be preserved

	Normalization	Denormalization
Goal	remove redundancy, protect consistency	speed up reads, simplify queries
Number of tables	more	fewer / wider
Writes	simple and cheap	more work: several copies to update
Reads	may need many joins	fewer joins
Risk	slower complex reads	inconsistent copies, more storage
Typical home	OLTP	OLAP, caches, NoSQL

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Denormalization is intentionally adding redundancy, like storing an order total or copying a customer's name, to make reads faster or simpler. It is the opposite trade-off to normalization: reads get cheaper, but writes get more expensive and we risk copies going out of sync, so we need triggers, refresh jobs or checks. I'd normalize transactional data to 3NF first, and then denormalize specific read-heavy paths, use materialized views or summary tables, or use a star schema in the warehouse."

COMMON TRAPS & MISTAKES

- Treating denormalization as "not knowing normalization". A good answer says you normalize first, then denormalize *on purpose*, with a reason.
- Forgetting the cost: someone must keep the copies in sync.
- Confusing a view (re-runs every time) with a materialized view (stored result, needs refresh).

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is denormalization?

- A** Deliberately adding redundant or pre-computed data to a design to make reads faster or simpler, at the cost of extra storage and more complex writes.

Q2 **COMPARISON** **View vs materialized view?**

- A** A view stores only the query and runs it on every read. A materialized view stores the result physically, so it is fast to read but must be refreshed to show new data.

INTERMEDIATE**Q3** **WHY** **When would you denormalize?**

- A** For read-heavy workloads where joins are a bottleneck, for reporting and warehouses, for NoSQL stores without efficient joins, and when a historical snapshot must be preserved (an invoice's price).

Q4 **HOW** **How do you keep denormalized data consistent?**

- A** Update all copies in the same transaction, use triggers, schedule refresh jobs for summary tables and materialized views, and run reconciliation checks that compare copies with the source.

SCENARIO-BASED**Q5** **SCENARIO** **Your product listing page joins six tables and is slow. What would you consider?**

- A** First check indexes and the query plan. If the joins are still the bottleneck, add a denormalized read model: a materialized view or a cached JSON document per product, refreshed when products change.

Q6 **SCENARIO** **Should an invoice store the product price or always look it up from the products table?**

- A** Store the price charged on the invoice line. Prices change; an old invoice must keep showing what was actually charged.

FOLLOW-UP / TRAP**Q7** **TRAP QUESTION** **Is denormalization the same as having no normalization?**

- A** No. Denormalization is a deliberate step taken *after* designing a normalized model, for a specific performance or simplicity reason.

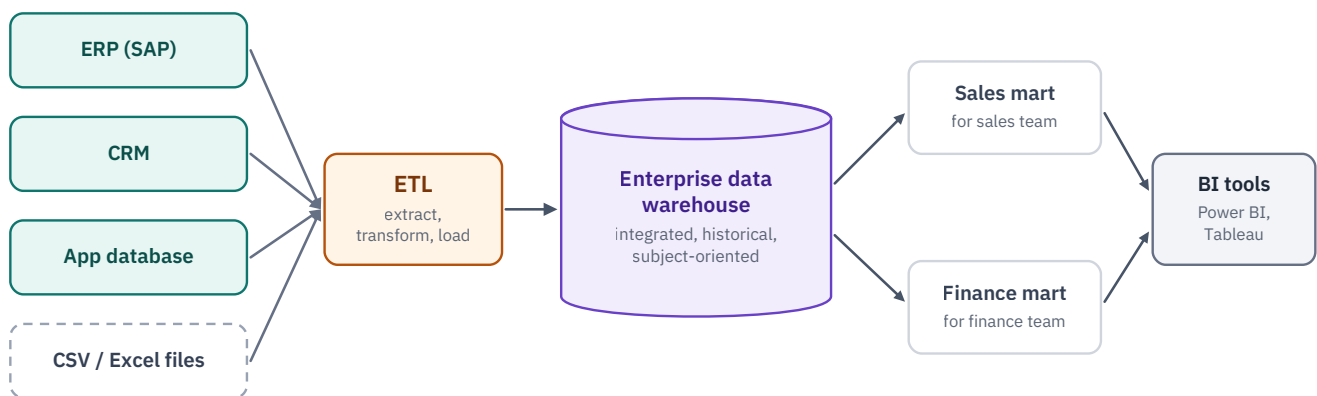
Session 2.7 — Data Warehousing and Dimensional Modeling

What is a data warehouse?

Simple meaning: a data warehouse is a central database built for **analysis**, not for running the business. It collects data from many operational systems, cleans and integrates it, and keeps years of history, so people can answer questions like "How did sales in each region change over the last three years?"

The classic definition (Bill Inmon) describes four properties. In plain words:

Property	Meaning
Subject-oriented	organised around business subjects (sales, customers, inventory), not around applications
Integrated	data from many sources made consistent (same codes, formats and definitions)
Time-variant	keeps history, so you can see how things changed over time
Non-volatile	data is loaded and read; it is not constantly updated or deleted like OLTP data



Schema-on-write: data is cleaned and modelled BEFORE it is loaded.

Figure 10. Classic warehouse architecture: sources → ETL → warehouse → data marts → BI tools.

- **Staging area:** a landing zone where raw extracts are cleaned before loading.
- **Data mart:** a smaller, focused slice of the warehouse for one department, such as a sales mart or finance mart.
- **BI tools:** Power BI, Tableau or Looker read from the warehouse or the marts.

Dimensional modeling: facts and dimensions

Warehouses are usually designed with **dimensional modeling** (Ralph Kimball's approach). Every analytical question has the same shape: *a number, sliced by some context*. "Revenue (number) by month (when), by product category (what), by city (where)."

- A **fact table** stores the **numbers** (measures) of a business event, plus foreign keys to its context. Example: `fact_sales(date_key, product_key, customer_key, quantity, sales_amount)`.
- A **dimension table** stores the **descriptive context**: who, what, where, when. Examples: `dim_date`, `dim_product`, `dim_customer`, `dim_store`.

	Fact table	Dimension table
Contains	measures (amount, quantity) + foreign keys	descriptive attributes (name, category, city)
Shape	very many rows, few columns (long and thin)	fewer rows, many columns (short and wide)
Changes	new rows appended for every event	changes slowly (a customer moves city)
Example row	(20240105, product 1, customer 1, qty 2, ₹1598)	(product 1, Wireless Mouse, Electronics)
Typical question it answers	"how much / how many?"	"by what / for whom / when / where?"

Grain: the first decision

The **grain** is what one row of the fact table represents, for example "one row per product per order line". Decide the grain *before* anything else; it determines which dimensions and measures make sense. Mixing grains (some rows per line, some per order) produces double counting.

Types of measures INTERVIEW EXTENSION

Type	Can you SUM it across everything?	Example
Additive	yes	sales amount, quantity
Semi-additive	across some dimensions, not time	account balance (summing balances across days is meaningless)
Non-additive	no	ratios and percentages (profit margin): recompute from components

Star schema

In a **star schema**, one fact table sits in the middle and each dimension connects to it directly, like the points of a star. Dimensions are **denormalized**: `dim_product` holds the product name, category and brand together.

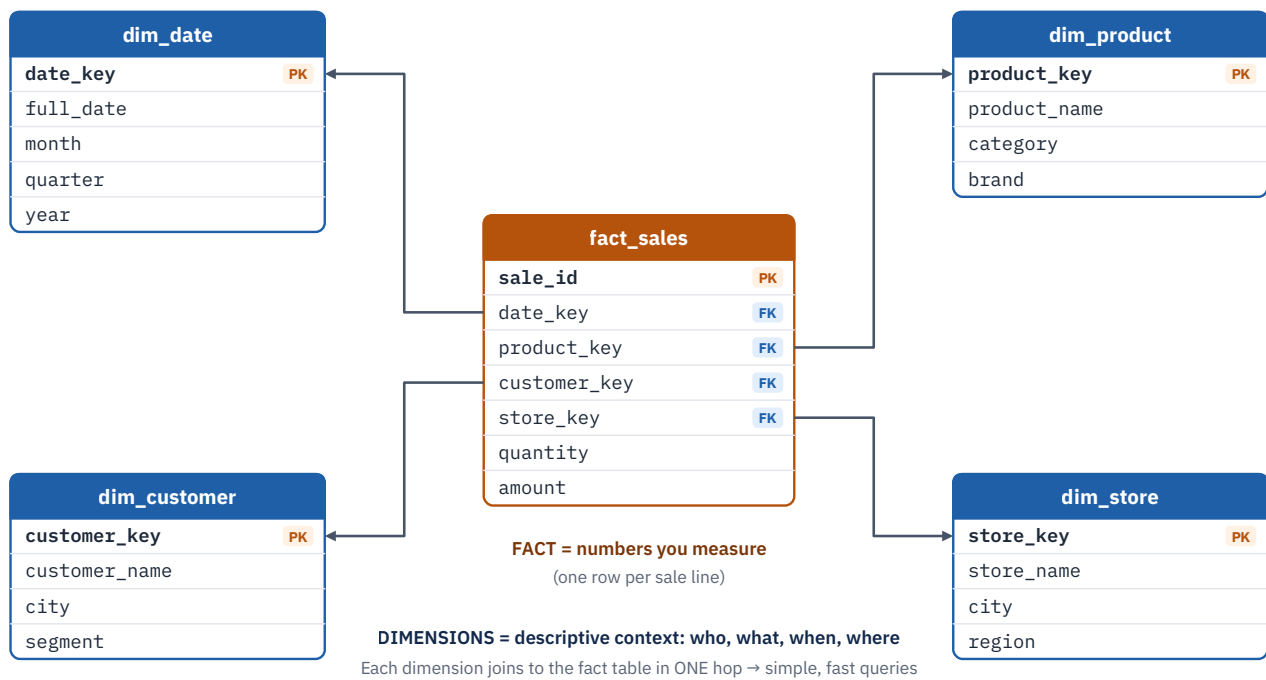


Figure 11. A star schema: every dimension is one join away from the fact table.

Building and querying a tiny star schema for real

The sample database is an OLTP design. The block below performs a mini ETL: it reshapes the OLTP tables into a star schema. In a real warehouse, a pipeline would do this, and the keys would be generated surrogate keys. Here the source IDs are reused to keep it short.

```
CREATE TABLE dim_date AS
SELECT DISTINCT
    to_char(order_date, 'YYYYMMDD')::int AS date_key,
    order_date AS full_date,
    to_char(order_date, 'Mon') AS month_name,
    EXTRACT(MONTH FROM order_date)::int AS month_no,
    EXTRACT(QUARTER FROM order_date)::int AS quarter,
    EXTRACT(YEAR FROM order_date)::int AS year
FROM orders;
ALTER TABLE dim_date ADD PRIMARY KEY (date_key);

CREATE TABLE dim_product AS
SELECT product_id AS product_key, product_name, category
FROM products;
ALTER TABLE dim_product ADD PRIMARY KEY (product_key);

CREATE TABLE dim_customer AS
SELECT customer_id AS customer_key, customer_name, city
FROM customers;
ALTER TABLE dim_customer ADD PRIMARY KEY (customer_key);

CREATE TABLE fact_sales AS
SELECT to_char(o.order_date, 'YYYYMMDD')::int AS date_key,
    i.product_id AS product_key,
    o.customer_id AS customer_key,
    i.quantity,
```

SQL · POSTGRESQL

```

        i.quantity * i.unit_price           AS sales_amount
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status <> 'CANCELLED';

```

The grain of `fact_sales` is one row per order line (cancelled orders excluded). A typical analytical question is "revenue and units by month and category":

```

SELECT d.month_name,
       p.category,
       SUM(f.sales_amount) AS revenue,
       SUM(f.quantity)    AS units
FROM fact_sales f
JOIN dim_date   d ON d.date_key   = f.date_key
JOIN dim_product p ON p.product_key = f.product_key
GROUP BY d.month_no, d.month_name, p.category
ORDER BY d.month_no, revenue DESC;

```

OUTPUT

month_name	category	revenue	units
Jan	Furniture	8999.00	1
Jan	Electronics	5097.00	3
Jan	Stationery	1245.00	5
Feb	Electronics	2798.00	2
Feb	Stationery	2486.00	14
Mar	Furniture	24998.00	2
Mar	Electronics	13393.00	7
Apr	Furniture	33997.00	3
Apr	Electronics	5498.00	2
Apr	Stationery	3235.00	15

10 rows

LINE BY LINE

Line	What it does
FROM fact_sales f	start from the numbers
JOIN dim_date d ... / JOIN dim_product p ...	each dimension is exactly one join away; that is the "star"
SUM(f.sales_amount) AS revenue, SUM(f.quantity) AS units	the measures, aggregated
GROUP BY d.month_no, d.month_name, p.category	slice by month and category; month_no is included so we can sort months correctly
ORDER BY d.month_no, revenue DESC	January first; inside each month, the biggest category first

Every star-schema query looks like this: **join the fact to the dimensions you need, filter and group by dimension attributes, aggregate the measures.**

Snowflake schema

A **snowflake schema** normalizes the dimensions into sub-dimensions: `dim_product` → `dim_category` → `dim_department`. The diagram branches out like a snowflake.

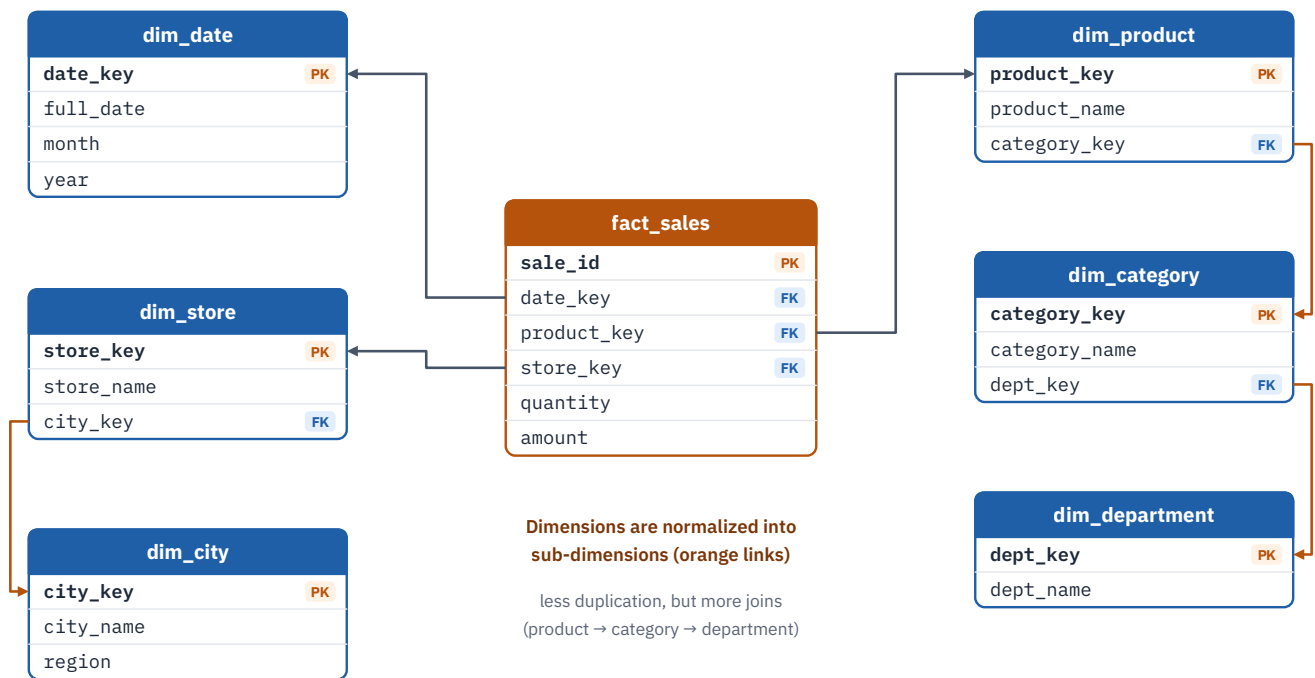


Figure 12. A snowflake schema: dimensions are split into further normalized tables.

	Star schema	Snowflake schema
Dimensions	denormalized, one table each	normalized into several tables
Joins per query	fewer (one hop)	more (several hops)
Query speed	usually faster	usually slower
Storage	some duplication inside dimensions	less duplication
Ease of use for analysts and BI tools	simpler	more complex
Maintenance of dimension data	updates may touch many rows	updates touch one place
Most common choice	yes, the default in most warehouses	when dimensions are huge or shared hierarchies matter

OLTP database vs data warehouse

	Operational database (OLTP)	Data warehouse (OLAP)
Purpose	run day-to-day transactions	analyse history and trends
Design	normalized (3NF)	dimensional (star/snowflake)
Data	current state	historical, time-stamped
Workload	many small reads and writes	few large reads and aggregations
Updates	constant inserts, updates, deletes	periodic loads, mostly appends
Users	applications, customers, staff	analysts, managers, data scientists
Examples	PostgreSQL, MySQL, Oracle	Snowflake, BigQuery, Redshift, Synapse

Slowly changing dimensions (SCD) INTERVIEW EXTENSION

What should happen in the warehouse when a customer moves from Mumbai to Pune? That is the **slowly changing dimension** problem.

Type	What happens	History kept?	Example use
Type 0	never change the value	original only	date of birth
Type 1	overwrite the old value	no	fixing a spelling mistake
Type 2	add a new row with validity dates; old row is closed	full history	customer city, for correct historical reports
Type 3	keep a "previous value" column	one level	<code>current_city</code> , <code>previous_city</code>

Type 2 is the one interviewers ask about. Each version of the customer gets its own surrogate key:

```
CREATE TABLE dim_customer_scd2 (
    customer_sk    INT PRIMARY KEY,      -- surrogate key: one per version
    customer_id    INT NOT NULL,         -- business key from the source system
    customer_name  VARCHAR(60) NOT NULL,
    city           VARCHAR(40) NOT NULL,
    valid_from     DATE NOT NULL,
    valid_to       DATE,                 -- NULL = still the current version
    is_current     BOOLEAN NOT NULL
);

INSERT INTO dim_customer_scd2 VALUES
(1, 1, 'Aarav Patel', 'Mumbai', '2023-11-05', '2024-02-29', false),
(2, 1, 'Aarav Patel', 'Pune', '2024-03-01', NULL, true),
(3, 2, 'Isha Kulkarni', 'Pune', '2023-12-12', NULL, true);
```

SQL · POSTGRES SQL

"Which city was Aarav in on 15 January 2024?" Sales made then should still count for Mumbai:

```
SELECT customer_sk, customer_name, city
FROM dim_customer_scd2
WHERE customer_id = 1
    AND DATE '2024-01-15' >= valid_from
    AND (valid_to IS NULL OR DATE '2024-01-15' <= valid_to);
```

SQL · POSTGRES SQL

OUTPUT

customer_sk	customer_name	city
1	Aarav Patel	Mumbai

1 row

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A data warehouse is a central store designed for analysis. It integrates data from many operational systems and keeps history. We usually model it dimensionally: fact tables hold the measurable events, like sales amount and quantity, with foreign keys to dimension tables that describe the context, such as date, product, customer and store. In a star schema each dimension joins directly to the fact table, which keeps queries simple and fast. A snowflake schema normalizes the dimensions further, saving space but adding joins. The first design decision is the grain, what one fact row means."

COMMON TRAPS & MISTAKES

- Putting descriptive text (product names) in the fact table instead of a dimension.
- Forgetting to define the grain, which leads to double counting.

- Summing a semi-additive measure such as account balance over time.
- Saying a snowflake schema is "more advanced, therefore better". The star is the usual default.
- Confusing a data warehouse (curated, modelled, schema-on-write) with a data lake (raw files, schema-on-read). See Session 4.7.

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a data warehouse?

A central, integrated, historical store of data designed for analysis and reporting, separate from the operational systems.

Q2 COMPARISON What is the difference between a fact table and a dimension table?

A fact table holds measures (numbers) of business events plus foreign keys; a dimension table holds descriptive attributes (who, what, when, where) used to filter and group the facts.

Q3 DEFINITION What is a star schema?

A dimensional design with one central fact table connected directly to denormalized dimension tables.

INTERMEDIATE

Q4 COMPARISON Star schema vs snowflake schema?

In a star, dimensions are denormalized single tables, so there are fewer joins and faster, simpler queries. In a snowflake, dimensions are normalized into sub-tables, so there is less redundancy but more joins.

Q5 DEFINITION What is the grain of a fact table?

The level of detail one row represents, for example one row per order line or one row per store per day.

Q6 DEFINITION What is a data mart?

A subject- or department-specific subset of the warehouse, such as a sales mart.

Q7 COMPARISON OLTP database vs data warehouse?

OLTP is normalized, holds current data and handles many small transactions; a warehouse is dimensional, holds history and handles large analytical queries.

SCENARIO-BASED

Q8 DESIGN QUESTION Design a star schema for a food-delivery company's order analytics.

A Fact: `fact_order_line(date_key, time_key, customer_key, restaurant_key, item_key, rider_key, quantity, item_amount, delivery_fee, delivery_minutes)` at the grain of one row per item ordered. Dimensions: date, time of day, customer (city, segment), restaurant (cuisine, area), menu item (category, veg/non-veg), rider.

Q9 SCENARIO A customer moved from Mumbai to Pune, and old sales now show under Pune. What went wrong and how do you fix it?

A The dimension was overwritten (SCD Type 1). Use SCD Type 2: close the old row with a `valid_to` date, insert a new row with a new surrogate key, and point new facts at the new key. Old facts keep pointing to the Mumbai version.

FOLLOW-UP / TRAP

Q10 TRAP QUESTION Why do warehouses use surrogate keys instead of source system IDs?

A To integrate several sources whose IDs may clash, to keep multiple versions of the same entity (SCD Type 2), and to stay independent of changes in source systems.

Q11 **TRAP QUESTION** Can you sum an account balance across months?

A No. Balance is semi-additive; sum it across accounts for one day, but use the closing or average balance across time.

Session 2.8 — NoSQL Data Modeling

Why NoSQL exists (the modeling view)

Relational databases are excellent, but some problems push against them:

- **Huge scale:** billions of rows and very high write rates, spread across many servers.
- **Changing structure:** products with completely different attributes (a laptop has RAM; a T-shirt has a size).
- **Known, simple access patterns:** "get a user's whole cart by user ID" a million times a second.

NoSQL databases relax some relational features (fixed schemas, joins, sometimes strict consistency) to gain flexibility and horizontal scale. Session 4.2 compares the database types; here we focus on **how you model data** in them, especially in document databases such as MongoDB.

Schema flexibility (and its catch)

- Relational: **schema-on-write**. The table structure is fixed first; every row must fit.
- Document stores: **flexible schema**. Documents in the same collection can have different fields.

```
// Two products in the same "products" collection
{ "_id": 1, "name": "ThinkPad E14", "category": "Laptop",
  "price": 62990, "specs": { "ram_gb": 16, "cpu": "Ryzen 5" } }

{ "_id": 2, "name": "Cotton Tee", "category": "Apparel",
  "price": 499, "sizes": ["S", "M", "L"], "colour": "navy" }
```

MONGODB SHELL (MONGOSH)

The catch: **"schemaless" does not mean "no design"**. The schema moves into your application code. Without discipline you get `"price": "499"` in one document and `"price": 499` in another. MongoDB supports **schema validation** rules to prevent this.

Embedding vs referencing

The central decision in document modeling: put related data **inside** one document (embed), or keep it in a separate document and store its ID (reference)?

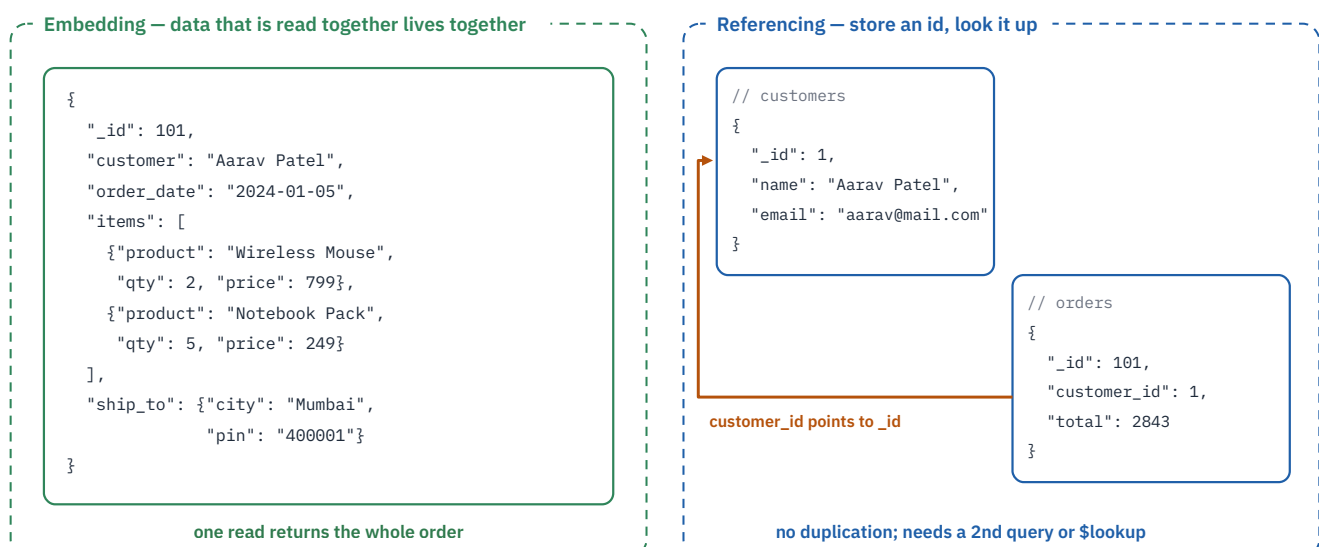


Figure 13. Embedding keeps data that is read together in one document; referencing links separate documents by ID.

	Embedding	Referencing
How	nested object or array inside the parent	store the other document's <code>_id</code>
Reads	one query returns everything	two queries, or a <code>\$lookup</code> (a join)
Writes	update one document (atomic)	update the shared document once, everywhere sees it
Duplication	data may be repeated across parents	no duplication
Growth	risky if the array grows without limit (MongoDB documents max out at 16 MB)	fine for large or unbounded sets
Use when	"contains" relationships, one-to-few, read together, owned by the parent	many-to-many, large or unbounded sets, data shared and updated independently

Rules of thumb:

- Order and its line items → **embed** (always read together, owned by the order, bounded).
- Order and customer → **reference** the customer (customer data is shared by many orders and changes independently). Optionally copy the name and address *as of the order* (denormalization for history).
- Blog post and comments → embed the latest few comments for fast display, and keep all comments in their own collection (a hybrid).
- Students and courses (many-to-many) → reference by ID on one or both sides.

Access-pattern-driven design

Relational modeling starts from the **data** ("what are the entities?") and normalizes. NoSQL modeling starts from the **queries** ("what will the app ask for, how often?") and shapes documents so the most frequent queries need a single read.

1. List the access patterns, for example: "show order with items" (very frequent), "list a customer's last 10 orders" (frequent), "monthly revenue report" (rare, can run in a warehouse).
2. Design documents so each frequent pattern is one read.
3. Add indexes for the filters you use (`customer_id`, `order_date`).
4. Accept some duplication where it saves joins, and decide how copies will be updated.

	Relational mindset	Document (NoSQL) mindset
Starting point	entities and normalization	access patterns (queries)
Duplication	avoid it	acceptable when it saves reads
Relationships	foreign keys + joins	embedding, or references with application-side joins
Schema	fixed, enforced by the DB	flexible, enforced by the app (plus optional validation)
Changing questions later	easy: new joins	harder: may need remodeling

A FOOD-DELIVERY ORDER AS ONE MONGODB DOCUMENT

MONGODB SHELL (MONGOSH)

```
{
  "_id": "ORD-9001",
  "customer": { "customer_id": 42, "name": "Diya Joshi" }, // snapshot copy
  "restaurant_id": 17, // reference
  "items": [ // embedded
    { "name": "Paneer Biryani", "qty": 1, "price": 249 },
    { "name": "Gulab Jamun", "qty": 2, "price": 60 }
  ],
  "delivery": { "address": "HSR Layout, Bengaluru", "status": "OUT_FOR_DELIVERY" },
  "total": 369,
  "created_at": ISODate("2024-04-19T19:42:00Z")
}
```

One read renders the whole order screen. The restaurant is referenced because its menu and details change independently; the customer's name is copied as it was at order time.

KEY-VALUE AND WIDE-COLUMN MODELING IN ONE PARAGRAPH

In key-value and wide-column stores (DynamoDB, Cassandra), you design around a **partition key** (which server holds the data, e.g. `customer_id`) and often a **sort key** (order inside the partition, e.g. `order_date`). Queries that do not use the partition key are expensive, so the table is shaped around exactly the queries you need. Some teams even store several entity types in one table ("single-table design").

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"In NoSQL, especially document databases like MongoDB, we model around access patterns instead of normalizing first. The main decision is embedding versus referencing. I embed data that belongs to the parent and is read with it, like an order's line items, because one read returns everything and the update is atomic. I reference data that is shared, large, unbounded or updated independently, like the customer or a restaurant, to avoid duplication and huge documents. Flexible schema doesn't mean no schema; the structure lives in the application, ideally backed by validation rules."

COMMON TRAPS & MISTAKES

- "NoSQL means no schema, so no design is needed." The design moves into the application and the access patterns.
- Embedding arrays that grow forever (all of a user's activity logs in the user document) and hitting the 16 MB document limit.
- Using MongoDB exactly like a relational database, with every relationship as a reference and `$lookup` everywhere. You lose the main benefit.
- Forgetting that duplicated (embedded) data must be updated in several places if it changes.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does "schema flexibility" mean in NoSQL?

A Records (documents) in the same collection can have different fields, and the structure can evolve without migrating a fixed table schema.

Q2 **COMPARISON** **Embedding vs referencing in MongoDB?**

- A** Embedding stores related data inside the parent document (one read, atomic updates, possible duplication); referencing stores the related document's ID (no duplication, but extra queries or `$lookup`).

INTERMEDIATE**Q3** **HOW** **How do you decide whether to embed or reference?**

- A** Embed when the data is owned by the parent, read together, and bounded (one-to-few). Reference when it is shared, many-to-many, large or unbounded, or updated independently.

Q4 **DEFINITION** **What is access-pattern-driven design?**

- A** Designing the data model starting from the application's most frequent queries, so each of them can be served efficiently, often with a single read.

Q5 **WHY** **Why is denormalization common in NoSQL?**

- A** Many NoSQL systems have limited or expensive joins, and they are optimised for fast single-key reads at scale, so related data is stored together.

SCENARIO-BASED**Q6** **DESIGN QUESTION** **Model a blog with posts and thousands of comments per post in MongoDB.**

- A** Store posts in a `posts` collection, possibly embedding the latest five comments for quick display; store all comments in a separate `comments` collection with `post_id` and an index on (`post_id`, `created_at`). Embedding every comment would make documents unbounded.

Q7 **SCENARIO** **A product catalogue has very different attributes per category. SQL or document model?**

- A** A document model fits well because each product can carry its own attributes. In PostgreSQL, a JSONB column for category-specific attributes is a good middle ground.

FOLLOW-UP / TRAP**Q8** **TRAP QUESTION** **Does MongoDB support joins?**

- A** Yes, the `$lookup` stage in the aggregation pipeline performs a left-outer-join-like operation, but heavy reliance on it usually means the model should embed more.

Q9 **TRAP QUESTION** **Is there a limit to embedding?**

- A** Yes. A MongoDB document can be at most 16 MB, and large, ever-growing arrays also make updates and reads slower.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Give one example each of an additive and a semi-additive measure.

P2. Which schema has fewer joins: star or snowflake?

P3. True or false: in MongoDB, two documents in the same collection must have the same fields.

LEVEL 2 – INTERVIEW APPLICATION

P4. Explain the difference between a view and a materialized view, and when you would use each.

P5. You must store a user and their three saved addresses in MongoDB. Embed or reference? Why?

P6. Name three things that go into a dimension table and three that go into a fact table for a retail store.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Design a star schema for a hospital that wants to analyse "number of appointments and revenue by doctor, department, month and patient age group". State the grain.

P8. Write a SQL query on this chapter's `fact_sales`, `dim_customer` and `dim_date` tables that returns revenue per customer city per quarter.

ANSWER / EXPLANATION

P1. Additive: sales amount (sum across any dimension). Semi-additive: inventory level or account balance (sum across products or stores, but not across days).

P2. The star schema.

P3. False. MongoDB has a flexible schema; documents can differ unless validation rules restrict them.

P4. A view is a saved query that runs on every read, so it is always current but costs the full query each time; use it to simplify or secure access. A materialized view stores the result, so it is fast to read but stale until refreshed; use it for expensive aggregations read often, such as dashboard summaries.

P5. Embed. The addresses belong to the user, there are only a few, and they are displayed together with the user, so one read and atomic updates are ideal.

P6. Dimension: product name and category, store city and region, the date's month and weekday. Fact: quantity sold, sales amount, discount amount.

P7. Grain: one row per appointment. Fact: `fact_appointment(date_key, doctor_key, department_key, patient_key, appointment_count = 1, consultation_fee, billed_amount)`. Dimensions: `dim_date` (day, month, quarter, year), `dim_doctor` (name, specialization, seniority), `dim_department` (name, building), `dim_patient` (gender, age_group, city). If age group changes over time, either store it in the fact at appointment time or use SCD Type 2.

P8. One possible answer:

```
SELECT c.city,
       d.year,
       d.quarter,
       SUM(f.sales_amount) AS revenue
FROM fact_sales f
JOIN dim_customer c ON c.customer_key = f.customer_key
JOIN dim_date      d ON d.date_key    = f.date_key
GROUP BY c.city, d.year, d.quarter
ORDER BY d.year, d.quarter, revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

city	year	quarter	revenue
Mumbai	2024	1	36840.00
Ahmedabad	2024	1	6998.00
Bengaluru	2024	1	6796.00
Pune	2024	1	5985.00
Delhi	2024	1	2397.00
Pune	2024	2	17998.00
Delhi	2024	2	15999.00
Mumbai	2024	2	5498.00
Ahmedabad	2024	2	3235.00

9 rows

All sample orders fall in Q1 and Q2 of 2024, so there are only two quarters.

Session 2.9 — Module 2 Summary & Rapid Revision

MODULE 2 SUMMARY

MOST IMPORTANT CONCEPTS

- **Data modeling** = entities, attributes, relationships and keys; conceptual → logical → physical.
- **Keys:** primary (unique + not null, one per table), foreign (points to a PK/unique key, enforces referential integrity), unique, composite, candidate, super, surrogate vs natural.
- **Relationships:** 1:N → FK on the many side; 1:1 → FK + UNIQUE; M:N → junction table.
- **Normalization:** removes update/insert/delete anomalies. 1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency; BCNF every determinant is a super key.
- **Denormalization:** deliberate redundancy for faster reads (derived columns, summary tables, materialized views, star schemas).
- **Warehousing:** facts (measures) + dimensions (context); grain first; star (default) vs snowflake; SCD Type 2 keeps history.
- **NoSQL modeling:** access-pattern-driven; embed what is owned and read together, reference what is shared or unbounded.

WHAT TO MEMORIZE

- "The key, the whole key, and nothing but the key" (1NF, 2NF, 3NF).
- PK vs FK vs UNIQUE differences, including NULL rules.
- Star = denormalized dimensions, fewer joins; snowflake = normalized dimensions, more joins.

WHAT TO UNDERSTAND

- *Why* anomalies happen (one table storing facts about several things).
- *When* to break normalization, and what it costs.
- *How* the grain controls a fact table's correctness.

MOST COMMON INTERVIEW QUESTIONS

- Normalize this table. • 2NF vs 3NF? • PK vs unique key? • Can a FK be NULL? • How to model many-to-many? • Star vs snowflake? • Fact vs dimension? • Embed or reference?

COMMON MISTAKES

- 2NF for single-column keys • FK on the wrong side • lists in a column • FLOAT for money • facts holding descriptive text • "NoSQL needs no design".

RAPID REVISION CHECKLIST

- | | |
|---|---|
| ☐ I can explain conceptual vs logical vs physical models | ☐ I can model 1:1, 1:N and M:N relationships |
| ☐ I can identify entities and relationships from a paragraph | ☐ I can name the three anomalies with examples |
| ☐ I know every type of key with an example | ☐ I can normalize a table to 3NF step by step |
| ☐ I can explain PK vs FK vs UNIQUE (and NULL rules) | ☐ I can explain BCNF in one sentence |
| ☐ I know the referential actions (CASCADE, SET NULL...) | ☐ I can justify when to denormalize |
| | ☐ I can design a star schema and state its grain |
| | ☐ I can compare star and snowflake schemas |
| | ☐ I can explain SCD Type 2 |
| | ☐ I can decide between embedding and referencing |

PART III • MODULE 3

Basics of SQL (Foundational)

SQL is the single most tested skill in data and software interviews, and it is where preparation pays off fastest. This module teaches SQL from the first SELECT to window functions, indexing and transactions, then drills the 22 query patterns that interviewers reuse again and again.

Every query in this module was **executed on PostgreSQL while this PDF was built**, and the output you see is the real output. Type the queries yourself: reading SQL and writing SQL are different skills.

COURSE COVERAGE

- ✓ **PostgreSQL Basics — Part 1:** PostgreSQL and psql, data types, SELECT, FROM, WHERE, DISTINCT, ORDER BY, LIMIT, OFFSET, aliases → Session 3.1
- ✓ **PostgreSQL Basics — Part 2:** comparison and logical operators, IN, BETWEEN, LIKE, NULL, aggregate, string, date and numeric functions, CASE → Session 3.2
- ✓ **DDL:** CREATE, ALTER, DROP, TRUNCATE, constraints → Session 3.3
- ✓ **DML:** INSERT, UPDATE, DELETE; also DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK) → Session 3.4
- ✓ Grouping and aggregation: GROUP BY, HAVING, WHERE vs HAVING † → Session 3.5
- ✓ Joins: inner, left, right, full outer, cross, self; referential integrity † → Session 3.6
- ✓ Subqueries and CTEs: scalar, correlated, in WHERE / FROM / SELECT; WITH; recursive CTE † → Session 3.7
- ✓ **Window Functions & Indexing:** OVER, PARTITION BY, ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, running and moving totals → Session 3.8; indexes, composite, clustered vs non-clustered → Session 3.9
- ✓ Transactions: COMMIT, ROLLBACK, ACID, isolation, dirty / non-repeatable / phantom reads † → Session 3.10
- ✓ Practice programs → the 22 SQL interview patterns, Session 3.11
- ✓ Module quiz concepts → interview questions in each session and Session 3.12

Topic	Priority
SELECT basics, filtering, NULL handling	HIGH PRIORITY
GROUP BY / HAVING, joins	HIGH PRIORITY MUST KNOW
Subqueries, CTEs	HIGH PRIORITY
Window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD)	HIGH PRIORITY
Indexing, transactions, ACID	HIGH PRIORITY
DDL, DML, DELETE vs TRUNCATE vs DROP	HIGH PRIORITY
Recursive CTEs, isolation levels	MEDIUM PRIORITY

PWC ANGLE

Public PwC interview write-ups repeatedly mention SQL joins and their types, GROUP BY queries, creating and altering tables, and DBMS basics such as ACID and normalization (see the PwC Interview Orientation chapter). Data-engineering candidates also report window functions and CTEs. If you master this module and the patterns in Session 3.11, you are covered for that range.

Session 3.0 – Meet the Sample Database

All examples in Modules 2 and 3 use one small, realistic database with an HR part, an e-commerce part and a few extra tables. Knowing its data makes every output easy to check. Bookmark this session.

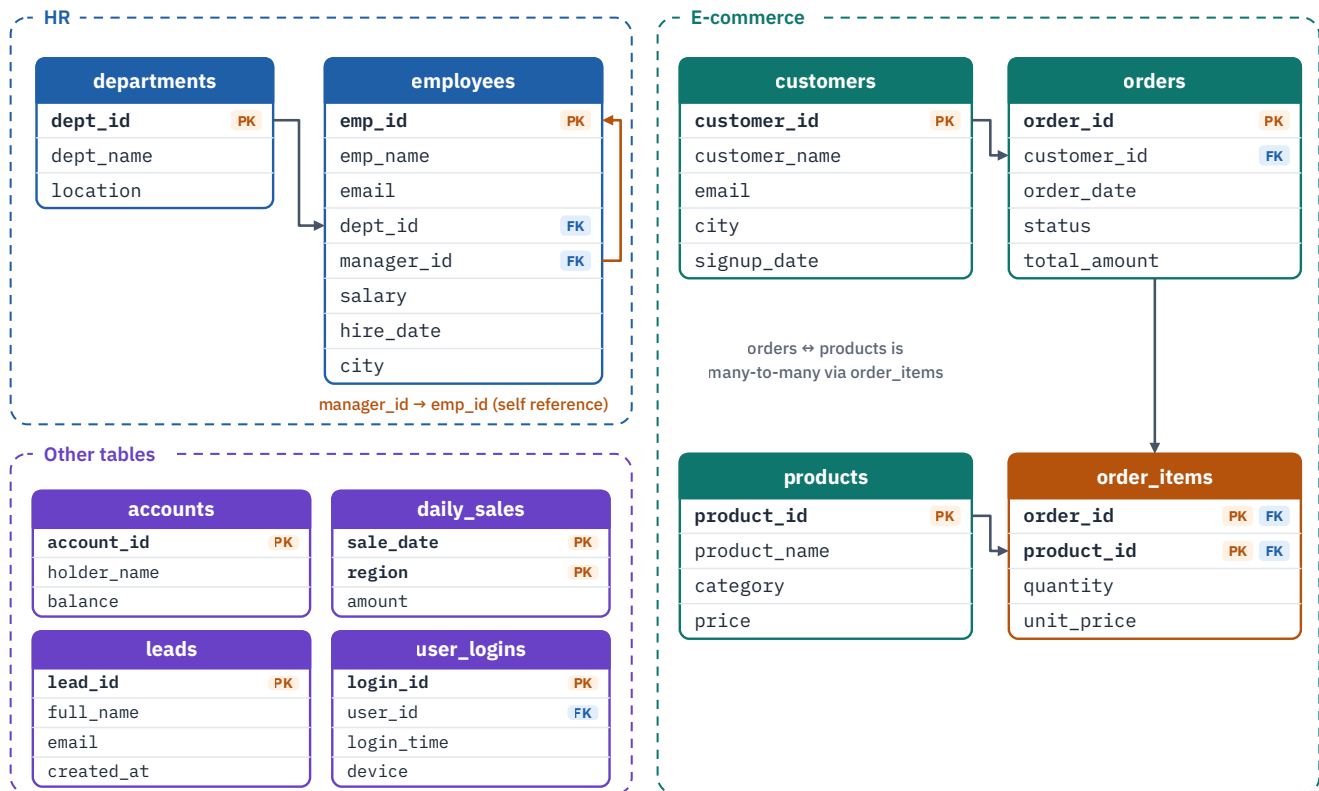


Figure 14. The sample database. Arrows point from a foreign key to the key it references.

Table	Rows	What it holds	Used for
departments	5	department name and city	joins, departments with no employees
employees	14	name, email, department, manager, salary, hire date, city	almost everything: filters, grouping, ranking, self joins
customers	7	e-commerce customers	joins, customers with no orders
products	8	catalogue with category and price	joins, products never sold
orders	14	order header: customer, date, status, total	dates, running totals, latest per customer
order_items	20	order lines: product, quantity, price	multi-table joins, many-to-many
accounts	3	bank balances	transactions and ACID
leads	7	marketing leads, with duplicates	finding and removing duplicates
daily_sales	14	sales per region per day	window functions
user_logins	10	app logins per customer	latest record, date analysis

Deliberate quirks (they make interview patterns possible):

- The **Legal** department has no employees.
- **Farhan Ali** (emp 13) has no department and no email (NULLs).
- Salaries contain **ties**: Rahul and Sneha both earn 95000; Karan and Meera both earn 70000.

- Customers **Saanvi** (6) and **Reyansh** (7) have never ordered; product **Monitor Arm** (8) was never sold.
- Order IDs **107** and **112** are missing (deleted orders).
- The `leads` table contains the same email several times.

The data you will see in outputs

```
SELECT * FROM employees ORDER BY emp_id;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name	email	dept_id	manager_id	salary	hire_date	city
1	Arjun Mehta	arjun.mehta@company.com	1	NULL	150000.00	2015-04-01	Bengaluru
2	Priya Sharma	priya.sharma@company.com	1	1	120000.00	2017-06-15	Bengaluru
3	Rahul Verma	rahul.verma@company.com	1	2	95000.00	2019-01-10	Pune
4	Sneha Iyer	sneha.iyer@company.com	1	2	95000.00	2020-03-23	Bengaluru
5	Vikram Rao	vikram.rao@company.com	1	2	80000.00	2021-07-01	Hyderabad
6	Ananya Das	ananya.das@company.com	2	1	110000.00	2016-09-12	Mumbai
7	Karan Malhotra	karan.malhotra@company.com	2	6	70000.00	2020-11-02	Mumbai
8	Meera Nair	meera.nair@company.com	2	6	70000.00	2022-02-14	Kochi
9	Rohan Gupta	rohan.gupta@company.com	3	1	65000.00	2018-05-21	Kolkata
10	Divya Menon	divya.menon@company.com	3	9	52000.00	2023-01-09	Kolkata
11	Aditya Singh	aditya.singh@company.com	4	1	90000.00	2019-08-19	Delhi
12	Neha Kapoor	neha.kapoor@company.com	4	11	60000.00	2021-12-06	Delhi
13	Farhan Ali	NULL	NULL	1	55000.00	2024-06-03	Pune
14	Pooja Reddy	pooja.reddy@company.com	1	3	72000.00	2024-02-12	Hyderabad

14 rows

```
SELECT * FROM departments ORDER BY dept_id;
```

SQL · POSTGRESQL

OUTPUT

dept_id	dept_name	location
1	Engineering	Bengaluru
2	Finance	Mumbai
3	HR	Kolkata
4	Marketing	Delhi
5	Legal	Hyderabad

5 rows

```
SELECT * FROM customers ORDER BY customer_id;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name	email	city	signup_date
1	Aarav Patel	aarav@mail.com	Mumbai	2023-11-05
2	Isha Kulkarni	isha@mail.com	Pune	2023-12-12
3	Kabir Khan	kabir@mail.com	Delhi	2024-01-03
4	Diya Joshi	diya@mail.com	Bengaluru	2024-01-20
5	Vivaan Shah	vivaan@mail.com	Ahmedabad	2024-02-08
6	Saanvi Rao	saanvi@mail.com	Chennai	2024-02-25
7	Reyansh Bose	NULL	Kolkata	2024-03-15

7 rows

```
SELECT * FROM products ORDER BY product_id;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name	category	price
1	Wireless Mouse	Electronics	799.00
2	Mechanical Keyboard	Electronics	3499.00
3	USB-C Hub	Electronics	1999.00
4	Office Chair	Furniture	8999.00
5	Standing Desk	Furniture	15999.00
6	Notebook Pack	Stationery	249.00
7	Gel Pen Set	Stationery	149.00
8	Monitor Arm	Furniture	2499.00

8 rows

```
SELECT * FROM orders ORDER BY order_id;
```

SQL · POSTGRESQL

OUTPUT

order_id	customer_id	order_date	status	total_amount
101	1	2024-01-05	DELIVERED	2843.00
102	2	2024-01-12	DELIVERED	3499.00
103	1	2024-01-28	DELIVERED	8999.00
104	3	2024-02-02	CANCELLED	15999.00
105	4	2024-02-14	DELIVERED	2798.00
106	2	2024-02-20	DELIVERED	2486.00
108	5	2024-03-03	SHIPPED	6998.00
109	1	2024-03-10	DELIVERED	24998.00
110	3	2024-03-18	DELIVERED	2397.00
111	4	2024-03-29	RETURNED	3998.00
113	2	2024-04-04	SHIPPED	17998.00
114	5	2024-04-11	PENDING	3235.00
115	1	2024-04-19	PENDING	5498.00
116	3	2024-04-19	DELIVERED	15999.00

14 rows

RUNNING THE EXAMPLES YOURSELF

Install PostgreSQL (or use any hosted PostgreSQL), create a database and load the script that ships with this handbook: `psql -d handbook -f handbook/sql/sample_db.sql`. The script drops and recreates all tables, so you can reload it whenever you want fresh data.

Session 3.1 — PostgreSQL Basics (Part 1)

What is PostgreSQL?

PostgreSQL ("Postgres") is a free, open-source relational database known for correctness and features: full ACID transactions, rich SQL support (CTEs, window functions), JSON support (`JSONB`), extensions and strong standards compliance. Companies use it for everything from startup apps to large enterprise systems, and every major cloud offers a managed version (Amazon RDS / Aurora, Google Cloud SQL, Azure Database for PostgreSQL).

How PostgreSQL organises things

cluster (server) → databases → schemas → tables, views, functions

- One PostgreSQL **server** can host many **databases** (`handbook` , `payroll`).
- Each database contains **schemas** (namespaces), `public` by default.
- Schemas contain **tables**, **views**, **sequences** and **functions**.

psql: the command-line client

`psql` is PostgreSQL's terminal client. Commands starting with a backslash are **psql meta-commands**: they are handled by the client, are not SQL, and do not end with a semicolon.

Command	What it does
<code>psql -h localhost -U postgres -d handbook</code>	connect to a database
<code>\l</code>	list databases
<code>\c handbook</code>	connect to another database
<code>\dt</code>	list tables in the current schema
<code>\d employees</code>	describe a table: columns, types, indexes, constraints
<code>\dn / \du</code>	list schemas / list users (roles)
<code>\i file.sql</code>	run a SQL script
<code>\x</code>	toggle expanded display (one column per line)
<code>\timing</code>	show how long each query takes
<code>\? / \h SELECT</code>	help on meta-commands / help on a SQL command
<code>\q</code>	quit

Common PostgreSQL data types

Category	Types	Notes
Whole numbers	<code>SMALLINT</code> , <code>INTEGER (INT)</code> , <code>BIGINT</code>	pick by range; IDs for big tables → <code>BIGINT</code>
Auto-numbering	<code>SERIAL</code> / <code>BIGSERIAL</code> , or <code>INT GENERATED ALWAYS AS IDENTITY</code>	<code>IDENTITY</code> is the SQL-standard, recommended form
Exact decimals	<code>NUMERIC(p, s)</code> / <code>DECIMAL(p, s)</code>	money: <code>NUMERIC(10,2)</code>
Approximate	<code>REAL</code> , <code>DOUBLE PRECISION</code>	scientific values, never money
Text	<code>VARCHAR(n)</code> , <code>TEXT</code> , <code>CHAR(n)</code>	<code>CHAR(n)</code> pads with spaces; <code>TEXT</code> has no length limit
True/false	<code>BOOLEAN</code>	<code>true</code> , <code>false</code> , or <code>NULL</code>
Date and time	<code>DATE</code> , <code>TIME</code> , <code>TIMESTAMP</code> , <code>TIMESTAMPTZ</code> , <code>INTERVAL</code>	<code>TIMESTAMPTZ</code> stores a moment in time with time-zone handling
Others	<code>UUID</code> , <code>JSON</code> / <code>JSONB</code> , arrays (<code>INT[]</code>)	<code>JSONB</code> is binary JSON that can be indexed

SQL is made of sub-languages

Sub-language	Purpose	Commands	Session
DQL (Data Query Language)	read data	<code>SELECT</code>	3.1–3.8
DDL (Data Definition Language)	define structure	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code>	3.3
DML (Data Manipulation Language)	change data	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	3.4
DCL (Data Control Language)	permissions	<code>GRANT</code> , <code>REVOKE</code>	3.4
TCL (Transaction Control Language)	transactions	<code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code>	3.4, 3.10

Some textbooks count `SELECT` as part of DML; either answer is accepted if you explain it.

SELECT and FROM: reading columns

```
SELECT emp_name, city, salary
FROM employees
ORDER BY emp_id
LIMIT 5;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Arjun Mehta	Bengaluru	150000.00
Priya Sharma	Bengaluru	120000.00
Rahul Verma	Pune	95000.00
Sneha Iyer	Bengaluru	95000.00
Vikram Rao	Hyderabad	80000.00

5 rows

- `SELECT` lists the columns you want; `FROM` names the table.
- `SELECT *` returns every column. It is fine for exploring, but avoid it in application code: it fetches more data than needed and breaks when columns change.

Expressions and aliases

A SELECT can compute new values. **Aliases** (AS) give columns and tables readable names.

```
SELECT e.emp_name,
       e.salary,
       e.salary * 12 AS annual_salary,
       e.salary * 12 * 0.10 AS annual_bonus
FROM employees AS e
ORDER BY annual_salary DESC
LIMIT 4;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary	annual_salary	annual_bonus
Arjun Mehta	150000.00	1800000.00	180000.0000
Priya Sharma	120000.00	1440000.00	144000.0000
Ananya Das	110000.00	1320000.00	132000.0000
Sneha Iyer	95000.00	1140000.00	114000.0000

4 rows

LINE BY LINE

Line	What it does
SELECT e.emp_name, e.salary,	two existing columns; e. says they come from the table aliased e
e.salary * 12 AS annual_salary	a calculated column, renamed with an alias
e.salary * 12 * 0.10 AS annual_bonus	another calculation: 10% of annual salary
FROM employees AS e	table alias: e now stands for employees (the AS is optional)
ORDER BY annual_salary DESC	ORDER BY can use a column alias, because it runs after SELECT
LIMIT 4	keep only the first four rows of the sorted result

An alias containing spaces or capitals needs double quotes: AS "Annual Salary". Single quotes are for **string values**, double quotes are for **names**.

WHERE: filtering rows

WHERE keeps only the rows for which the condition is true.

Operator	Meaning	Example
=	equal	city = 'Pune'
<> or !=	not equal	status <> 'CANCELLED'
<, >, <=, >=	comparisons	salary >= 90000

```
SELECT emp_name, city, salary
FROM employees
WHERE salary >= 90000
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Arjun Mehta	Bengaluru	150000.00
Priya Sharma	Bengaluru	120000.00
Ananya Das	Mumbai	110000.00
Rahul Verma	Pune	95000.00
Sneha Iyer	Bengaluru	95000.00
Aditya Singh	Delhi	90000.00

6 rows

Text comparisons are **case-sensitive** in PostgreSQL: `city = 'pune'` finds nothing. Text and date literals go in **single quotes**.

Why you cannot use a SELECT alias in WHERE

```
SELECT emp_name, salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 1000000;
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR column "annual_salary" does not exist

PostgreSQL evaluates **WHERE before SELECT**, so the alias does not exist yet. Repeat the expression (`WHERE salary * 12 > 1000000`) or use a subquery/CTE. The full logical order is explained in Session 3.5.

DISTINCT: removing duplicate rows

```
SELECT DISTINCT city
FROM employees
ORDER BY city;
```

SQL · POSTGRESQL

OUTPUT

city
Bengaluru
Delhi
Hyderabad
Kochi
Kolkata
Mumbai
Pune

7 rows

DISTINCT applies to the **whole row of selected columns**. `SELECT DISTINCT dept_id, city` returns each unique *combination* of department and city.

DISTINCT ON (POSTGRESQL ONLY)

`SELECT DISTINCT ON (dept_id) dept_id, emp_name, salary FROM employees ORDER BY dept_id, salary DESC;` keeps the **first row per department** according to the ORDER BY, i.e. the top earner in each department. It is a handy shortcut for "top 1 per group" (Pattern 12 in Session 3.11).

ORDER BY: sorting

```
SELECT emp_name, dept_id, salary
FROM employees
ORDER BY dept_id ASC NULLS FIRST, salary DESC
LIMIT 8;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_id	salary
Farhan Ali	NULL	55000.00
Arjun Mehta	1	150000.00
Priya Sharma	1	120000.00
Rahul Verma	1	95000.00
Sneha Iyer	1	95000.00
Vikram Rao	1	80000.00
Pooja Reddy	1	72000.00
Ananya Das	2	110000.00

8 rows

- `ASC` (default) or `DESC`, per column; later columns break ties.
- `NULLS FIRST` / `NULLS LAST` controls where NULLs go. In PostgreSQL, NULLs sort as if **larger** than every value: last in ascending order, first in descending order.
- Without `ORDER BY`, the row order is not guaranteed, even if it looks stable.

LIMIT and OFFSET: top-N and pagination

```
SELECT emp_id, emp_name
FROM employees
ORDER BY emp_id
LIMIT 5 OFFSET 5;      -- page 2 when each page has 5 rows
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name
6	Ananya Das
7	Karan Malhotra
8	Meera Nair
9	Rohan Gupta
10	Divya Menon

5 rows

- `LIMIT n` returns at most n rows; `OFFSET k` skips the first k rows. Page p of size n uses `OFFSET (p - 1) * n`.
- Always pair them with `ORDER BY`, otherwise "page 2" is undefined.
- The SQL-standard spelling is `OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY`. SQL Server uses `TOP`, Oracle uses `FETCH FIRST` (or `ROWNUM` in old versions).
- A large `OFFSET` (page 10,000) is slow, because the database still reads and throws away all skipped rows. Big apps use **keyset pagination**: `WHERE emp_id > :last_seen_id ORDER BY emp_id LIMIT 5`.

Comments and style

```
-- single-line comment
/* multi-line
   comment */
SELECT emp_name      -- one column per line is easy to read
FROM employees
WHERE dept_id = 1;    -- keywords in capitals is a common convention
```

SQL · POSTGRESQL

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A basic SQL query has SELECT for the columns I want, FROM for the table, WHERE to filter rows, ORDER BY to sort, and LIMIT to restrict the number of rows. I can compute expressions and name them with aliases. One subtle point is that the database logically runs FROM and WHERE before SELECT, so a column alias defined in SELECT can't be used in WHERE, but it can be used in ORDER BY. And without ORDER BY, the order of rows is never guaranteed."

COMMON TRAPS & MISTAKES

- Using a SELECT alias in WHERE: error, because WHERE runs first.
- Expecting a fixed row order without ORDER BY.
- LIMIT without ORDER BY for "top N": you get *some* N rows, not the top ones.
- Thinking DISTINCT works on one column when several are selected; it de-duplicates whole rows.
- Double quotes for strings: WHERE city = "Pune" looks for a *column* named Pune.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does DISTINCT do?

A It removes duplicate rows from the result, considering all selected columns together.

Q2 **HOW** How do you get the 5 highest-paid employees?

A SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5;

Q3 **DEFINITION** What is an alias?

A A temporary name for a column or table in a query, created with AS, e.g. salary * 12 AS annual_salary or FROM employees e.

Q4 **COMPARISON** CHAR vs VARCHAR vs TEXT in PostgreSQL?

A CHAR(n) is fixed length and pads with spaces; VARCHAR(n) is variable length with a maximum; TEXT is variable length without a limit. In PostgreSQL VARCHAR and TEXT perform the same.

INTERMEDIATE

Q5 **WHY** Why can't you use a column alias in the WHERE clause?

A WHERE is evaluated before SELECT in the logical processing order, so the alias doesn't exist yet. Repeat the expression or wrap the query in a subquery or CTE.

Q6 **HOW** How do you implement pagination in SQL?

A ORDER BY a unique column with LIMIT page_size OFFSET (page - 1) * page_size. For large tables, keyset pagination (WHERE id > last_id ORDER BY id LIMIT n) is faster.

Q7 **COMPARISON** **TIMESTAMP vs TIMESTAMPTZ?**

A **TIMESTAMP** stores a date and time without a time zone. **TIMESTAMPTZ** stores an absolute moment (internally in UTC) and converts it to the session's time zone on display, which is safer for global apps.

Q8 **HOW** **Where do NULLs appear when you sort in PostgreSQL?**

A Last in ascending order and first in descending order by default; you can override this with `NULLS FIRST` or `NULLS LAST`.

SCENARIO-BASED**Q9** **SQL PROBLEM** **List all distinct cities where employees earning more than 70000 work, alphabetically.**

A `SELECT DISTINCT city FROM employees WHERE salary > 70000 ORDER BY city;`

Q10 **SCENARIO** **Page 50,000 of a product listing is extremely slow. Why, and what would you do?**

A `OFFSET` makes the database read and discard all earlier rows. Switch to keyset (cursor) pagination using the last seen sort key, with an index on that key.

FOLLOW-UP / TRAP**Q11** **TRAP QUESTION** **Does `SELECT DISTINCT dept_id, city FROM employees` return unique departments?**

A No, it returns unique (dept_id, city) combinations, so a department appears once per distinct city.

Q12 **TRAP QUESTION** **Is `\dt` a SQL command?**

A No, it is a psql meta-command handled by the client. It works only in psql, not from an application.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Write a query that shows each product's name and price, most expensive first.

P2. Write a query that lists the distinct product categories.

P3. What does `LIMIT 3 OFFSET 6` return?

LEVEL 2 – INTERVIEW APPLICATION

P4. Show the name and monthly salary of employees in Bengaluru, plus a column `monthly_tax` equal to 20% of salary, highest salary first.

P5. Return page 3 of customers ordered by `customer_id`, with 2 customers per page.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P6. A query `SELECT emp_name, salary*12 AS yearly FROM employees WHERE yearly > 1200000;` fails. Fix it in two different ways.

P7. Return the three most recently hired employees, showing name, city and hire date.

ANSWER / EXPLANATION

P1.

```
SELECT product_name, price
FROM products
ORDER BY price DESC;
```

SQL · POSTGRESQL

OUTPUT

product_name	price
Standing Desk	15999.00
Office Chair	8999.00
Mechanical Keyboard	3499.00
Monitor Arm	2499.00
USB-C Hub	1999.00
Wireless Mouse	799.00
Notebook Pack	249.00
Gel Pen Set	149.00

8 rows

P2.

```
SELECT DISTINCT category
FROM products
ORDER BY category;
```

SQL · POSTGRESQL

OUTPUT

category
Electronics
Furniture
Stationery

3 rows

P3. It skips the first 6 rows of the (ordered) result and returns the next 3, i.e. rows 7, 8 and 9. Without ORDER BY, which rows those are is not defined.

P4.

```
SELECT emp_name,
       salary,
       salary * 0.20 AS monthly_tax
FROM employees
WHERE city = 'Bengaluru'
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary	monthly_tax
Arjun Mehta	150000.00	30000.0000
Priya Sharma	120000.00	24000.0000
Sneha Iyer	95000.00	19000.0000

3 rows

P5. Page 3 with 2 per page skips $(3 - 1) \times 2 = 4$ rows:

```
SELECT customer_id, customer_name
FROM customers
ORDER BY customer_id
LIMIT 2 OFFSET 4;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name
5	Vivaan Shah
6	Saanvi Rao

2 rows

P6. (a) Repeat the expression: `WHERE salary * 12 > 1200000`. (b) Compute it in a subquery first:

```
SELECT emp_name, yearly
FROM (SELECT emp_name, salary * 12 AS yearly FROM employees) AS t
WHERE yearly > 1200000;
```

SQL · POSTGRESQL

OUTPUT

emp_name	yearly
Arjun Mehta	1800000.00
Priya Sharma	1440000.00
Ananya Das	1320000.00

3 rows

P7.

```
SELECT emp_name, city, hire_date
FROM employees
ORDER BY hire_date DESC
LIMIT 3;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	hire_date
Farhan Ali	Pune	2024-06-03
Pooja Reddy	Hyderabad	2024-02-12
Divya Menon	Kolkata	2023-01-09

3 rows

Session 3.2 — PostgreSQL Basics (Part 2)

Combining conditions: AND, OR, NOT

- **AND** : both conditions must be true.
- **OR** : at least one must be true.
- **NOT** : reverses a condition.

Precedence trap: **AND** is evaluated before **OR**, just as multiplication comes before addition. "Employees in Pune or Mumbai earning more than 60000", written without parentheses:

```
SELECT emp_name, city, salary
FROM employees
WHERE city = 'Pune' OR city = 'Mumbai' AND salary > 60000
ORDER BY emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Ananya Das	Mumbai	110000.00
Farhan Ali	Pune	55000.00
Karan Malhotra	Mumbai	70000.00
Rahul Verma	Pune	95000.00

4 rows

Farhan (Pune, 55000) sneaked in, because the database read it as `city = 'Pune' OR (city = 'Mumbai' AND salary > 60000)`. With parentheses:

```
SELECT emp_name, city, salary
FROM employees
WHERE (city = 'Pune' OR city = 'Mumbai') AND salary > 60000
ORDER BY emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Ananya Das	Mumbai	110000.00
Karan Malhotra	Mumbai	70000.00
Rahul Verma	Pune	95000.00

3 rows

Rule: whenever you mix AND and OR, add parentheses.

IN, BETWEEN and LIKE

```
SELECT emp_name, city
FROM employees
WHERE city IN ('Pune', 'Kochi', 'Delhi')
ORDER BY city, emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city
Aditya Singh	Delhi
Neha Kapoor	Delhi
Meera Nair	Kochi
Farhan Ali	Pune
Rahul Verma	Pune

5 rows

`IN (list)` is shorthand for several ORs. `NOT IN` excludes a list (careful with NULLs, see Session 3.7).

```
SELECT emp_name, hire_date
FROM employees
WHERE hire_date BETWEEN '2019-01-01' AND '2020-12-31'
ORDER BY hire_date;
```

SQL · POSTGRESQL

OUTPUT

emp_name	hire_date
Rahul Verma	2019-01-10
Aditya Singh	2019-08-19
Sneha Iyer	2020-03-23
Karan Malhotra	2020-11-02

4 rows

`BETWEEN a AND b` is **inclusive** at both ends (`>= a AND <= b`).

BETWEEN WITH TIMESTAMPS

For a `TIMESTAMP` column, `BETWEEN '2024-04-01' AND '2024-04-30'` stops at **midnight at the start of 30 April**, so almost the whole last day is missed. The safe pattern is a half-open range: `login_time >= '2024-04-01' AND login_time < '2024-05-01'`.

`LIKE` matches text patterns: `%` means "any number of characters" (including none), `_` means "exactly one character".

Pattern	Matches
'A%'	starts with A
'%a'	ends with a
'%Rao%'	contains "Rao"
'_r%'	second letter is r
ILIKE 'a%'	starts with a or A (PostgreSQL's case-insensitive LIKE)

```
SELECT emp_name, email
FROM employees
WHERE emp_name LIKE '_t%'
      OR email ILIKE 'NEHA%';
```

SQL · POSTGRESQL

OUTPUT

emp_name	email
Arjun Mehta	arjun.mehta@company.com
Priya Sharma	priya.sharma@company.com
Neha Kapoor	neha.kapoor@company.com

3 rows

NULL: the value that means "unknown"

NULL means **missing or unknown**. It is not zero, not an empty string, and not false. This leads to **three-valued logic**: a condition can be TRUE, FALSE or **UNKNOWN**, and **WHERE** keeps only TRUE rows.

- **NULL = NULL** is not true; it is UNKNOWN. So **WHERE email = NULL** returns **nothing**:

```
SELECT emp_name FROM employees WHERE email = NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name

0 rows

- Use **IS NULL** / **IS NOT NULL** :

```
SELECT emp_name, email, dept_id
FROM employees
WHERE email IS NULL OR dept_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name	email	dept_id
Farhan Ali	NULL	NULL

1 row

Expression	Result
NULL + 100	NULL (arithmetic with NULL gives NULL)
'Hi ' NULL	NULL (concatenation with gives NULL)
CONCAT('Hi ', NULL)	'Hi ' (CONCAT ignores NULLs)
NULL = NULL	NULL (unknown), never TRUE
TRUE OR NULL	TRUE
FALSE AND NULL	FALSE

COALESCE returns its first non-NULL argument, which is perfect for defaults. **NULLIF(a, b)** returns NULL when a equals b, which is handy to avoid division by zero: **x / NULLIF(y, 0)**.

```

SELECT emp_name,
       COALESCE(email, 'not assigned') AS email,
       COALESCE(dept_id::text, 'no department') AS department
FROM employees
WHERE emp_id IN (12, 13);

```

SQL · POSTGRESQL

OUTPUT

emp_name	email	department
Neha Kapoor	neha.kapoor@company.com	4
Farhan Ali	not assigned	no department

2 rows

Aggregate functions over the whole table

Aggregates turn many rows into one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. They **ignore NULLs**, except `COUNT(*)`, which counts rows.

```

SELECT COUNT(*) AS all_rows,
       COUNT(email) AS with_email,
       COUNT(DISTINCT city) AS cities,
       SUM(salary) AS payroll,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(hire_date) AS first_hire,
       MAX(salary) AS top_salary
FROM employees;

```

SQL · POSTGRESQL

OUTPUT

all_rows	with_email	cities	payroll	avg_salary	first_hire	top_salary
14	13	7	1184000.00	84571.43	2015-04-01	150000.00

1 row

LINE BY LINE

Expression	Meaning here
<code>COUNT(*)</code>	number of rows: 14 employees
<code>COUNT(email)</code>	rows where email is not NULL : 13, because Farhan has no email
<code>COUNT(DISTINCT city)</code>	number of different cities
<code>SUM(salary)</code>	total monthly payroll
<code>ROUND(AVG(salary), 2)</code>	average, rounded to 2 decimals
<code>MIN(hire_date)</code>	MIN and MAX also work on dates and text

`AVG(column)` ignores NULLs too: the average of (100, NULL, 200) is 150, not 100. If NULL should count as zero, write `AVG(COALESCE(column, 0))`. Grouping (one result per department, etc.) comes in Session 3.5.

String functions

Function	Example	Result
UPPER, LOWER, INITCAP	INITCAP('raHUL verma')	Rahul Verma
LENGTH	LENGTH('Pune')	4
TRIM, LTRIM, RTRIM	TRIM(' Pune ')	Pune
SUBSTRING(s FROM start FOR n)	SUBSTRING('PostgreSQL' FROM 1 FOR 4)	Post
LEFT, RIGHT	RIGHT('ORD-9001', 4)	9001
POSITION(sub IN s) / STRPOS	POSITION('@' IN 'a@b.com')	2
REPLACE	REPLACE('2024/01/05', '/', '-')	2024-01-05
CONCAT,	'Mr. ' 'Rao'	Mr. Rao
SPLIT_PART(s, sep, n)	SPLIT_PART('a@mail.com', '@', 2)	mail.com
LPAD, RPAD	LPAD('42', 5, '0')	00042
STRING_AGG(col, sep) (aggregate)	names joined with commas	Asha, Ravi

```

SELECT emp_name,
       UPPER(emp_name)           AS shouting,
       SPLIT_PART(emp_name, ' ', 1) AS first_name,
       SPLIT_PART(email, '@', 2)  AS email_domain,
       LENGTH(emp_name)          AS name_length
FROM employees
WHERE dept_id = 2;

```

SQL · PostgreSQL

OUTPUT

emp_name	shouting	first_name	email_domain	name_length
Ananya Das	ANANYA DAS	Ananya	company.com	10
Karan Malhotra	KARAN MALHOTRA	Karan	company.com	14
Meera Nair	MEERA NAIR	Meera	company.com	10

3 rows

Date and time functions

Real queries use `CURRENT_DATE` or `NOW()`. To keep this book's outputs stable, the examples treat **31 December 2024** as "today".

Function	Example	Result
CURRENT_DATE, NOW()	today's date, current timestamp	depends on when you run it
EXTRACT(part FROM d)	EXTRACT(YEAR FROM DATE '2024-03-05')	2024
DATE_TRUNC('month', d)	DATE_TRUNC('month', TIMESTAMP '2024-03-05 10:00')	2024-03-01 00:00:00
date - date	DATE '2024-03-05' - DATE '2024-03-01'	4 (days, as an integer)
date + interval	DATE '2024-01-31' + INTERVAL '1 month'	2024-02-29 00:00:00
AGE(a, b)	AGE(DATE '2024-12-31', DATE '2015-04-01')	9 years 8 mons 30 days
TO_CHAR(d, fmt)	TO_CHAR(DATE '2024-03-05', 'DD Mon YYYY')	05 Mar 2024
TO_DATE(s, fmt) / cast	TO_DATE('05/03/2024', 'DD/MM/YYYY')	2024-03-05

```

SELECT emp_name,
       hire_date,
       EXTRACT(YEAR FROM hire_date)      AS hire_year,
       TO_CHAR(hire_date, 'Mon YYYY')    AS hired_in,
       DATE '2024-12-31' - hire_date     AS days_employed,
       AGE(DATE '2024-12-31', hire_date) AS tenure
FROM employees
WHERE emp_id IN (1, 10, 14);

```

SQL · POSTGRESQL

OUTPUT

emp_name	hire_date	hire_year	hired_in	days_employed	tenure
Arjun Mehta	2015-04-01	2015	Apr 2015	3562	9 years 8 mons 30 days
Divya Menon	2023-01-09	2023	Jan 2023	722	1 year 11 mons 22 days
Pooja Reddy	2024-02-12	2024	Feb 2024	323	10 mons 19 days

3 rows

Numeric functions and the integer-division trap

Function	Example	Result
ROUND(x, d)	ROUND(84571.4286, 2)	84571.43
CEIL, FLOOR	CEIL(4.1), FLOOR(4.9)	5, 4
TRUNC(x, d)	TRUNC(4.567, 1)	4.5
ABS, POWER, SQRT	ABS(-7), POWER(2, 10)	7, 1024
MOD(a, b) or a % b	7 % 3	1

```

SELECT 5 / 2      AS int_division,
       5 / 2.0    AS decimal_division,
       5::numeric / 2 AS cast_first,
       7 % 3      AS remainder;

```

SQL · POSTGRESQL

OUTPUT

int_division	decimal_division	cast_first	remainder
2	2.5000000000000000	2.5000000000000000	1

1 row

When **both** operands are integers, PostgreSQL does integer division and drops the fraction: $5 / 2 = 2$. This silently breaks percentage calculations like `delivered_count / total_count * 100`. Cast one side first: `delivered_count::numeric / total_count`.

`CAST(x AS type)` and the PostgreSQL shortcut `x::type` convert between types: `'2024-03-05'::date`, `salary::int`, `dept_id::text`.

CASE WHEN: if-then-else inside SQL

`CASE` evaluates conditions **in order** and returns the result of the first true one; `ELSE` covers everything else (without `ELSE`, the result is NULL).

SQL · POSTGRESQL

```

SELECT emp_name,
       salary,
       CASE
         WHEN salary >= 110000 THEN 'Band A'
         WHEN salary >= 80000  THEN 'Band B'
         WHEN salary >= 60000  THEN 'Band C'
         ELSE 'Band D'
       END AS salary_band
FROM employees
ORDER BY salary DESC;

```

OUTPUT

emp_name	salary	salary_band
Arjun Mehta	150000.00	Band A
Priya Sharma	120000.00	Band A
Ananya Das	110000.00	Band A
Sneha Iyer	95000.00	Band B
Rahul Verma	95000.00	Band B
Aditya Singh	90000.00	Band B
Vikram Rao	80000.00	Band B
Pooja Reddy	72000.00	Band C
Meera Nair	70000.00	Band C
Karan Malhotra	70000.00	Band C
Rohan Gupta	65000.00	Band C
Neha Kapoor	60000.00	Band C
Farhan Ali	55000.00	Band D
Divya Menon	52000.00	Band D

14 rows

LINE BY LINE

Line	What it does
CASE	start of the conditional expression
WHEN salary >= 110000 THEN 'Band A'	checked first; 150000, 120000 and 110000 stop here
WHEN salary >= 80000 THEN 'Band B'	only reached if the first test failed, so this means 80000–109999
ELSE 'Band D'	everything that matched no WHEN
END AS salary_band	closes the CASE and names the new column

CASE also works inside `ORDER BY` (custom sort orders) and inside aggregates, the basis of **conditional aggregation** (Pattern 18):

SQL · POSTGRESQL

```

SELECT COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered,
       SUM(CASE WHEN status = 'PENDING' THEN 1 ELSE 0 END) AS pending,
       COUNT(*) AS total
FROM orders;

```

OUTPUT

delivered	pending	total
8	2	14

1 row

`COUNT(*) FILTER (WHERE ...)` is PostgreSQL's neat syntax; `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` works in every database.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"NULL in SQL means unknown, not zero or empty, so comparisons with NULL return unknown, and WHERE only keeps rows where the condition is true. That's why we must write `IS NULL` instead of `= NULL`. Aggregates like SUM and AVG ignore NULLs, and COUNT of a column counts only non-null values, while `COUNT(*)` counts all rows. To replace NULLs with a default, I use COALESCE, and NULLIF helps avoid division by zero."

COMMON TRAPS & MISTAKES

- `WHERE col = NULL` returns nothing; use `IS NULL`.
- Mixing AND/OR without parentheses.
- `BETWEEN` on timestamps misses most of the last day; use `>= start AND < next_day`.
- `LIKE` is case-sensitive in PostgreSQL; use `ILIKE` or `LOWER(col) LIKE ...`.
- Integer division: `5/2 = 2`.
- `COUNT(column)` silently skips NULLs; `AVG` ignores NULLs instead of counting them as zero.
- `CASE` without `ELSE` returns NULL for unmatched rows.

INTERVIEW QUESTIONS

BASIC

Q1 **COMPARISON** What is the difference between `IN` and `BETWEEN`?

A `IN` matches any value in a list (`city IN ('Pune', 'Delhi')`); `BETWEEN` matches a continuous, inclusive range (`salary BETWEEN 50000 AND 90000`).

Q2 **DEFINITION** What do `%` and `_` mean in `LIKE`?

A `%` matches any sequence of characters (including none); `_` matches exactly one character.

Q3 **HOW** How do you find rows where a column has no value?

A `WHERE column IS NULL`. Using `= NULL` never matches.

Q4 **COMPARISON** `COUNT(*)` vs `COUNT(column)`?

A `COUNT(*)` counts all rows; `COUNT(column)` counts rows where that column is not NULL.

INTERMEDIATE

Q5 **DEFINITION** What is `COALESCE`?

A A function returning its first non-NULL argument, used to substitute defaults: `COALESCE(phone, 'N/A')`.

Q6 **WHY** What does `SELECT 7/2` return in PostgreSQL, and why?

A 3, because both operands are integers, so integer division truncates the fraction. Use `7/2.0` or `7::numeric/2` for 3.5.

Q7 **HOW** How would you extract the domain from an email address?

A `SPLIT_PART(email, '@', 2)`, or `SUBSTRING(email FROM POSITION('@' IN email) + 1)`.

Q8 **DEFINITION** What is three-valued logic?

A SQL conditions evaluate to TRUE, FALSE or UNKNOWN (when NULL is involved). WHERE keeps only TRUE, so UNKNOWN rows are filtered out.

Q9 HOW How do you count delivered and pending orders in one query?

A Conditional aggregation: `SUM(CASE WHEN status='DELIVERED' THEN 1 ELSE 0 END)` and the same for PENDING, or PostgreSQL's `COUNT(*) FILTER (WHERE status = 'DELIVERED')`.

SCENARIO-BASED**Q10 SQL PROBLEM Find employees whose name starts with "A" or "R" and who were hired after 2018.**

A `SELECT emp_name, hire_date FROM employees WHERE (emp_name LIKE 'A%' OR emp_name LIKE 'R%') AND hire_date >= '2019-01-01';` Note the parentheses.

Q11 SCENARIO A report divides `delivered_orders` / `total_orders` and always shows 0. Why?

A Both values are integers, so integer division truncates to 0. Cast one side to numeric and use `NULLIF(total_orders, 0)` to avoid division by zero.

Q12 SCENARIO An average rating looks too high because unrated items have NULL. What's happening?

A AVG ignores NULLs, so only rated items are averaged. That's usually correct; if unrated must count as 0, use `AVG(COALESCE(rating, 0))`, and state the business rule clearly.

FOLLOW-UP / TRAP**Q13 TRAP QUESTION Is `NULL = NULL` true?**

A No, it is UNKNOWN (NULL). Use `IS NULL`, or `IS NOT DISTINCT FROM` to compare two possibly-NULL values.

Q14 TRAP QUESTION Is an empty string the same as NULL?

A In PostgreSQL, no: `''` is a known, empty value, while NULL is unknown. (Oracle is the exception: it treats `''` as NULL.)

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. List products in the Furniture or Stationery category priced under 3000.

P2. Show employees with no manager.

P3. What does `COALESCE(NULL, NULL, 'x', 'y')` return?

LEVEL 2 – INTERVIEW APPLICATION

P4. Show each order's id, status and a column `is_open` that is 'Yes' for PENDING or SHIPPED orders and 'No' otherwise.

P5. Count customers who have an email and customers who don't, in one row.

P6. List employees hired in 2021 or later whose email domain is `company.com`, showing the year they joined.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Marketing wants leads whose email does not end with `mail.com`. Write it, and say what happens to rows whose email is NULL.

P8. Calculate the percentage of orders that are DELIVERED, rounded to one decimal place, avoiding integer division.

ANSWER / EXPLANATION

P1.

```
SELECT product_name, category, price
FROM products
WHERE category IN ('Furniture', 'Stationery')
      AND price < 3000
ORDER BY price;
```

SQL · POSTGRESQL

OUTPUT

product_name	category	price
Gel Pen Set	Stationery	149.00
Notebook Pack	Stationery	249.00
Monitor Arm	Furniture	2499.00

3 rows

P2.

```
SELECT emp_id, emp_name
FROM employees
WHERE manager_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name
1	Arjun Mehta

1 row

P3. 'x', the first non-NULL argument.

P4.

```
SELECT order_id,
       status,
       CASE WHEN status IN ('PENDING', 'SHIPPED') THEN 'Yes' ELSE 'No' END AS is_open
FROM orders
ORDER BY order_id;
```

SQL · POSTGRESQL

OUTPUT

order_id	status	is_open
101	DELIVERED	No
102	DELIVERED	No
103	DELIVERED	No
104	CANCELLED	No
105	DELIVERED	No
106	DELIVERED	No
108	SHIPPED	Yes
109	DELIVERED	No
110	DELIVERED	No
111	RETURNED	No
113	SHIPPED	Yes
114	PENDING	Yes
115	PENDING	Yes
116	DELIVERED	No

14 rows

P5. COUNT(email) skips NULLs, so the difference from COUNT(*) is the number without an email:

```
SELECT COUNT(email) AS with_email,
       COUNT(*) - COUNT(email) AS without_email
FROM customers;
```

SQL · POSTGRESQL

OUTPUT

with_email	without_email
6	1

1 row

P6.

```
SELECT emp_name,
       EXTRACT(YEAR FROM hire_date) AS joined_year
FROM employees
WHERE hire_date >= '2021-01-01'
      AND SPLIT_PART(email, '@', 2) = 'company.com'
ORDER BY hire_date;
```

SQL · POSTGRESQL

OUTPUT

emp_name	joined_year
Vikram Rao	2021
Neha Kapoor	2021
Meera Nair	2022
Divya Menon	2023
Pooja Reddy	2024

5 rows

Farhan (hired 2024) is missing: his email is NULL, so the domain test is UNKNOWN and the row is filtered out.

P7. `SELECT * FROM leads WHERE email NOT LIKE '%mail.com';` In this table every lead's email ends with `mail.com`, so it returns no rows. In general, rows with a NULL email would also be excluded, because `NULL NOT LIKE ...` is UNKNOWN. Add `OR email IS NULL` if they should be included.

P8.

```
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'DELIVERED') / COUNT(*), 1)
       AS delivered_pct
FROM orders;
```

SQL · POSTGRESQL

OUTPUT

delivered_pct
57.1

1 row

Multiplying by `100.0` (a decimal) first makes the whole calculation decimal, so nothing is truncated.

Session 3.3 — DDL: Data Definition Language

Simple meaning

DDL commands define and change the **structure of the database**: creating tables, adding columns, removing tables. They work on the containers, not on the data inside (DML does that, Session 3.4).

Command	What it does
CREATE	create a database object: database, schema, table, view, index, sequence
ALTER	change an existing object: add, drop or rename columns; change types; add constraints
DROP	delete an object completely, structure and data
TRUNCATE	remove all rows quickly but keep the table's structure

POSTGRESQL DETAIL THAT IMPRESSES INTERVIEWERS

In PostgreSQL, DDL is **transactional**: CREATE , ALTER and even DROP TABLE inside a transaction can be rolled back. In MySQL and Oracle most DDL commits automatically. You will see a live demonstration below.

CREATE TABLE

```
CREATE TABLE projects (
  project_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  project_name VARCHAR(80) NOT NULL,
  client VARCHAR(60) NOT NULL,
  dept_id INT REFERENCES departments(dept_id),
  budget NUMERIC(12,2) CONSTRAINT chk_budget_positive CHECK (budget > 0),
  start_date DATE NOT NULL DEFAULT DATE '2024-01-01',
  status VARCHAR(12) NOT NULL DEFAULT 'PLANNED'
);
```

SQL · POSTGRESQL

RESULT CREATE TABLE

LINE BY LINE

Line	What it means
project_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY	the database generates 1, 2, 3... automatically; it is the primary key
project_name VARCHAR(80) NOT NULL	text up to 80 characters, required
dept_id INT REFERENCES departments(dept_id)	foreign key; NULL allowed (a project may have no department yet)
CONSTRAINT chk_budget_positive CHECK (budget > 0)	a named constraint, so error messages and later changes are clearer
DEFAULT DATE '2024-01-01' / DEFAULT 'PLANNED'	values used when an INSERT does not supply them (normally you would write DEFAULT CURRENT_DATE)

SERIAL is the older PostgreSQL shortcut for auto-numbering; GENERATED ... AS IDENTITY is the SQL-standard way and is preferred today. MySQL uses AUTO_INCREMENT ; SQL Server uses IDENTITY(1,1) .

ALTER TABLE: changing structure

```
ALTER TABLE projects ADD COLUMN manager_id INT REFERENCES employees(emp_id);
ALTER TABLE projects RENAME COLUMN client TO client_name;
ALTER TABLE projects ALTER COLUMN project_name TYPE VARCHAR(120);
ALTER TABLE projects ADD CONSTRAINT uq_project_name UNIQUE (project_name);
ALTER TABLE projects ALTER COLUMN status SET DEFAULT 'ACTIVE';
```

SQL · POSTGRESQL

Statement	Effect
ADD COLUMN manager_id INT REFERENCES employees(emp_id)	new nullable column with a foreign key
RENAME COLUMN client TO client_name	renames the column; the data stays
ALTER COLUMN project_name TYPE VARCHAR(120)	widens the column (narrowing can fail if data is too long)
ADD CONSTRAINT uq_project_name UNIQUE (project_name)	adds a rule; fails if existing data already breaks it
ALTER COLUMN status SET DEFAULT 'ACTIVE'	new default for future inserts

Other common forms: DROP COLUMN , ALTER COLUMN ... SET NOT NULL / DROP NOT NULL , DROP CONSTRAINT , RENAME TO new_table_name .

You can inspect a table's structure through the standard information_schema views (psql users simply type \d projects):

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'projects'
ORDER BY ordinal_position;
```

SQL · POSTGRESQL

OUTPUT

column_name	data_type	is_nullable	column_default
project_id	integer	NO	NULL
project_name	character varying	NO	NULL
client_name	character varying	NO	NULL
dept_id	integer	YES	NULL
budget	numeric	YES	NULL
start_date	date	NO	'2024-01-01'::date
status	character varying	NO	'ACTIVE'::character varying
manager_id	integer	YES	NULL

8 rows

Creating a table from an existing table (CTAS)

"Create another table with the help of existing tables" appears in reported PwC interview write-ups. **CREATE TABLE AS SELECT** copies the *result of a query* into a new table:

```
CREATE TABLE high_earners AS
SELECT emp_id, emp_name, dept_id, salary
FROM employees
WHERE salary > 90000;

SELECT * FROM high_earners ORDER BY salary DESC;
```

SQL · PostgreSQL

OUTPUT

emp_id	emp_name	dept_id	salary
1	Arjun Mehta	1	150000.00
2	Priya Sharma	1	120000.00
6	Ananya Das	2	110000.00
3	Rahul Verma	1	95000.00
4	Sneha Iyer	1	95000.00

5 rows

- CTAS copies column names, types and data, but **not** primary keys, foreign keys, defaults or indexes. Add them with ALTER TABLE if needed.
- `CREATE TABLE copy_structure (LIKE employees INCLUDING ALL);` copies only the structure, including defaults, constraints and indexes, with no rows.
- `INSERT INTO existing_table SELECT ...` copies rows into a table that already exists.

DROP and TRUNCATE

```
DROP TABLE IF EXISTS high_earners;      -- no error if it doesn't exist
DROP TABLE departments CASCADE;         -- also drops dependent foreign keys and views
(dangerous!)
TRUNCATE TABLE leads;                   -- remove all rows, keep the table
TRUNCATE TABLE projects RESTART IDENTITY; -- also reset the auto-numbering
```

SQL · PostgreSQL

PostgreSQL refuses to truncate a table that other tables reference with foreign keys, because that would leave orphan rows:

```
TRUNCATE TABLE customers;
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR cannot truncate a table referenced in a foreign key constraint

DETAIL: Table "orders" references "customers".

HINT: Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.

`TRUNCATE customers CASCADE` would also truncate `orders`, `order_items` and `user_logins`, which is a good reason to read every CASCADE twice.

DDL inside a transaction (PostgreSQL)

Watch a dropped table come back:

SQL · POSTGRESQL

```
BEGIN;
DROP TABLE leads;
ROLLBACK;

SELECT COUNT(*) AS leads_still_here FROM leads;
```

OUTPUT

leads_still_here

7

1 row

DELETE vs TRUNCATE vs DROP

MUST KNOW This comparison is asked constantly.

	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	chosen rows (WHERE) or all rows	all rows	the whole table: rows, structure, indexes, constraints
WHERE allowed	yes	no	no
Speed on big tables	slower: row by row, each row logged	very fast: deallocates storage pages	fast
Triggers	row-level DELETE triggers fire	DELETE triggers do not fire	—
Auto-number (identity)	not reset	can be reset (RESTART IDENTITY)	gone with the table
Table still exists	yes	yes	no
Rollback	yes, inside a transaction	PostgreSQL: yes, inside a transaction; MySQL/Oracle: no (auto-commits)	PostgreSQL: yes, inside a transaction; most others: no
Use when	removing specific rows	emptying a staging table quickly	the table is no longer needed

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"DELETE is a DML command that removes rows one by one; it can take a WHERE clause, fires triggers and can be rolled back. TRUNCATE is DDL; it removes all rows at once by releasing the data pages, which makes it much faster on big tables, but there's no WHERE, row triggers don't fire, and it can reset identity columns. DROP removes the table itself, structure and all. In PostgreSQL both TRUNCATE and DROP can even be rolled back inside a transaction, whereas in MySQL or Oracle they auto-commit."

COMMON TRAPS & MISTAKES

- Saying "TRUNCATE can never be rolled back". True for MySQL and Oracle, but not for PostgreSQL inside a transaction.
- Saying "DELETE without WHERE is the same as TRUNCATE". Same final rows, but different speed, logging, triggers and identity behaviour.
- Assuming CTAS copies constraints and indexes. It copies only columns and data.
- Using DROP ... CASCADE casually; it silently removes dependent objects.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is DDL? Give examples.

A Data Definition Language defines database structure: CREATE, ALTER, DROP and TRUNCATE (some also include RENAME and COMMENT).

Q2 **COMPARISON** DELETE vs TRUNCATE vs DROP?

A DELETE removes selected or all rows (DML, WHERE allowed, triggers fire, slower). TRUNCATE removes all rows quickly but keeps the table (DDL, no WHERE). DROP removes the entire table, structure included.

Q3 **HOW** How do you add a column to an existing table?

A `ALTER TABLE employees ADD COLUMN phone VARCHAR(15);`

INTERMEDIATE

Q4 **HOW** How do you create a new table from an existing one?

A `CREATE TABLE new_table AS SELECT ... FROM old_table WHERE ...;` It copies columns and data but not keys, defaults or indexes. Use `CREATE TABLE t (LIKE old INCLUDING ALL)` to copy only the structure.

Q5 **COMPARISON** SERIAL vs IDENTITY in PostgreSQL?

A Both auto-generate numbers through a sequence. IDENTITY is the SQL-standard syntax, with clearer ownership and permissions; SERIAL is an older PostgreSQL shortcut.

Q6 **WHY** Why would TRUNCATE fail on a table?

A If other tables reference it with foreign keys. PostgreSQL requires those tables to be truncated too, or `TRUNCATE ... CASCADE`.

Q7 **WHY** Why name constraints explicitly?

A Error messages become readable (`chk_budget_positive`), and it's easier to drop or alter a constraint later.

SCENARIO-BASED

Q8 **SCENARIO** You need to empty a 50-million-row staging table every night before reloading it. DELETE or TRUNCATE?

A TRUNCATE. It's dramatically faster and generates far less log. DELETE would process and log every row.

Q9 **SCENARIO** You must change a column from VARCHAR(20) to VARCHAR(10) on a live table. What do you check first?

A Whether existing data fits: `SELECT MAX(LENGTH(col)) FROM t;` . Narrowing fails if values are too long. Also consider locks on a big table and whether applications expect the old length.

FOLLOW-UP / TRAP

Q10 **TRAP QUESTION** Can you roll back a DROP TABLE?

A In PostgreSQL, yes, if it was run inside an explicit transaction that hasn't been committed. In MySQL and Oracle, DDL auto-commits, so no (outside of features like Oracle's recycle bin).

Q11 **TRAP QUESTION** Does TRUNCATE fire DELETE triggers?

A No. Row-level DELETE triggers do not fire (PostgreSQL has separate TRUNCATE triggers).

Session 3.4 — DML: Data Manipulation Language (plus DCL and TCL)

Simple meaning

DML commands change the data inside tables: `INSERT` adds rows, `UPDATE` changes rows, `DELETE` removes rows. (`SELECT` is sometimes counted as DML too, and sometimes as DQL.)

INSERT

```
INSERT INTO departments (dept_id, dept_name, location)
VALUES (6, 'Data Science', 'Bengaluru'),
       (7, 'Operations', 'Chennai')
RETURNING *;
```

SQL · PostgreSQL

OUTPUT

dept_id	dept_name	location
6	Data Science	Bengaluru
7	Operations	Chennai

2 rows

- Always list the columns: the statement keeps working if the table gets new columns, and it documents which value goes where.
- Several rows can be inserted in one statement.
- `RETURNING` (PostgreSQL) gives back the inserted rows, including generated IDs and defaults, without another query.

INSERT ... SELECT copies rows from a query:

```
INSERT INTO projects (project_name, client_name, dept_id, budget)
SELECT 'Audit automation for ' || dept_name, 'Internal', dept_id, 500000
FROM departments
WHERE location IN ('Mumbai', 'Kolkata')
RETURNING project_id, project_name, budget, status;
```

SQL · PostgreSQL

OUTPUT

project_id	project_name	budget	status
1	Audit automation for Finance	500000.00	ACTIVE
2	Audit automation for HR	500000.00	ACTIVE

2 rows

`project_id` came from the identity column and `status` from the column default ('ACTIVE', set by the ALTER in Session 3.3).

UPSERT: insert, or update if it already exists

A very common need: "add this product; if its ID already exists, update the price instead." PostgreSQL uses `ON CONFLICT`:

```

INSERT INTO products (product_id, product_name, category, price)
VALUES (1, 'Wireless Mouse', 'Electronics', 749.00),
      (9, 'Laptop Stand', 'Furniture', 1299.00)
ON CONFLICT (product_id)
DO UPDATE SET price = EXCLUDED.price
RETURNING product_id, product_name, price;

```

SQL · POSTGRESQL

OUTPUT

product_id	product_name	price
1	Wireless Mouse	749.00
9	Laptop Stand	1299.00

2 rows

LINE BY LINE

Line	What it means
VALUES (1, ..., 749.00), (9, ..., 1299.00)	product 1 already exists; product 9 is new
ON CONFLICT (product_id)	if inserting would violate the unique/primary key on product_id...
DO UPDATE SET price = EXCLUDED.price	...update the existing row instead; EXCLUDED means "the row we tried to insert"
RETURNING ...	shows the final state of both rows: one updated, one inserted

ON CONFLICT ... DO NOTHING simply skips duplicates. MySQL writes ON DUPLICATE KEY UPDATE ; the SQL standard (and PostgreSQL 15+, SQL Server, Oracle) also offers MERGE .

UPDATE

```

UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 3
RETURNING emp_id, emp_name, salary;

```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name	salary
9	Rohan Gupta	71500.00
10	Divya Menon	57200.00

2 rows

- SET can change several columns: SET salary = 70000, city = 'Pune' .
- The new value can be an expression using the old one (salary * 1.10).
- **Without WHERE , every row is updated.** That is the most famous production accident in SQL.

Updating with data from another table (PostgreSQL's UPDATE ... FROM): mark every pending order of customers from Ahmedabad as SHIPPED.

```

UPDATE orders o
SET status = 'SHIPPED'
FROM customers c
WHERE c.customer_id = o.customer_id
      AND c.city = 'Ahmedabad'
      AND o.status = 'PENDING'
RETURNING o.order_id, c.customer_name, o.status;

```

SQL · POSTGRESQL

OUTPUT

order_id	customer_name	status
114	Vivaan Shah	SHIPPED

1 row

The same can be written with a subquery: `WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'Ahmedabad')`.

DELETE

```

DELETE FROM leads
WHERE created_at < '2024-01-05'
RETURNING lead_id, full_name, created_at;

```

SQL · POSTGRESQL

OUTPUT

lead_id	full_name	created_at
1	Amit Kumar	2024-01-02
2	Sara Thomas	2024-01-03

2 rows

- `DELETE ... USING other_table` deletes based on a join, like `UPDATE ... FROM`.
- Deleting a parent row that children reference fails unless the foreign key says `ON DELETE CASCADE` or `SET NULL` (Session 2.3).

SAFE HABITS FOR UPDATE AND DELETE

1. Write the `SELECT` with the same `WHERE` first and check the rows it returns.
2. Run the change inside `BEGIN; ...` and check the row count ("UPDATE 3") before `COMMIT`.
3. Use `RETURNING` to see exactly what changed.
4. In production, prefer a **soft delete** (an `is_deleted` flag or `deleted_at` timestamp) when you may need the data back or for audit.

DCL: Data Control Language (permissions)

DCL decides **who may do what**.

```

CREATE ROLE reporting_user LOGIN PASSWORD 'change-me';
GRANT SELECT ON orders, customers TO reporting_user; -- read-only access to two tables
GRANT INSERT, UPDATE ON orders TO app_user;
REVOKE UPDATE ON orders FROM app_user; -- take a privilege back

```

SQL · POSTGRESQL

- Follow the **principle of least privilege**: each user or application gets only the permissions it needs. A reporting tool needs `SELECT`, never `DROP`.
- Privileges are usually granted to **roles**, and users are made members of roles.
- PostgreSQL also supports **row-level security** (users see only their own rows).

TCL: Transaction Control Language

TCL groups statements into **transactions** that succeed or fail as a unit. Full details, including ACID, come in Session 3.10.

Command	Meaning
BEGIN (or START TRANSACTION)	start a transaction
COMMIT	make all changes permanent
ROLLBACK	undo all changes since BEGIN
SAVEPOINT name	a checkpoint inside a transaction
ROLLBACK TO SAVEPOINT name	undo back to the checkpoint only

By default PostgreSQL runs in **autocommit** mode: each statement outside an explicit `BEGIN` is its own transaction and commits immediately.

SQL · POSTGRES SQL

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 99; -- wrong account: 0 rows
ROLLBACK TO SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
COMMIT;

SELECT * FROM accounts ORDER BY account_id;
```

OUTPUT

account_id	holder_name	balance
1	Aarav Patel	49000.00
2	Isha Kulkarni	21000.00
3	Kabir Khan	5000.00

3 rows

LINE BY LINE

Line	What happens
BEGIN;	start a transaction; nothing is permanent yet
UPDATE ... - 1000 WHERE account_id = 1;	debit Aarav
SAVEPOINT after_debit;	remember this point
UPDATE ... WHERE account_id = 99;	a mistake: no such account, 0 rows changed
ROLLBACK TO SAVEPOINT after_debit;	undo only what happened after the savepoint (the debit is kept)
UPDATE ... + 1000 WHERE account_id = 2;	credit the right account (Isha)
COMMIT;	both changes become permanent together

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"DML is the part of SQL that changes data: INSERT adds rows, UPDATE modifies them and DELETE removes them. In PostgreSQL I often use RETURNING to get back generated IDs or the changed rows, and INSERT ... ON CONFLICT for upserts. The biggest risk is an UPDATE or DELETE without a WHERE clause, so I first run a SELECT with the same condition and wrap the change in a transaction. DCL handles permissions with GRANT and REVOKE, and TCL controls transactions with BEGIN, COMMIT, ROLLBACK and SAVEPOINT."

COMMON TRAPS & MISTAKES

- UPDATE or DELETE without WHERE changes every row.
- Inserting without a column list: it breaks as soon as the table changes.
- Forgetting COMMIT after BEGIN : the changes are invisible to others and are lost when the session ends.
- Confusing ROLLBACK TO SAVEPOINT (partial undo) with ROLLBACK (undo everything).
- Granting broad rights (ALL PRIVILEGES) to application or reporting users.

INTERVIEW QUESTIONS**BASIC****Q1 DEFINITION What is DML?**

A Data Manipulation Language: the commands that change data, namely INSERT, UPDATE and DELETE (SELECT is sometimes included).

Q2 COMPARISON DDL vs DML?

A DDL defines structure (CREATE, ALTER, DROP, TRUNCATE); DML changes the data inside the structure (INSERT, UPDATE, DELETE).

Q3 DEFINITION What are DCL and TCL?

A DCL controls permissions (GRANT, REVOKE). TCL controls transactions (BEGIN, COMMIT, ROLLBACK, SAVEPOINT).

INTERMEDIATE**Q4 DEFINITION What is an upsert and how is it written in PostgreSQL?**

A Insert a row, or update it if a row with the same key exists: `INSERT ... ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col;` (`DO NOTHING` to skip instead).

Q5 HOW How do you update a table using values from another table?

A PostgreSQL: `UPDATE t SET ... FROM other o WHERE o.id = t.id AND ...;` Portable: a subquery in SET or WHERE. SQL Server uses `UPDATE ... FROM ... JOIN;` MySQL uses `UPDATE t JOIN o ON ... SET ...`.

Q6 DEFINITION What does RETURNING do?

A It returns the rows affected by INSERT, UPDATE or DELETE (for example generated IDs), saving an extra SELECT.

Q7 DEFINITION What is a savepoint?

A A named checkpoint inside a transaction; `ROLLBACK TO SAVEPOINT` undoes changes after it while keeping earlier work.

SCENARIO-BASED

Q8 **SCENARIO** Someone ran `UPDATE employees SET salary = 0;` in production without a **WHERE** clause. How could it have been prevented, and how do you recover?

A Prevention: run the `SELECT` first, use explicit transactions, restrict permissions, require code review for production SQL. Recovery: if the transaction isn't committed, `ROLLBACK`; otherwise restore from a backup or point-in-time recovery, or use audit/history tables.

Q9 **SCENARIO** A nightly job inserts customers from a file; some already exist. How do you avoid duplicates?

A Use an upsert keyed on a unique business key (email or customer code): `ON CONFLICT (email) DO UPDATE` (or `DO NOTHING`), with a `UNIQUE` constraint so the database enforces it.

Q10 **DESIGN QUESTION** What permissions would you give a BI dashboard's database user?

A A read-only role with `SELECT` on only the reporting tables or views it needs (ideally a reporting schema or read replica); no `INSERT`, `UPDATE`, `DELETE` or `DDL` rights.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Is `TRUNCATE DML`?

A No, it is `DDL`, even though it removes data.

Q12 **TRAP QUESTION** What happens if you run `BEGIN`, an `UPDATE`, and then close the session without `COMMIT`?

A The transaction is rolled back; the update is lost.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Write the SQL to add a nullable `phone` column (up to 15 characters) to `customers`.

P2. Which command removes all rows but keeps the table: `DELETE`, `TRUNCATE` or `DROP`? Which one can use `WHERE`?

P3. Insert a new customer, id 8, "Kiara Menon", kiara@mail.com, Kochi, signed up 1 May 2024.

LEVEL 2 – INTERVIEW APPLICATION

P4. Give every employee in Engineering (dept 1) a 5% raise and return their new salaries.

P5. Create a table `dept_payroll` from existing data, holding each department's name and total salary.

P6. Write an upsert that sets the price of product 2 to 3299, inserting the product if it does not exist.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Delete all leads whose email appears more than once, but only the copies created *after* the first one. (Hint: compare each lead with an earlier lead that has the same email.)

P8. A colleague wants to empty `orders` with `TRUNCATE` before a reload. What will happen in PostgreSQL, and what are the options?

ANSWER / EXPLANATION

P1. `ALTER TABLE customers ADD COLUMN phone VARCHAR(15);`

P2. `TRUNCATE` removes all rows and keeps the table. Only `DELETE` accepts a `WHERE` clause.

P3.

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Kiara Menon', 'kiara@mail.com', 'Kochi', '2024-05-01')
RETURNING *;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name	email	city	signup_date
8	Kiara Menon	kiara@mail.com	Kochi	2024-05-01

1 row

P4.

```
UPDATE employees
SET salary = ROUND(salary * 1.05, 2)
WHERE dept_id = 1
RETURNING emp_name, salary;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary
Arjun Mehta	157500.00
Priya Sharma	126000.00
Rahul Verma	99750.00
Sneha Iyer	99750.00
Vikram Rao	84000.00
Pooja Reddy	75600.00

6 rows

P5. A CTAS with a join and GROUP BY (grouping is covered in Session 3.5):

```
CREATE TABLE dept_payroll AS
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM dept_payroll ORDER BY total_salary DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	total_salary
Engineering	612000.00
Finance	250000.00
Marketing	150000.00
HR	117000.00

4 rows

P6.

```
INSERT INTO products (product_id, product_name, category, price)
VALUES (2, 'Mechanical Keyboard', 'Electronics', 3299.00)
ON CONFLICT (product_id) DO UPDATE SET price = EXCLUDED.price
RETURNING *;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name	category	price
2	Mechanical Keyboard	Electronics	3299.00

1 row

P7. A lead is a later duplicate if another lead with the same email has a smaller `lead_id` :

```
DELETE FROM leads l
USING leads earlier
WHERE earlier.email = l.email
      AND earlier.lead_id < l.lead_id
RETURNING l.lead_id, l.full_name, l.email;
```

SQL · POSTGRESQL

OUTPUT

lead_id	full_name	email
3	Amit Kumar	amit@mail.com
5	Sara Thomas	sara@mail.com
6	Amit K.	amit@mail.com

3 rows

Leads 3 and 6 (later copies of amit@mail.com) and 5 (a later copy of sara@mail.com) are removed; the first lead for each email stays. Session 3.11 (Pattern 6) shows the window-function version.

P8. It fails, because `order_items` and other tables reference `orders` through foreign keys. Options: `TRUNCATE orders, order_items;` (truncate the related tables together), `TRUNCATE orders CASCADE` (truncates every referencing table, so be careful), or load into a separate staging table and swap.

Session 3.5 — Grouping and Aggregation: GROUP BY and HAVING

HIGH PRIORITY

MUST KNOW

Simple meaning

GROUP BY puts rows that share a value into the same group, then an aggregate function produces one result per group. "How many employees per department?" "Revenue per month?" "Average salary per city?" are all GROUP BY questions.

SORTING EXAM PAPERS

A teacher has a pile of answer sheets from three classes. She sorts them into one pile per class (**GROUP BY class**), then counts each pile and finds its highest mark (**COUNT, MAX**). She ends with one line per class, not one line per student.

Your first GROUP BY

```
SELECT dept_id,
       COUNT(*)           AS employees,
       ROUND(AVG(salary), 2) AS avg_salary,
       MAX(salary)        AS max_salary
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

SQL · POSTGRES SQL

OUTPUT

dept_id	employees	avg_salary	max_salary
1	6	102000.00	150000.00
2	3	83333.33	110000.00
3	2	58500.00	65000.00
4	2	75000.00	90000.00
NULL	1	55000.00	55000.00

5 rows

LINE BY LINE

Line	What it does
SELECT dept_id,	one output row per department, so show which department it is
COUNT(*) AS employees	how many rows (employees) fell into each group
ROUND(AVG(salary), 2) AS avg_salary	average salary of the group, two decimals
MAX(salary) AS max_salary	highest salary in the group
GROUP BY dept_id	make one group per distinct dept_id value
ORDER BY dept_id	sort the final result

Notice the last row: Farhan's dept_id is NULL, and **all NULLs form one group of their own**.

The golden rule of GROUP BY

Every column in the SELECT list must either be **in the GROUP BY** or **inside an aggregate function**. Otherwise the database cannot know which of the group's many values to show:

```
SELECT dept_id, emp_name, COUNT(*)
FROM employees
GROUP BY dept_id;
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR column "employees.emp_name" must appear in the GROUP BY clause or be used in an aggregate function

Engineering has six employees; which one name should appear on its single row? The database refuses to guess. (MySQL in its old, non-strict mode *did* guess, which hid bugs. Do not rely on that.)

A POSTGRESQL CONVENIENCE

If you group by a table's **primary key**, PostgreSQL lets you select that table's other columns without listing them, because the key functionally determines them: `SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) FROM departments d LEFT JOIN employees e ... GROUP BY d.dept_id;` is valid.

Grouping by several columns, and grouping joined data

```
SELECT d.dept_name,
       e.city,
       COUNT(*) AS headcount
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name, e.city
ORDER BY d.dept_name, headcount DESC;
```

SQL · PostgreSQL

OUTPUT

dept_name	city	headcount
Engineering	Bengaluru	3
Engineering	Hyderabad	2
Engineering	Pune	1
Finance	Mumbai	2
Finance	Kochi	1
HR	Kolkata	2
Marketing	Delhi	2

7 rows

One row per **combination** of department and city. Joins are covered properly in Session 3.6.

HAVING: filtering groups

`WHERE` filters **rows before** grouping. `HAVING` filters **groups after** aggregation, so it can use aggregates. "Departments with more than two employees":

```
SELECT dept_id, COUNT(*) AS employees
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2
ORDER BY employees DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_id	employees
1	6
2	3

2 rows

Using both: "Among employees hired before 2022, which cities have an average salary above 80000?"

```
SELECT city,
       COUNT(*) AS employees,
       ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE hire_date < '2022-01-01'      -- row filter: runs first
GROUP BY city
HAVING AVG(salary) > 80000          -- group filter: runs after grouping
ORDER BY avg_salary DESC;
```

SQL · POSTGRESQL

OUTPUT

city	employees	avg_salary
Bengaluru	3	121666.67
Pune	1	95000.00
Mumbai	2	90000.00

3 rows

	WHERE	HAVING
Filters	individual rows	groups
Runs	before GROUP BY	after GROUP BY
Can use aggregates (COUNT , SUM ...)	no	yes
Works without GROUP BY	yes	yes (the whole result is one group), but rarely useful
Performance tip	filter as early as possible here	only for conditions that need aggregates

The logical order of execution

SQL is *written* in one order but *processed* in another. Knowing the processing order explains most beginner errors:

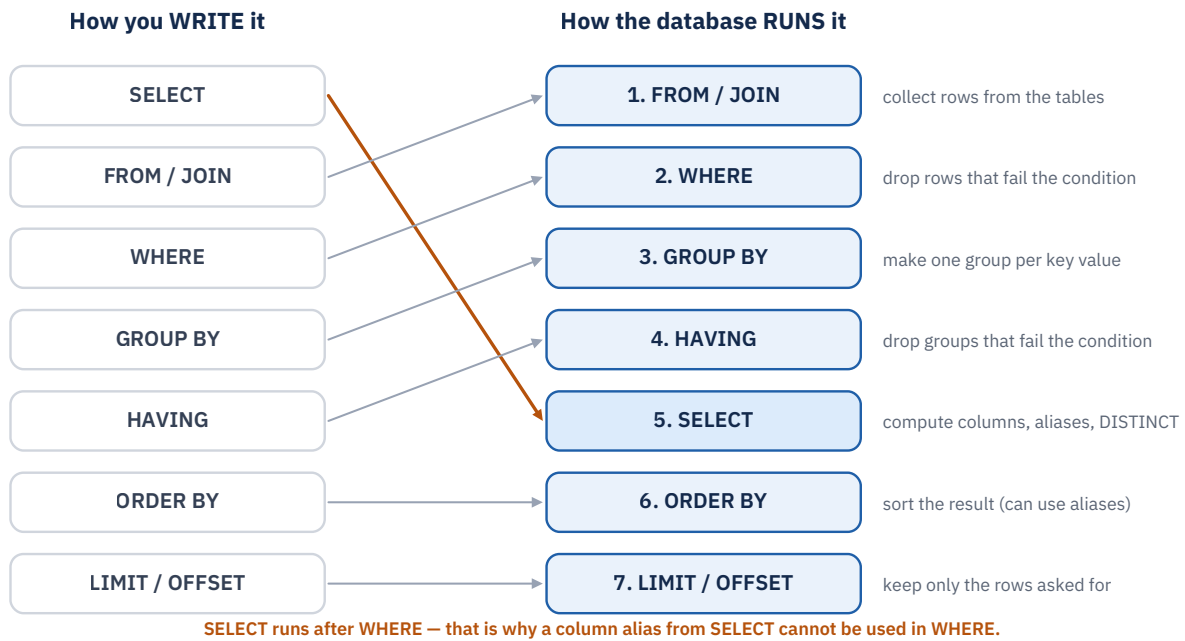


Figure 15. Written order vs logical processing order. SELECT is evaluated late.

This order explains:

- **Why WHERE cannot use SELECT aliases** (WHERE runs before SELECT).
- **Why WHERE cannot use aggregates** (groups do not exist yet).
- **Why ORDER BY can use aliases** (it runs after SELECT).
- **Why HAVING cannot use a SELECT alias in PostgreSQL** (HAVING runs before SELECT):

```
SELECT dept_id, COUNT(*) AS n
FROM employees
GROUP BY dept_id
HAVING n > 2;
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR column "n" does not exist

Write `HAVING COUNT(*) > 2` instead. (MySQL accepts aliases in HAVING; PostgreSQL and SQL Server do not.) PostgreSQL *does* accept output-column aliases in `GROUP BY` as a convenience, but the portable habit is to repeat the expression.

THE PHYSICAL PLAN CAN DIFFER

This is the *logical* order: it defines what the result must be. The optimizer may physically do things differently (for example, push a filter into an index scan), as long as the result is the same.

Listing the members of each group

`STRING_AGG` joins the group's values into one string; `ARRAY_AGG` builds an array.

SQL · POSTGRES SQL

```

SELECT d.dept_name,
       COUNT(e.emp_id) AS employees,
       STRING_AGG(e.emp_name, ', ' ORDER BY e.emp_name) AS members
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC;

```

OUTPUT

dept_name	employees	members
Engineering	6	Arjun Mehta, Pooja Reddy, Priya Sharma, Rahul Verma, Sneha Iyer, Vikram Rao
Finance	3	Ananya Das, Karan Malhotra, Meera Nair
HR	2	Divya Menon, Rohan Gupta
Marketing	2	Aditya Singh, Neha Kapoor

4 rows

Subtotals with ROLLUP INTERVIEW EXTENSION

GROUP BY ROLLUP (a, b) adds subtotal rows per a and a grand total. CUBE adds totals for every combination; GROUPING SETS lets you list exactly which groupings you want.

SQL · POSTGRES SQL

```

SELECT COALESCE(p.category, 'ALL CATEGORIES') AS category,
       SUM(i.quantity * i.unit_price) AS revenue
FROM order_items i
JOIN products p ON p.product_id = i.product_id
GROUP BY ROLLUP (p.category)
ORDER BY revenue;

```

OUTPUT

category	revenue
Stationery	6966.00
Electronics	26786.00
Furniture	83993.00
ALL CATEGORIES	117745.00

4 rows

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"GROUP BY splits rows into groups that share a value and lets aggregate functions like COUNT, SUM and AVG return one row per group, for example the number of employees per department. Every selected column must be grouped or aggregated. WHERE filters rows before grouping, so it can't use aggregates; HAVING filters groups after aggregation, for example departments with more than two employees. Logically the database runs FROM, WHERE, GROUP BY, HAVING, then SELECT and ORDER BY, which is also why aliases work in ORDER BY but not in WHERE."

COMMON TRAPS & MISTAKES

- Putting an aggregate in WHERE (WHERE COUNT(*) > 2): it must go in HAVING.
- Selecting a column that is neither grouped nor aggregated.
- Using HAVING for row-level filters that belong in WHERE (works, but it is slower and less clear).
- Forgetting that NULLs form their own group.
- Using a SELECT alias in HAVING in PostgreSQL.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does GROUP BY do?

A It groups rows with the same values in the given columns so aggregate functions return one result per group.

Q2 **COMPARISON** WHERE vs HAVING?

A WHERE filters rows before grouping and cannot use aggregates; HAVING filters groups after grouping and can use aggregates.

Q3 **SQL PROBLEM** Count employees in each city.

A `SELECT city, COUNT(*) FROM employees GROUP BY city;`

INTERMEDIATE

Q4 **WHY** Why does `SELECT dept_id, emp_name, COUNT(*) FROM employees GROUP BY dept_id` fail?

A emp_name is neither grouped nor aggregated, so there is no single value to show per department.

Q5 **HOW** What is the logical order of SQL clauses?

A FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET.

Q6 **HOW** Can you use HAVING without GROUP BY?

A Yes, then the whole table is one group: `SELECT COUNT(*) FROM employees HAVING COUNT(*) > 10` returns the count only if it exceeds 10. It's rarely needed.

Q7 **HOW** How are NULLs treated in GROUP BY?

A All NULLs are placed together into one group.

Q8 **DEFINITION** What does ROLLUP do?

A It adds subtotal and grand-total rows to a GROUP BY result, e.g. revenue per category plus an overall total.

SCENARIO-BASED

Q9 **SQL PROBLEM** Find cities with at least two employees and an average salary above 70000.

A `SELECT city, COUNT(*), AVG(salary) FROM employees GROUP BY city HAVING COUNT(*) >= 2 AND AVG(salary) > 70000;`

Q10 **SQL PROBLEM** Total order value per customer for delivered orders only, top spender first.

A `SELECT customer_id, SUM(total_amount) AS spent FROM orders WHERE status = 'DELIVERED' GROUP BY customer_id ORDER BY spent DESC;`

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Is `WHERE salary > 50000` the same as `HAVING salary > 50000` in a grouped query?

A No. HAVING can only refer to grouped columns or aggregates, so the HAVING version errors unless salary is grouped. Row conditions belong in WHERE.

Q12 **TRAP QUESTION** Does `COUNT(*)` in a grouped query ever return 0?

A Not for a group produced by GROUP BY, since every group has at least one row. Zero counts appear only with outer joins and `COUNT(column)` (see the next session).

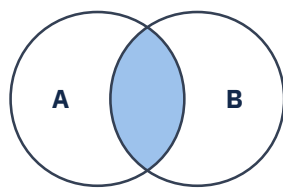
Session 3.6 – Joins

HIGH PRIORITY **MUST KNOW** Joins and their types are among the questions most often mentioned in PwC interview write-ups.

Why joins exist

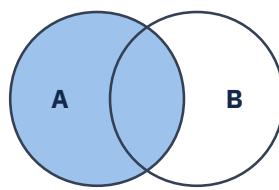
Normalization (Session 2.5) splits data into tables: employee names in one, department names in another. A **join** combines rows from two tables **based on a related column**, usually a foreign key matching a primary key, so you can see the pieces together again.

The join types at a glance



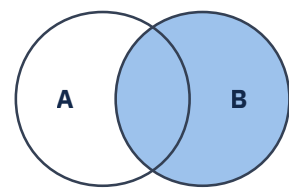
INNER JOIN

only rows that match in both



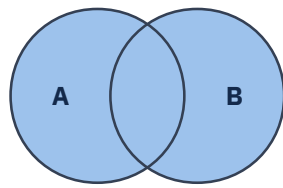
LEFT JOIN

all of A + matching rows of B



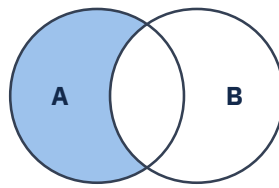
RIGHT JOIN

all of B + matching rows of A



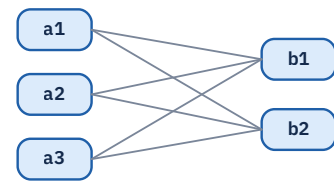
FULL OUTER JOIN

everything from A and B



LEFT JOIN ... WHERE B.key IS NULL

rows of A with NO match (anti-join)



CROSS JOIN

every A row × every B row ($3 \times 2 = 6$)

Figure 16. What each join returns. A is the left table (written first), B is the right table.

Join	Returns	Typical question
INNER JOIN	only rows with a match in both tables	"employees with their department names"
LEFT [OUTER] JOIN	all rows from the left table + matches from the right (NULLs when none)	"all customers, with orders if any"
RIGHT [OUTER] JOIN	all rows from the right table + matches from the left	same as LEFT with the tables swapped
FULL [OUTER] JOIN	all rows from both; NULLs where there is no match	"reconcile two lists: what's missing on either side?"
CROSS JOIN	every combination of rows (Cartesian product)	"every store × every day"
Self join	a table joined to itself	"employee and their manager"

INNER JOIN

```
SELECT e.emp_name, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_name, e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name	location
Arjun Mehta	Engineering	Bengaluru
Pooja Reddy	Engineering	Bengaluru
Priya Sharma	Engineering	Bengaluru
Rahul Verma	Engineering	Bengaluru
Sneha Iyer	Engineering	Bengaluru
Vikram Rao	Engineering	Bengaluru
Ananya Das	Finance	Mumbai
Karan Malhotra	Finance	Mumbai
Meera Nair	Finance	Mumbai
Divya Menon	HR	Kolkata
Rohan Gupta	HR	Kolkata
Aditya Singh	Marketing	Delhi
Neha Kapoor	Marketing	Delhi

13 rows

LINE BY LINE

Line	What it does
FROM employees e	the left table, aliased e
INNER JOIN departments d	the right table, aliased d (INNER is optional: plain JOIN means inner join)
ON d.dept_id = e.dept_id	the join condition: pair rows whose department IDs are equal
SELECT e.emp_name, d.dept_name, d.location	columns from both tables, prefixed by alias to avoid ambiguity

Two rows are missing on purpose: **Farhan** (no department, NULL never equals anything) and the **Legal** department (no employees). An inner join keeps only matches.

LEFT JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_name NULLS FIRST, e.emp_name
LIMIT 5;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name
Farhan Ali	NULL
Arjun Mehta	Engineering
Pooja Reddy	Engineering
Priya Sharma	Engineering
Rahul Verma	Engineering

5 rows

Every employee appears; Farhan's department columns are NULL. A LEFT JOIN answers "all of A, plus B where it exists".

Counting with a LEFT JOIN, the classic trap: employees per department, *including* departments with nobody.

```
SELECT d.dept_name,
       COUNT(*)      AS count_star_wrong,
       COUNT(e.emp_id) AS employees
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	count_star_wrong	employees
Engineering	6	6
Finance	3	3
HR	2	2
Marketing	2	2
Legal	1	0

5 rows

Legal shows **1** with `COUNT(*)`, because the left join produced one row for Legal (with NULL employee columns), and `COUNT(*)` counts rows. `COUNT(e.emp_id)` counts only real employees: **0**. Always count a column from the right-hand table.

RIGHT JOIN

A RIGHT JOIN keeps every row of the *right* table. It is exactly a LEFT JOIN with the tables written in the opposite order, so many teams use only LEFT JOIN for readability.

```
SELECT d.dept_name, e.emp_name
FROM employees e
RIGHT JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

dept_name	emp_name
Legal	NULL

1 row

This finds departments with no employees: Legal.

FULL OUTER JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name
Farhan Ali	NULL
NULL	Legal

2 rows

Both kinds of unmatched rows in one query: an employee without a department *and* a department without employees. FULL OUTER JOIN is great for **reconciliation**, such as comparing yesterday's file with today's. (MySQL has no FULL JOIN; emulate it with a LEFT JOIN UNION a RIGHT JOIN.)

CROSS JOIN

```
SELECT r.region, s.size
FROM (VALUES ('North'), ('South')) AS r(region)
CROSS JOIN (VALUES ('S'), ('M'), ('L')) AS s(size)
ORDER BY r.region, s.size;
```

SQL · POSTGRESQL

OUTPUT

region	size
North	L
North	M
North	S
South	L
South	M
South	S

6 rows

2 rows × 3 rows = 6 rows. Use it deliberately for **all combinations** (every product × every month for a sales grid). Accidentally forgetting a join condition with the old comma syntax (FROM a, b) creates a cross join, which can explode into millions of rows.

SELF JOIN

A self join treats one table as two, using two aliases. Employees and their managers live in the same table:

```
SELECT e.emp_name AS employee,
       m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY m.emp_name NULLS FIRST, e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

employee	manager
Arjun Mehta	NULL
Neha Kapoor	Aditya Singh
Karan Malhotra	Ananya Das
Meera Nair	Ananya Das
Aditya Singh	Arjun Mehta
Ananya Das	Arjun Mehta
Farhan Ali	Arjun Mehta
Priya Sharma	Arjun Mehta
Rohan Gupta	Arjun Mehta
Rahul Verma	Priya Sharma
Sneha Iyer	Priya Sharma
Vikram Rao	Priya Sharma
Pooja Reddy	Rahul Verma
Divya Menon	Rohan Gupta

14 rows

LINE BY LINE

Line	What it does
FROM employees e	the table in its "employee" role
LEFT JOIN employees m	the same table again in its "manager" role
ON m.emp_id = e.manager_id	an employee's manager_id points at the manager's emp_id
LEFT	keeps Arjun, who has no manager (an INNER join would drop him)

Self joins also find pairs, such as employees who work in the same city. The condition `e1.emp_id < e2.emp_id` avoids pairing someone with themselves and listing each pair twice:

```
SELECT e1.emp_name, e2.emp_name, e1.city
FROM employees e1
JOIN employees e2
  ON e1.city = e2.city
 AND e1.emp_id < e2.emp_id
WHERE e1.city IN ('Pune', 'Mumbai')
ORDER BY e1.city;
```

SQL · POSTGRESQL

OUTPUT

emp_name	emp_name	city
Ananya Das	Karan Malhotra	Mumbai
Rahul Verma	Farhan Ali	Pune

2 rows

Anti-join: rows with no match

"Customers who have never ordered" is one of the most common interview questions. A LEFT JOIN plus `IS NULL` on the right table's key:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name
6	Saanvi Rao
7	Reyansh Bose

2 rows

The equivalent `NOT EXISTS` form (often the clearest) appears in Session 3.7.

Joining many tables

Revenue per customer per product category needs four tables. Join them one step at a time, following the foreign keys:

```
SELECT c.customer_name,
       p.category,
       SUM(i.quantity * i.unit_price) AS revenue
FROM customers c
JOIN orders o    ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id   = o.order_id
JOIN products p  ON p.product_id  = i.product_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.customer_name, p.category
ORDER BY c.customer_name, revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

customer_name	category	revenue
Aarav Patel	Furniture	33997.00
Aarav Patel	Electronics	7096.00
Aarav Patel	Stationery	1245.00
Diya Joshi	Electronics	6796.00
Isha Kulkarni	Furniture	17998.00
Isha Kulkarni	Electronics	3499.00
Isha Kulkarni	Stationery	2486.00
Kabir Khan	Furniture	15999.00
Kabir Khan	Electronics	2397.00
Vivaan Shah	Electronics	6998.00
Vivaan Shah	Stationery	3235.00

11 rows

Join pitfalls that break real reports

1. Fan-out (row multiplication). Joining a one-to-many relationship repeats the "one" side once per match. Summing a column from the "one" side then double counts:

```
SELECT c.customer_name,
       SUM(o.total_amount) AS inflated_total    -- WRONG
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id    = o.order_id
GROUP BY c.customer_name
ORDER BY c.customer_name;
```

OUTPUT

customer_name	inflated_total
Aarav Patel	75677.00
Diya Joshi	9594.00
Isha Kulkarni	26469.00
Kabir Khan	34395.00
Vivaan Shah	13468.00

5 rows

Aarav's real total is 42338, but each order total was repeated once per order line. Fix it by not joining tables you do not need, or by aggregating each side *before* joining:

```
SELECT c.customer_name,
       SUM(o.total_amount) AS correct_total
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY c.customer_name;
```

OUTPUT

customer_name	correct_total
Aarav Patel	42338.00
Diya Joshi	6796.00
Isha Kulkarni	23983.00
Kabir Khan	34395.00
Vivaan Shah	10233.00

5 rows

2. A WHERE clause that silently turns a LEFT JOIN into an INNER JOIN. "All departments with their employees earning more than 60000":

```
SELECT d.dept_name, e.emp_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.salary > 60000
ORDER BY d.dept_name
LIMIT 4;
```

OUTPUT

dept_name	emp_name
Engineering	Priya Sharma
Engineering	Rahul Verma
Engineering	Sneha Iyer
Engineering	Arjun Mehta

4 rows

Legal disappeared: for Legal, `e.salary` is NULL, `NULL > 60000` is UNKNOWN, and WHERE drops the row. Put conditions on the right table **inside the ON clause** to keep all left rows:

```
SELECT d.dept_name, COUNT(e.emp_id) AS well_paid
FROM departments d
LEFT JOIN employees e
    ON e.dept_id = d.dept_id
    AND e.salary > 60000
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

SQL · POSTGRESQL

OUTPUT

dept_name	well_paid
Engineering	6
Finance	3
HR	1
Legal	0
Marketing	1

5 rows

3. NULLs never match. `NULL = NULL` is not true, so rows with NULL join keys never pair up (Farhan above).

4. Ambiguous columns. If both tables have `dept_id`, writing just `dept_id` fails ("column reference is ambiguous"). Always prefix with aliases.

Referential integrity and joins

Because `orders.customer_id` is a NOT NULL foreign key, every order is guaranteed to have a matching customer, so `orders JOIN customers` never loses an order. Without the foreign key, orphan orders would silently vanish from inner-join reports. Constraints protect your query results, not just your inserts.

INNER JOIN vs LEFT JOIN

	INNER JOIN	LEFT JOIN
Rows kept	only matching pairs	all left rows, matched or not
Unmatched left rows	dropped	kept, with NULLs for right-side columns
Typical use	"orders with their customer"	"all customers, with orders if any"; finding missing matches
Row count	≤ the matching combinations	≥ the number of left rows
Counting	<code>COUNT(*)</code> is fine	count a right-table column, e.g. <code>COUNT(o.order_id)</code>

Two more syntax notes

- `JOIN ... USING (dept_id)` is shorthand when both columns have the same name; the column appears once in the output.
- Old comma syntax: `FROM employees e, departments d WHERE e.dept_id = d.dept_id` is an inner join written the 1980s way. Explicit `JOIN ... ON` is clearer and avoids accidental cross joins.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A join combines rows from two tables based on a related column, usually a foreign key and a primary key. An inner join returns only rows that match in both tables. A left join returns every row from the left table plus matching rows from the right, with NULLs where there's no match; that's how I find, for example, customers with no orders, by checking that the order key is NULL. Right join is the mirror image, full outer join keeps unmatched rows from both sides, cross join gives every combination, and a self join joins a table to itself, like employees to their managers. Two things I watch for are duplicate rows when joining one-to-many before summing, and WHERE conditions on the right table that turn a left join into an inner join."

COMMON TRAPS & MISTAKES

- COUNT(*) with a LEFT JOIN counts unmatched rows as 1; count a right-table column instead.
- Filtering the right table in WHERE after a LEFT JOIN removes the unmatched rows; move the condition into ON.
- Summing a "one-side" column after joining a "many" table (fan-out) inflates totals.
- Saying "RIGHT JOIN is different from LEFT JOIN". It is the same operation with the tables swapped.
- Forgetting that NULL keys never match.
- Saying "a self join needs a special keyword". It is a normal join with two aliases.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is a join?

A An operation that combines rows from two or more tables based on a related column, usually a foreign key matching a primary key.

Q2 **COMPARISON** INNER JOIN vs LEFT JOIN?

A INNER returns only rows with matches in both tables. LEFT returns all rows from the left table, with NULLs for right-side columns where there is no match.

Q3 **DEFINITION** What is a self join? Give an example.

A Joining a table to itself using two aliases, e.g. employees joined to employees to show each employee's manager.

Q4 **DEFINITION** What is a CROSS JOIN?

A The Cartesian product: every row of one table combined with every row of the other (m × n rows).

INTERMEDIATE**Q5** **SQL PROBLEM** Find customers who have never placed an order.

A `SELECT c.* FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL; or with NOT EXISTS.`

Q6 **COMPARISON** What is the difference between putting a condition in ON vs WHERE for a LEFT JOIN?

A In ON, it controls which right rows match while all left rows are kept. In WHERE, it filters after the join and removes left rows whose right side is NULL, turning it into an inner join.

Q7 **WHY** Why might a SUM be too high after joining several tables?

A One-to-many joins duplicate the "one" side's rows (fan-out), so its values are summed several times. Aggregate before joining or remove unneeded joins.

Q8 DEFINITION What does a FULL OUTER JOIN return, and when is it useful?

A All rows from both tables, matched where possible and with NULLs otherwise; useful for reconciliation, such as finding records missing on either side.

Q9 HOW If table A has 4 rows and B has 3, how many rows can each join return?

A CROSS: exactly 12. INNER: 0 to 12, depending on matches. LEFT: at least 4 (up to 12). FULL: at least $\max(4, 3)$, at most 12 (or 7 if nothing matches).

SCENARIO-BASED**Q10 SQL PROBLEM List each department with its number of employees, including departments with none.**

A `SELECT d.dept_name, COUNT(e.emp_id) FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id GROUP BY d.dept_name;`

Q11 SQL PROBLEM Show each employee with their manager's name; employees without a manager should still appear.

A `SELECT e.emp_name, m.emp_name AS manager FROM employees e LEFT JOIN employees m ON m.emp_id = e.manager_id;`

Q12 SCENARIO Yesterday's and today's product files are loaded into two tables. How do you find added, removed and changed products?

A FULL OUTER JOIN on product_id. Rows with only the new side are added, only the old side are removed, and both sides with different values (e.g. price) are changed.

FOLLOW-UP / TRAP**Q13 TRAP QUESTION Is there any difference between JOIN and INNER JOIN ?**

A No. JOIN defaults to an inner join.

Q14 TRAP QUESTION Can a join condition use something other than equality?

A Yes, any condition: ranges (`o.order_date BETWEEN p.start AND p.end`), inequalities (`e1.emp_id < e2.emp_id`) and so on. These are called non-equi joins.

Q15 TRAP QUESTION Two employees have NULL dept_id . Will an inner join on dept_id pair them with each other?

A No. NULL never equals NULL, so NULL keys never match in a join.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Show each order's id, date and customer name.

P2. Count orders per status.

P3. Which join keeps all rows from both tables?

LEVEL 2 – INTERVIEW APPLICATION

P4. List products that have never been ordered.

P5. Show each department's name, number of employees and total salary, only for departments with total salary above 150000.

P6. For every employee, show their name, department name (or 'Unassigned') and manager's name (or 'None').

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Find the product categories each customer has bought from, as one comma-separated list per customer (exclude cancelled orders).

P8. Find pairs of employees in the same department where the first earns more than the second, showing both names and the salary difference. Limit the output to Finance.

ANSWER / EXPLANATION**P1.**

```
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ORDER BY o.order_id
LIMIT 6;
```

SQL · POSTGRESQL

OUTPUT

order_id	order_date	customer_name
101	2024-01-05	Aarav Patel
102	2024-01-12	Isha Kulkarni
103	2024-01-28	Aarav Patel
104	2024-02-02	Kabir Khan
105	2024-02-14	Diya Joshi
106	2024-02-20	Isha Kulkarni

6 rows

P2.

```
SELECT status, COUNT(*) AS orders
FROM orders
GROUP BY status
ORDER BY orders DESC;
```

SQL · POSTGRESQL

OUTPUT

status	orders
DELIVERED	8
SHIPPED	2
PENDING	2
CANCELLED	1
RETURNED	1

5 rows

P3. FULL OUTER JOIN.**P4.** Anti-join from products to order_items:

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items i ON i.product_id = p.product_id
WHERE i.product_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name
8	Monitor Arm

1 row

P5.

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS employees,
       SUM(e.salary)   AS total_salary
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING SUM(e.salary) > 150000
ORDER BY total_salary DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	employees	total_salary
Engineering	6	612000.00
Finance	3	250000.00

2 rows

P6. Two LEFT JOINs: one to departments, one self join to managers:

```
SELECT e.emp_name,
       COALESCE(d.dept_name, 'Unassigned') AS department,
       COALESCE(m.emp_name, 'None')       AS manager
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
```

SQL · POSTGRESQL

OUTPUT

emp_name	department	manager
Arjun Mehta	Engineering	None
Priya Sharma	Engineering	Arjun Mehta
Rahul Verma	Engineering	Priya Sharma
Sneha Iyer	Engineering	Priya Sharma
Vikram Rao	Engineering	Priya Sharma
Ananya Das	Finance	Arjun Mehta
Karan Malhotra	Finance	Ananya Das
Meera Nair	Finance	Ananya Das
Rohan Gupta	HR	Arjun Mehta
Divya Menon	HR	Rohan Gupta
Aditya Singh	Marketing	Arjun Mehta
Neha Kapoor	Marketing	Aditya Singh
Farhan Ali	Unassigned	Arjun Mehta
Pooja Reddy	Engineering	Rahul Verma

14 rows

P7.

SQL · PostgreSQL

```
SELECT c.customer_name,  
       STRING_AGG(DISTINCT p.category, ', ' ORDER BY p.category) AS categories  
FROM customers c  
JOIN orders o      ON o.customer_id = c.customer_id  
JOIN order_items i ON i.order_id    = o.order_id  
JOIN products p    ON p.product_id  = i.product_id  
WHERE o.status <> 'CANCELLED'  
GROUP BY c.customer_name  
ORDER BY c.customer_name;
```

OUTPUT

customer_name	categories
Aarav Patel	Electronics, Furniture, Stationery
Diya Joshi	Electronics
Isha Kulkarni	Electronics, Furniture, Stationery
Kabir Khan	Electronics, Furniture
Vivaan Shah	Electronics, Stationery

5 rows

DISTINCT inside STRING_AGG removes repeated categories.

P8.

SQL · PostgreSQL

```
SELECT a.emp_name AS higher_paid,  
       b.emp_name AS lower_paid,  
       a.salary - b.salary AS difference  
FROM employees a  
JOIN employees b  
  ON a.dept_id = b.dept_id  
 AND a.salary > b.salary  
WHERE a.dept_id = 2  
ORDER BY difference DESC;
```

OUTPUT

higher_paid	lower_paid	difference
Ananya Das	Karan Malhotra	40000.00
Ananya Das	Meera Nair	40000.00

2 rows

Karan and Meera both earn 70000, so neither appears as "higher paid" than the other: the strict > excludes equal salaries.